Unit 4

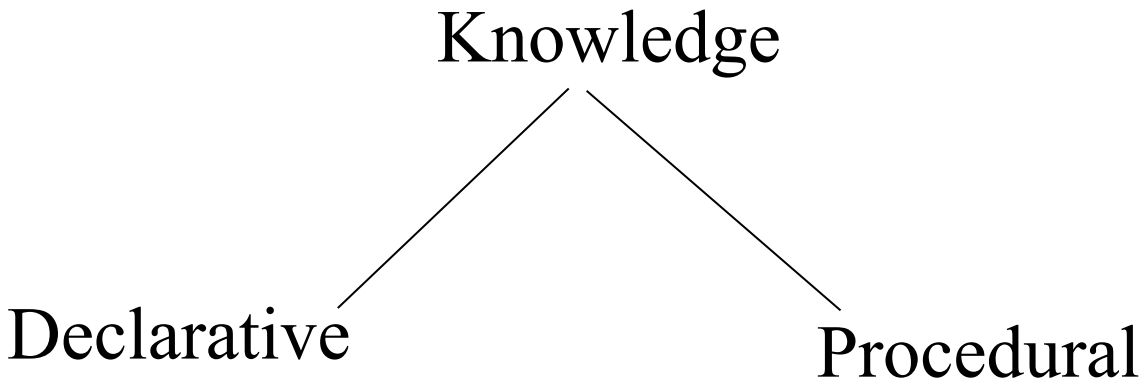
Knowledge and Reasoning
Table of Contents

- Knowledge and reasoning-Approaches and issues of knowledge reasoning-Knowledge base agents
- Logic Basics-Logic-Propositional logic-syntax ,semantics and inferences-Propositional logic- Reasoning patterns
- Unification and Resolution-Knowledge representation using rules-Knowledge representation using semantic nets
- Knowledge representation using frames-Inferences-
- Uncertain Knowledge and reasoning-Methods-Bayesian probability and belief network
- Probabilistic reasoning-Probabilistic reasoning over time-Probabilistic reasoning over time
- Other uncertain techniques-Data mining-Fuzzy logic-Dempster -shafer theory

Knowledge Representation & Reasoning

- The second most important concept in AI
- If we are going to act rationally in our environment, then we must have some way of describing that environment and drawing inferences from that representation.
  - how do we describe what we know about the world ?
  - how do we describe it *concisely* ?
  - how do we describe it so that we can get hold of the right piece of knowledge when we need it ?
  - how do we generate new pieces of knowledge ?
  - how do we deal with *uncertain* knowledge ?

Knowledge Representation & Reasoning



- Declarative knowledge deals with factoid questions (what is the capital of India? Etc.)
- Procedural knowledge deals with "How"
- Procedural knowledge can be embedded in declarative knowledge

Planning

Given a set of goals, construct a sequence of actions that achieves those goals:

- often very large search space
- but most parts of the world are independent of most other parts
- often start with goals and connect them to actions
- no necessary connection between order of planning and order of execution
- what happens if the world changes as we execute the plan and/or our actions don't produce the expected results?

Learning

- If a system is going to act truly appropriately, then it must be able to change its actions in the light of experience:
  - how do we generate new facts from old ?
  - how do we generate new concepts ?
  - how do we learn to distinguish different situations in new environments ?

What is knowledge representation?

- Knowledge representation and reasoning (KR, KRR) is the part of Artificial intelligence which concerned with AI agents thinking and how thinking contributes to intelligent behavior of agents.
- It is responsible for representing information about the real world so that a computer can understand and can utilize this knowledge to solve the complex real world problems such as diagnosis a medical condition or communicating with humans in natural language.
- It is also a way which describes how we can represent knowledge in artificial intelligence. Knowledge representation is not just storing data into some database, but it also enables an intelligent machine to learn from that knowledge and experiences so that it can behave intelligently like a human.

What to Represent?

Following are the kind of knowledge which needs to be represented in AI systems:

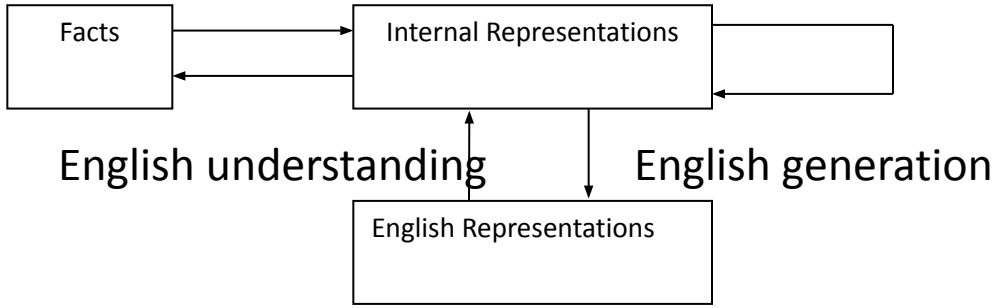
- **Object:** All the facts about objects in our world domain. E.g., Guitars contains strings, trumpets are brass instruments.
- **Events:** Events are the actions which occur in our world.
- **Performance:** It describe behavior which involves knowledge about how to do things.
- **Meta-knowledge:** It is knowledge about what we know.
- **Facts:** Facts are the truths about the real world and what we represent.
- **Knowledge-Base:** The central component of the knowledge-based agents is the knowledge base. It is represented as KB. The Knowledgebase is a group of the Sentences (Here, sentences are used as a technical term and not identical with the English language).

Approaches to knowledge Representation

- Representational adequacy the ability to represent all of the kinds of knowledge that are needed in that domain.
- Inferential Adequacy: - the ability to manipulate the representation structures in such a way as to derive new structures corresponding to new knowledge inferred from ol.
- Inferential Efficiency: - the ability to incorporate into the knowledge structure additional information that can be used to focus the attention of the inference mechanism in the most promising directions.
- Acquisitioned Efficiency: - the ability to acquire new information easily. The simplest case involves direct insertion by a person of new knowledge into the database.

Knowledge Representation Issues

- It becomes clear that particular knowledge representation models allow for more specific more powerful problem solving mechanisms that operate on them.
- Examine specific techniques that can be used for representing & manipulating knowledge within programs.
- **Representation & Mapping**
- Facts :- truths in some relevant world
- These are the things we want to represent.
- Representations of facts in some chosen formalism.
- Things we are actually manipulating. Structuring these entities is as two levels.
- The knowledge level, at which facts concluding each agents behavior & current goals are described.



Knowledge and Reasoning
Table of Contents

- Knowledge and reasoning-Approaches and issues of knowledge reasoning-Knowledge base agents
- Logic Basics-Logic-Propositional logic-syntax ,semantics and inferences-Propositional logic-Reasoning patterns
- Unification and Resolution-Knowledge representation using rules-Knowledge representation using semantic nets
- Knowledge representation using frames-Inferences-
- Uncertain Knowledge and reasoning-Methods-Bayesian probability and belief network
- Probabilistic reasoning-Probabilistic reasoning over time-Probabilistic reasoning over time
- Other uncertain techniques-Data mining-Fuzzy logic-Dempster -shafer theory

A KNOWLEDGE-BASED AGENT

- A knowledge-based agent includes a knowledge base and an inference system.
- A knowledge base is a set of representations of facts of the world.
- Each individual representation is called a **sentence**.
- The sentences are expressed in a **knowledge representation language**.
- The agent operates as follows:
  1. It TELLs the knowledge base what it perceives.
  2. It ASKs the knowledge base what action it should perform.
  3. It performs the chosen action.

Requirements for a Knowledge-Based Agent

1. \what it already knows" [McCarthy '59]

A knowledge base of beliefs.

2. \it must rst be capable of being told" [McCarthy '59]

A way to put new beliefs into the knowledge base.

3. \automatically deduces for itself a suciently wide class of immediate consequences" [McCarthy '59]

A reasoning mechanism to derive new beliefs from ones already in the knowledge base.

ARCHITECTURE OF A KNOWLEDGE-BASED AGENT

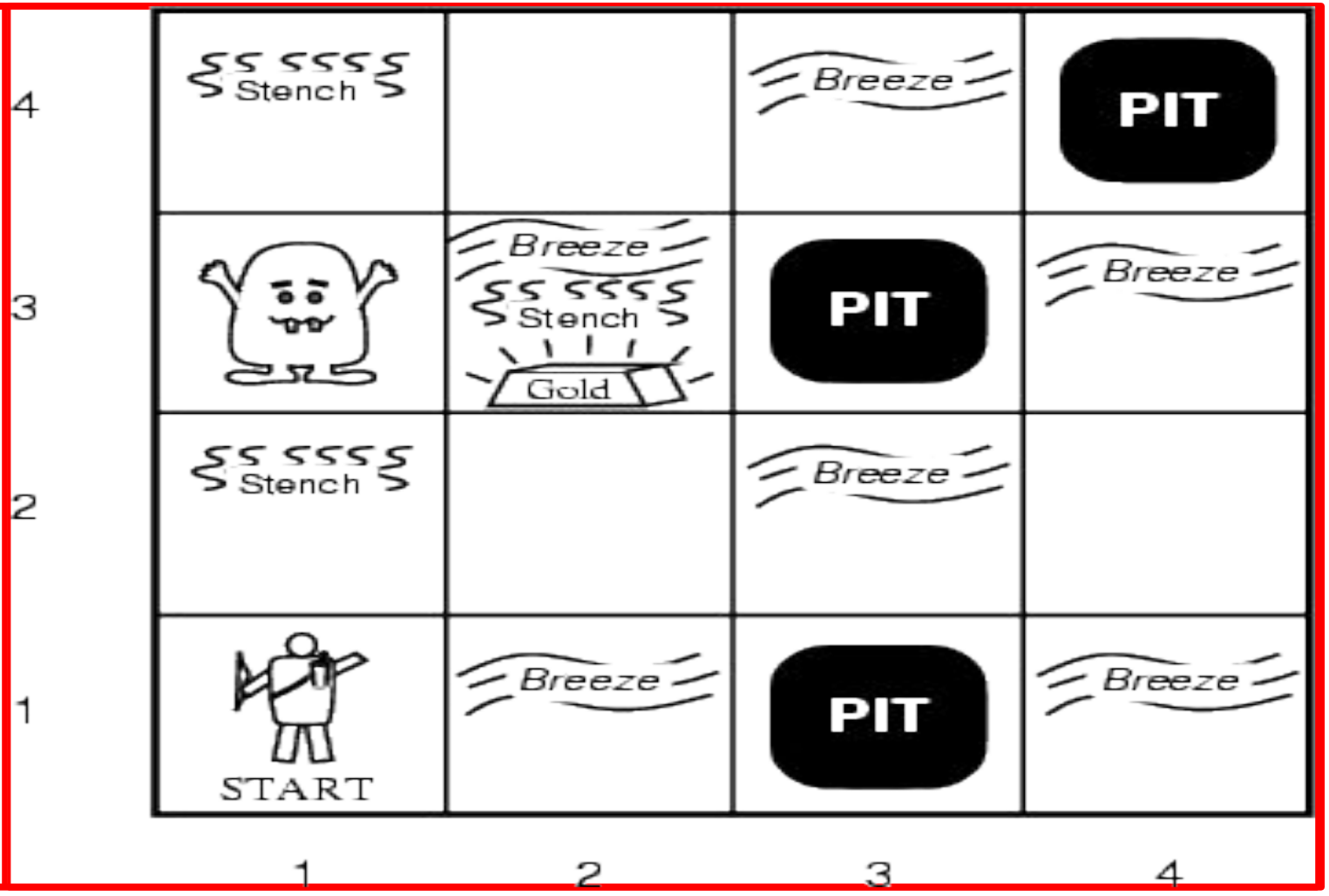
- **Knowledge Level.**
  - The most abstract level: describe agent by saying what it knows.
  - Example: A taxi agent might know that the Golden Gate Bridge connects San Francisco with the Marin County.
- **Logical Level.**
  - The level at which the knowledge is encoded into sentences.
  - Example: Links(GoldenGateBridge, SanFrancisco, MarinCounty).
- **Implementation Level.**
  - The physical representation of the sentences in the logical level.
  - Example: ‘(links goldengatebridge sanfrancisco marincounty)

THE WUMPUS WORLD ENVIRONMENT

- The Wumpus computer game
- The agent explores a cave consisting of rooms connected by passageways.
- Lurking somewhere in the cave is the Wumpus, a beast that eats any agent that enters its room.
- Some rooms contain bottomless pits that trap any agent that wanders into the room.
- Occasionally, there is a heap of gold in a room.
- The goal is to collect the gold and exit the world without being eaten

A TYPICAL WUMPUS WORLD

- The agent always starts in the field [1,1].
- The task of the agent is to find the gold, return to the field [1,1] and climb out of the cave.



AGENT IN A WUMPUS WORLD: PERCEPTS

- The agent perceives
  - a stench in the square containing the Wumpus and in the adjacent squares (not diagonally)
  - a breeze in the squares adjacent to a pit
  - a glitter in the square where the gold is
  - a bump, if it walks into a wall
  - a woeful scream everywhere in the cave, if the wumpus is killed
- The percepts are given as a five-symbol list. If there is a stench and a breeze, but no glitter, no bump, and no scream, the percept is

  [Stench, Breeze, None, None, None]

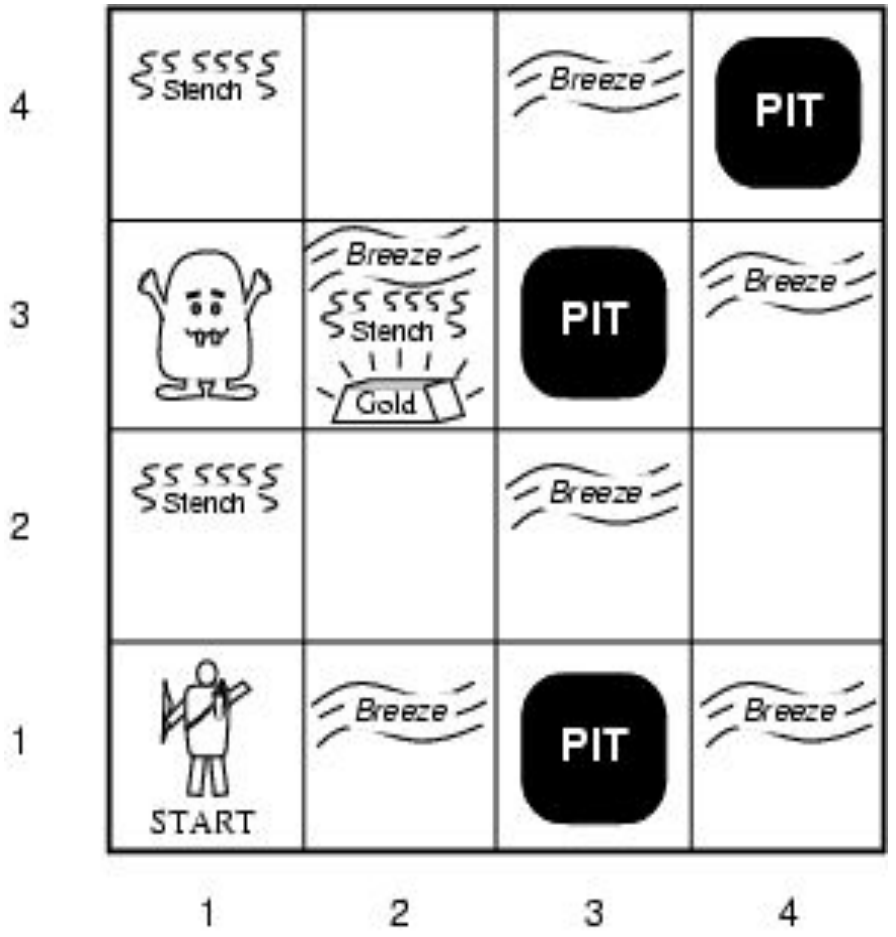
WUMPUS WORLD ACTIONS

- **go forward**
- **turn right** 90 degrees
- **turn left** 90 degrees
- **grab**: Pick up an object that is in the same square as the agent
- **shoot**: Fire an arrow in a straight line in the direction the agent is facing. The arrow continues until it either hits and kills the wumpus or hits the outer wall. The agent has only one arrow, so only the first Shoot action has any effect
- **climb** is used to leave the cave. This action is only effective in the start square
- **die**: This action automatically and irretrievably happens if the agent enters a square with a pit or a live wumpus

ILLUSTRATIVE EXAMPLE: WUMPUS WORLD

- **Performance measure**
  - gold +1000,
  - death -1000

  (falling into a pit or being eaten by the wumpus)
  - -1 per step, -10 for using the arrow
- **Environment**
  - Rooms / squares connected by doors.
  - Squares adjacent to wumpus are smelly
  - Squares adjacent to pit are breezy
  - Glitter iff gold is in the same square
  - Shooting kills wumpus if you are facing it
  - Shooting uses up the only arrow
  - Grabbing picks up gold if in same square
  - Releasing drops the gold in same square
  - Randomly generated at start of game. Wumpus only senses current room.
- **Sensors:** Stench, Breeze, Glitter, Bump, Scream [perceptual inputs]
- **Actuators:** Left turn, Right turn, Forward, Grab, Release, Shoot



WUMPUS WORLD CHARACTERIZATION

Fully Observable No – only local perception

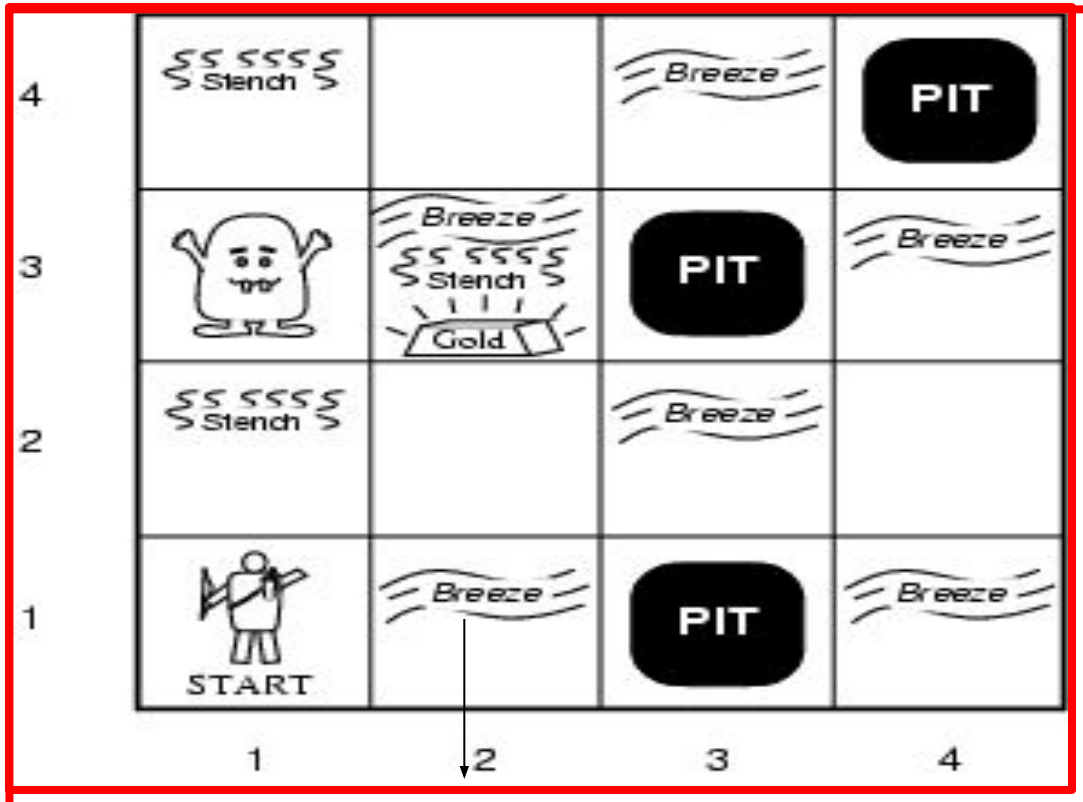
Deterministic Yes – outcomes exactly specified

Static Yes – Wumpus and Pits do not move

Discrete Yes

Single-agent? Yes – Wumpus is essentially a “natural feature.”

EXPLORING A WUMPUS WORLD



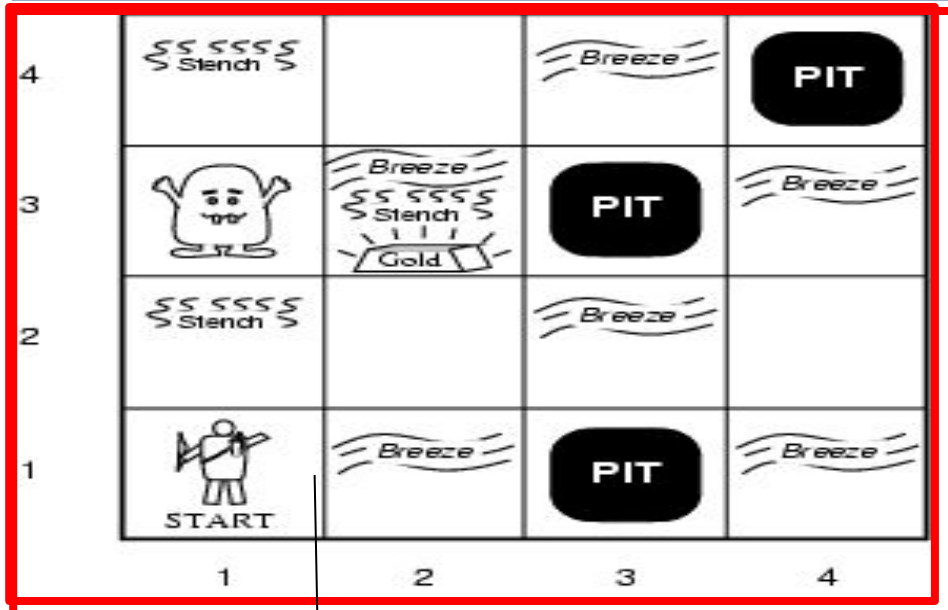
The knowledge base of the agent consists of the rules of the Wumpus world plus the percept "nothing" in [1,1]

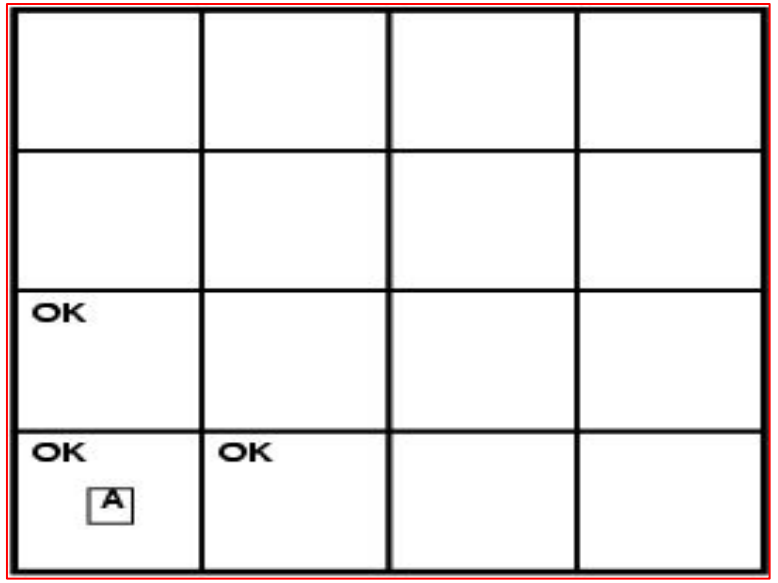
Boolean percept feature values:
<0, 0, 0, 0, 0>

None, none, none, none, none

Stench, Breeze, Glitter, Bump, Scream

EXPLORING A WUMPUS WORLD





None, none, none, none, none

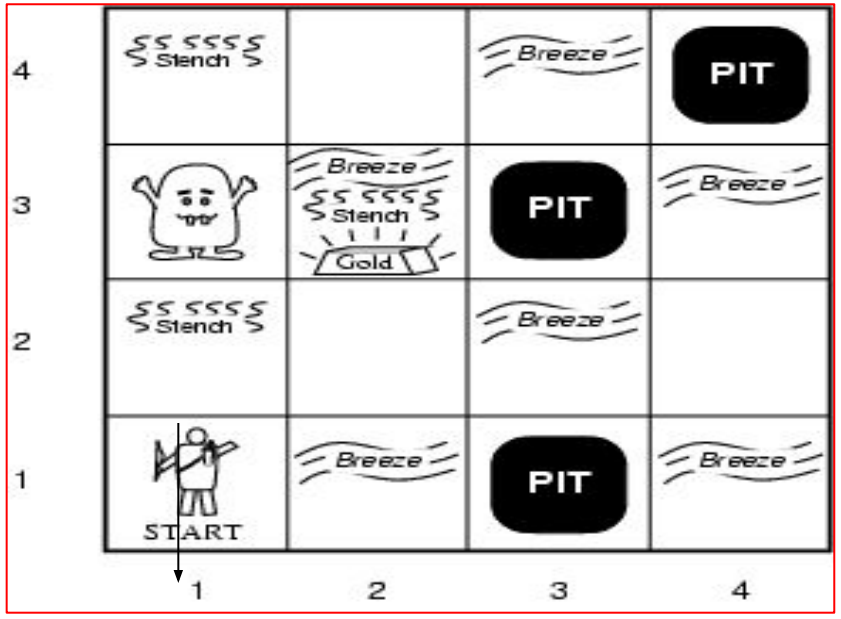
Stench, Breeze, Glitter, Bump, Scream

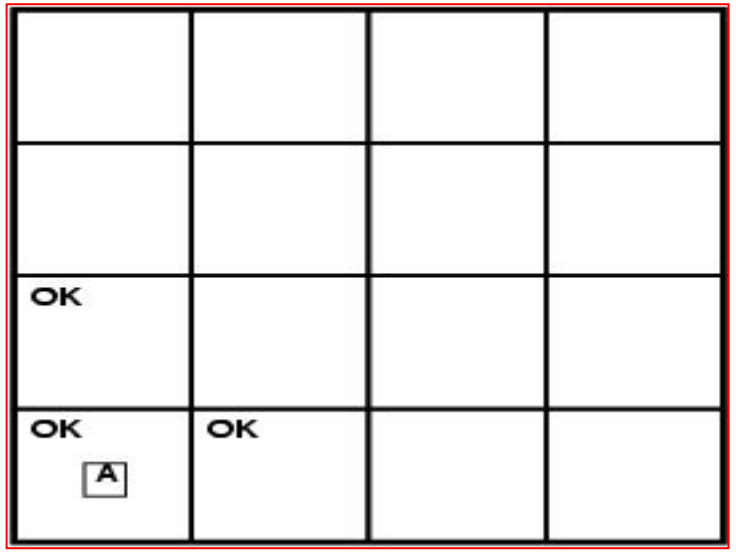
World "known" to agent at time = 0.

T=0 The KB of the agent consists of the rules of the Wumpus world plus the percept "nothing" in [1,1].
By inference, the agent's knowledge base also has the information that [2,1] and [1,2] are okay.
Added as propositions.

EXPLORING A WUMPUS WORLD

T = 0



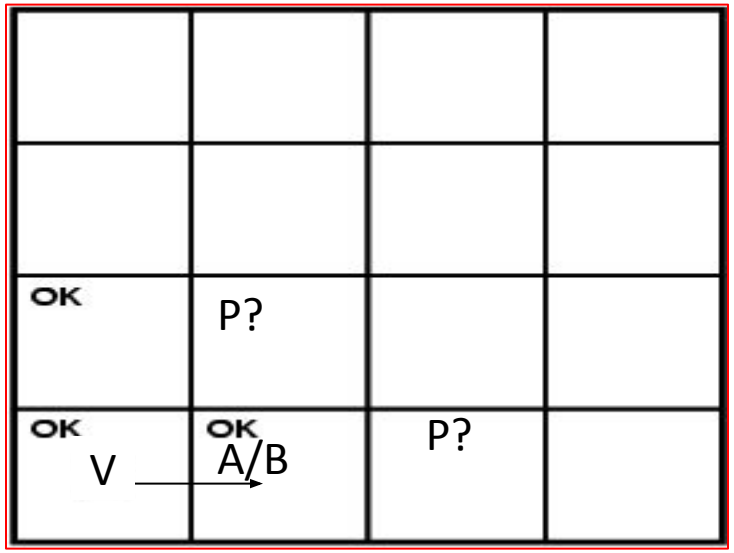


None, none, none, none, none

Stench, Breeze, Glitter, Bump, Scream

Where next?

T = 1

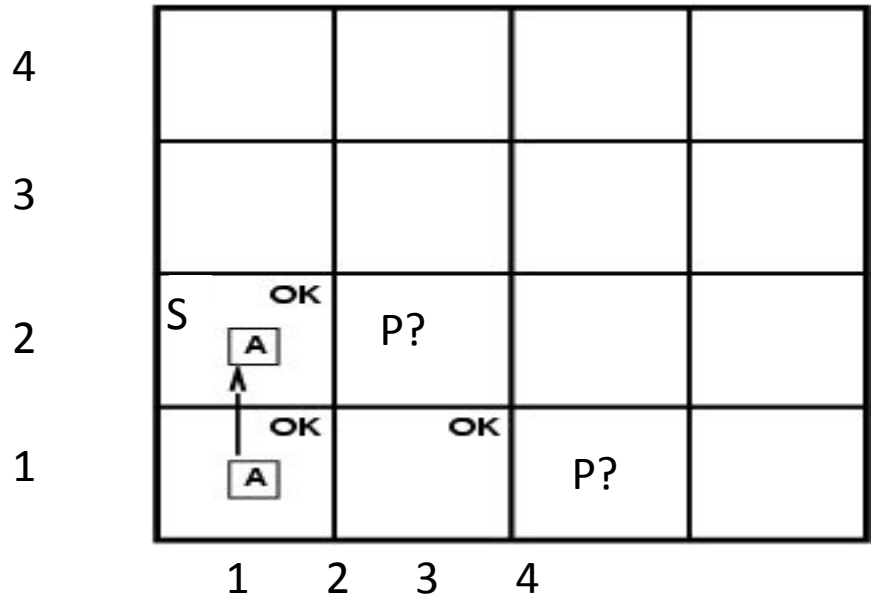


None, breeze, none, none, none

A – agent
V – visited
B - breeze

@ T = 1 What follows?
Pit(2,2) or Pit(3,1)

EXPLORING A WUMPUS WORLD

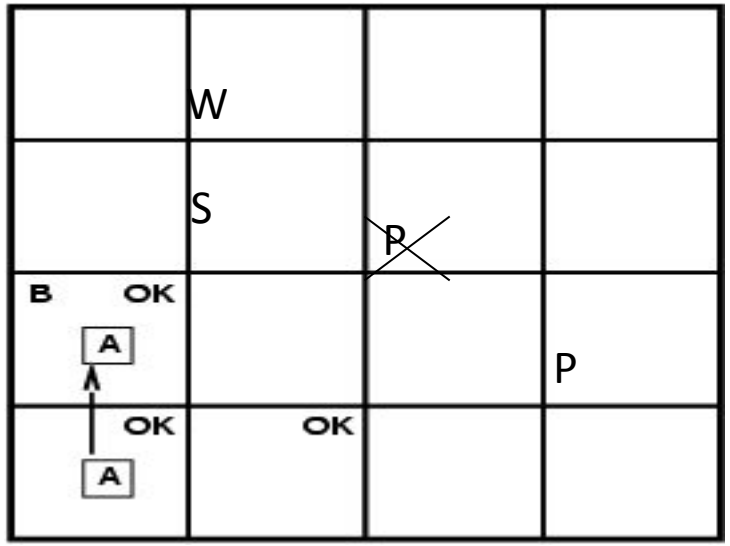


Stench, none, none, none, none

Stench, Breeze, Glitter, Bump, Scream

Where is Wumpus?

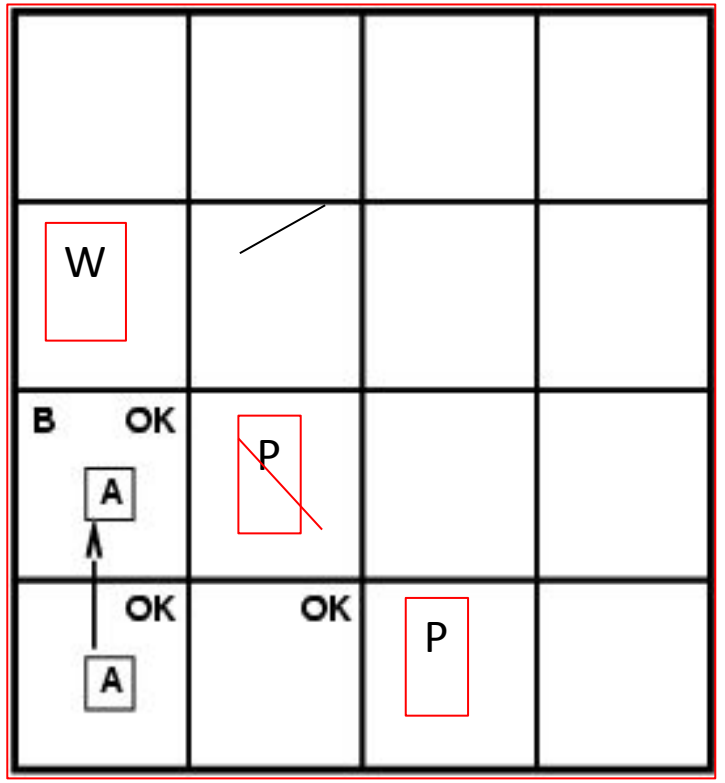
T=3



Wumpus cannot be in (1,1) or in (2,2) (Why?)□ Wumpus in (1,3)
Not breeze in (1,2) □ no pit in (2,2); but we know there is
pit in (2,2) or (3,1) □ pit in (3,1)

EXPLORING A WUMPUS WORLD

We reasoned about the possible states the Wumpus world can be in, given our percepts and our knowledge of the rules of the Wumpus world.
I.e., the content of KB at T=3.



What follows is what holds true in all those worlds that satisfy what is known at that time T=3 about the particular Wumpus world we are in.

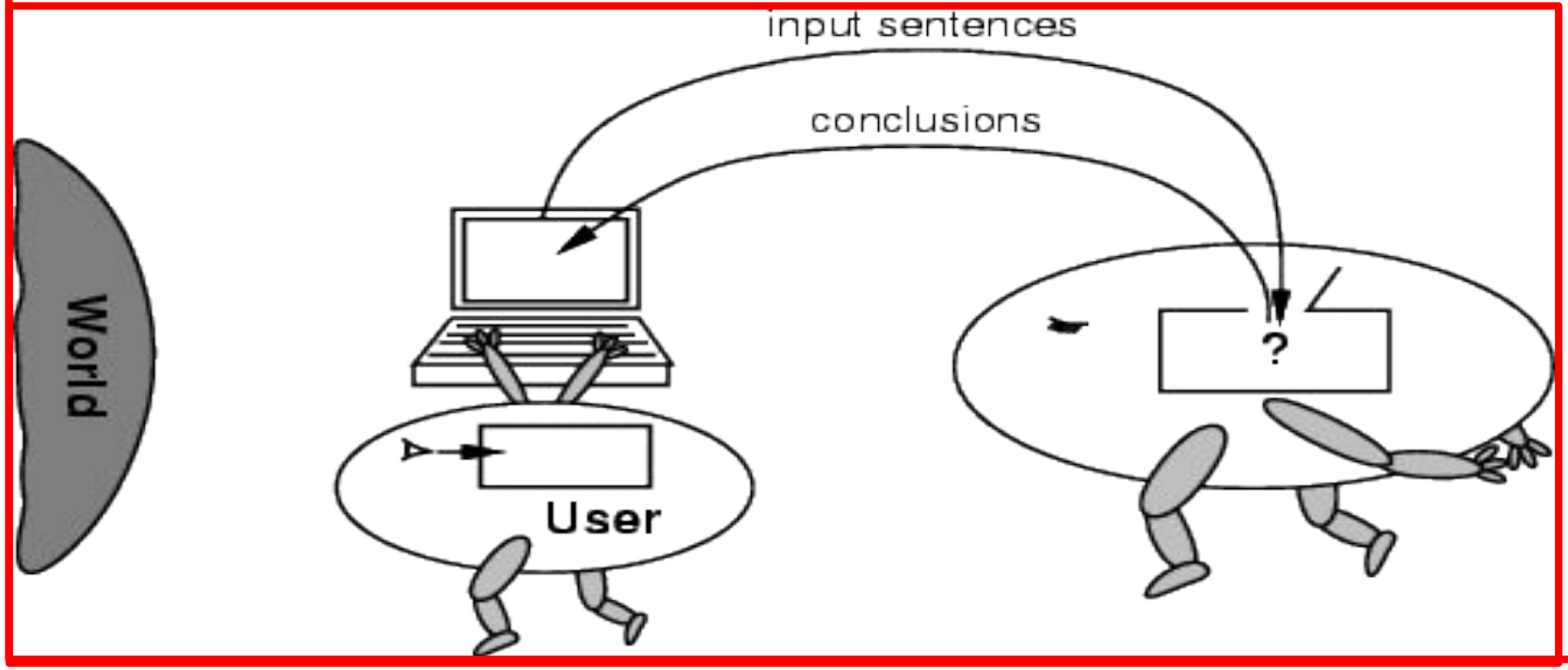
Example property: P_in_(3,1)

Models(KB) ⊆ Models(P_in_(3,1))

Essence of logical reasoning:
Given *all we know*, Pit_in_(3,1) holds.
("The world cannot be different.")

NO INDEPENDENT ACCESS TO THE WORLD

- The reasoning agent often gets its knowledge about the facts of the world as a sequence of logical sentences and must draw conclusions only from them without independent access to the world.
- Thus it is very important that the agent's reasoning is sound!



SUMMARY OF KNOWLEDGE BASED AGENTS

- Intelligent agents need knowledge about the world for making good decisions.
- The knowledge of an agent is stored in a knowledge base in the form of **sentences** in a knowledge representation language.
- A knowledge-based agent needs a **knowledge base** and an **inference mechanism**. It operates by storing sentences in its knowledge base, inferring new sentences with the inference mechanism, and using them to deduce which actions to take.
- A **representation language** is defined by its syntax and semantics, which specify the structure of sentences and how they relate to the facts of the world.
- The **interpretation** of a sentence is the fact to which it refers. If this fact is part of the actual world, then the sentence is true.

Knowledge and Reasoning
Table of Contents

- Knowledge and reasoning-Approaches and issues of knowledge reasoning-Knowledge base agents
- Logic Basics-Logic-Propositional logic-syntax ,semantics and inferences-Propositional logic-Reasoning patterns
- Unification and Resolution-Knowledge representation using rules-Knowledge representation using semantic nets
- Knowledge representation using frames-Inferences-
- Uncertain Knowledge and reasoning-Methods-Bayesian probability and belief network
- Probabilistic reasoning-Probabilistic reasoning over time-Probabilistic reasoning over time
- Other uncertain techniques-Data mining-Fuzzy logic-Dempster -shafer theory

What is a Logic?

- A language with concrete rules
  - No ambiguity in representation (may be other errors!)
  - Allows unambiguous communication and processing
  - Very unlike natural languages e.g. English
- Many ways to translate between languages
  - A statement can be represented in different logics
  - And perhaps differently in same logic
- **Expressiveness** of a logic
  - How much can we say in this language?
- Not to be confused with logical reasoning
  - Logics are languages, reasoning is a process (may **use** logic)

Syntax and Semantics

- Syntax
  - Rules for constructing legal sentences in the logic
  - Which symbols we can use (English: letters, punctuation)
  - How we are allowed to combine symbols
- Semantics
  - How we interpret (read) sentences in the logic
  - Assigns a meaning to each sentence
- Example: “All lecturers are seven foot tall”
  - A valid sentence (syntax)
  - And we can understand the meaning (semantics)
  - This sentence happens to be false (there is a counterexample)

Propositional Logic

- Syntax
  - Propositions, e.g. “it is wet”
  - Connectives: and, or, not, implies, iff (equivalent)
  - Brackets, T (true) and F (false) $\wedge$ $\vee$ $\neg$ $\rightarrow$ $\leftrightarrow$
- Semantics (Classical AKA Boolean)
  - Define how connectives affect truth
    - “P and Q” is true if and only if P is true and Q is true
  - Use **truth tables** to work out the truth of statements

Predicate Logic

- Propositional logic combines atoms
  - An atom contains no propositional connectives
  - Have no structure (today_is_wet, john_likes_apples)
- **Predicates** allow us to talk about objects
  - Properties: is_wet(today)
  - Relations: likes(john, apples)
  - True or false
- In predicate logic each atom is a predicate
  - e.g. first order logic, higher-order logic

First Order Logic

- More expressive logic than propositional
  - Used in this course (Lecture 6 on representation in FOL)
- **Constants** are objects: john, apples
- **Predicates** are properties and relations:
  - likes(john, apples)
- **Functions** transform objects:
  - likes(john, fruit_of(apple_tree))
- **Variables** represent any object: likes(X, apples)
- **Quantifiers** qualify values of variables
  - True for all objects (Universal): ∀X. likes(X, apples)
  - Exists at least one object (Existential): ∃X. likes(X, apples)

Example: FOL Sentence

- "Every rose has a thorn"

- For all X $\forall X.(rose(X) \rightarrow \exists Y.(has(X,Y) \wedge thorn(Y)))$
  - if (X is a rose)
  - then there exists Y
    - (X has Y) and (Y is a thorn)

Example: FOL Sentence

- "On Mondays and Wednesdays I go to John's house for dinner"

$$\forall X.((is\_mon(X) \vee is\_wed(X)) \longrightarrow eat\_meal(me, houseOf(john), X))$$

- Note the change from "and" to "or"
  - Translating is problematic

Higher Order Logic

- More expressive than first order
- Functions and predicates are also objects
  - Described by predicates: binary(addition)
  - Transformed by functions: differentiate(square)
  - Can quantify over both
- E.g. define red functions as having zero at 17

$$\forall F.(red(F) \leftrightarrow F(0) = 17)$$

- Much harder to reason with

Beyond True and False

- Multi-valued logics
  - More than two truth values
  - e.g., true, false & unknown
  - **Fuzzy logic** uses probabilities, truth value in [0,1]
- Modal logics
  - Modal operators define mode for propositions
  - **Epistemic logics** (belief)
    - e.g. ☐p (necessarily p), ◊p (possibly p), …
  - **Temporal logics** (time)
    - e.g. ☐p (always p), ◊p (eventually p), …

Knowledge and Reasoning
Table of Contents

- Knowledge and reasoning-Approaches and issues of knowledge reasoning-Knowledge base agents
- Logic Basics-Logic-Propositional logic-syntax ,semantics and inferences-Propositional logic- Reasoning patterns
- Unification and Resolution-Knowledge representation using rules-Knowledge representation using semantic nets
- Knowledge representation using frames-Inferences-
- Uncertain Knowledge and reasoning-Methods-Bayesian probability and belief network
- Probabilistic reasoning-Probabilistic reasoning over time-Probabilistic reasoning over time
- Other uncertain techniques-Data mining-Fuzzy logic-Dempster -shafer theory

Knowledge and Reasoning
Table of Contents

- Knowledge and reasoning-Approaches and issues of knowledge reasoning-Knowledge base agents
- Logic Basics-Logic-Propositional logic-syntax ,semantics and inferences-Propositional logic- Reasoning patterns
- Unification and Resolution-Knowledge representation using rules-Knowledge representation using semantic nets
- Knowledge representation using frames-Inferences-
- Uncertain Knowledge and reasoning-Methods-Bayesian probability and belief network
- Probabilistic reasoning-Probabilistic reasoning over time-Probabilistic reasoning over time
- Other uncertain techniques-Data mining-Fuzzy logic-Dempster -shafer theory

Propositional logic

- Propositional logic consists of:
  - The logical values true and false (T and F)
  - Propositions: "Sentences," which
    - Are atomic (that is, they must be treated as indivisible units, with no internal structure), and
    - Have a single logical value, either true or false
  - Operators, both unary and binary; when applied to logical values, yield logical values
    - The usual operators are and, or, not, and implies

Truth tables

- Logic, like arithmetic, has operators, which apply to one, two, or more values (operands)
- A truth table lists the results for each possible arrangement of operands
  - Order is important: ***x op y*** may or may not give the same result as ***y op x***
- The rows in a truth table list all possible sequences of truth values for n operands, and specify a result for each sequence
  - Hence, there are 2^n rows in a truth table for n operands

Unary operators

- There are four possible unary operators:

X	Constant true, (T)
T	T
F	T

X	Identity, (X)
T	T
F	F

X	Constant false, (F)
T	F
F	F

X	**Negation, ¬X**
T	F
F	T

- Only the last of these (negation) is widely used (and has a symbol,¬ ,for the operation

Combined tables for unary operators

X	Constant T	Constant F	Identity	¬X
T	T	F	T	F
F	T	F	F	T

Binary operators

- There are sixteen possible binary operators:

X	Y																
T	T	T	T	T	T	T	T	T	T	F	F	F	F	F	F	F	F
T	F	T	T	T	T	F	F	F	F	T	T	T	T	F	F	F	F
F	T	T	T	F	F	T	T	F	F	T	T	F	F	T	T	F	F
F	F	T	F	T	F	T	F	T	F	T	F	T	F	T	F	T	F

- All these operators have names, but I haven't tried to fit them in
- Only a few of these operators are normally used in logic

Useful binary operators

- Here are the binary operators that are traditionally used:

X	Y	AND X ∧ Y	OR X ∨ Y	IMPLIES X ⇒ Y	BICONDITIONAL X ⇔ Y
T	T	T	T	T	T
T	F	F	T	F	F
F	T	F	T	T	F
F	F	F	F	T	T

- Notice in particular that material implication (⇒) only approximately means the same as the English word "implies"
- All the other operators can be constructed from a combination of these (along with unary not, ¬)

Logical expressions

- All logical expressions can be computed with some combination of and ($\wedge$), or ($\vee$), and not ($\neg$) operators
- For example, logical implication can be computed this way:

X	Y	$\neg X$	$\neg X \vee Y$	$X \Rightarrow Y$
T	T	F	T	T
T	F	F	F	F
F	T	T	T	T
F	F	T	T	T

- Notice that $\neg X \vee Y$ is equivalent to $X \Rightarrow Y$

X	Y	¬X	X <=Y> = X =>Y AND Y => X	X <=Y> = X =>Y AND Y => X	X <=Y> = X =>Y AND Y => X
T	T	F	T	T	T
T	F	F	F	T	F
F	T	T	T	F	F
F	F	T	T	T	T

Another example

- Exclusive or (xor) is true if exactly one of its operands is true

X	Y	¬X	¬Y	¬X ∧ Y	X ∧ ¬Y	(¬X∧Y)∨(X∧¬Y)	X xor Y
T	T	F	F	F	F	F	F
T	F	F	T	F	T	T	T
F	T	T	F	T	F	T	T
F	F	T	T	F	F	F	F

- Notice that $(\neg X \wedge Y) \vee (X \wedge \neg Y)$ is equivalent to X xor Y

Given a propositional logic, check if it is valid

- (X-> Y o (x v y))
- Valid

X	Y	X->Y	X v Y	X->Y or X v Y
0	0	1	0	1
0	1	0	1	1
1	0	1	1	1
1	1	1	1	1

World

- A world is a collection of prepositions and logical expressions relating those prepositions
- Example:
  - Propositions: JohnLovesMary, MaryIsFemale, MaryIsRich
  - Expressions:
    MaryIsFemale $\wedge$ MaryIsRich $\Rightarrow$ JohnLovesMary
- A proposition "says something" about the world, but since it is atomic (you can't look inside it to see component parts), propositions tend to be very specialized and inflexible

Models

A model is an assignment of a truth value to each proposition, for example:

- JohnLovesMary: T, MaryIsFemale: T, MaryIsRich: F

- An expression is satisfiable if there is a model for which the expression is true
  - For example, the above model satisfies the expression
    MaryIsFemale ∧ MaryIsRich ⇒ JohnLovesMary
- An expression is valid if it is satisfied by *every* model
  - This expression is *not* valid:
    MaryIsFemale ∧ MaryIsRich ⇒ JohnLovesMary
    because it is not satisfied by this model:
    JohnLovesMary: F, MaryIsFemale: T, MaryIsRich: T
  - But this expression *is* valid:
    MaryIsFemale ∧ MaryIsRich ⇒ MaryIsFemale

Inference rules in propositional logic

- Here are just a few of the rules you can apply when reasoning in propositional logic:

$$(\alpha \wedge \beta) \equiv (\beta \wedge \alpha) \quad \text{commutativity of } \wedge$$
$$(\alpha \vee \beta) \equiv (\beta \vee \alpha) \quad \text{commutativity of } \vee$$
$$((\alpha \wedge \beta) \wedge \gamma) \equiv (\alpha \wedge (\beta \wedge \gamma)) \quad \text{associativity of } \wedge$$
$$((\alpha \vee \beta) \vee \gamma) \equiv (\alpha \vee (\beta \vee \gamma)) \quad \text{associativity of } \vee$$
$$\neg(\neg\alpha) \equiv \alpha \quad \text{double-negation elimination}$$
$$(\alpha \Rightarrow \beta) \equiv (\neg\beta \Rightarrow \neg\alpha) \quad \text{contraposition}$$
$$(\alpha \Rightarrow \beta) \equiv (\neg\alpha \vee \beta) \quad \text{implication elimination}$$
$$(\alpha \Leftrightarrow \beta) \equiv ((\alpha \Rightarrow \beta) \wedge (\beta \Rightarrow \alpha)) \quad \text{biconditional elimination}$$
$$\neg(\alpha \wedge \beta) \equiv (\neg\alpha \vee \neg\beta) \quad \text{de Morgan}$$
$$\neg(\alpha \vee \beta) \equiv (\neg\alpha \wedge \neg\beta) \quad \text{de Morgan}$$
$$(\alpha \wedge (\beta \vee \gamma)) \equiv ((\alpha \wedge \beta) \vee (\alpha \wedge \gamma)) \quad \text{distributivity of } \wedge \text{ over } \vee$$
$$(\alpha \vee (\beta \wedge \gamma)) \equiv ((\alpha \vee \beta) \wedge (\alpha \vee \gamma)) \quad \text{distributivity of } \vee \text{ over } \wedge$$

Implication elimination

- A particularly important rule allows you to get rid of the implication operator, $\Rightarrow$:
  - $X \Rightarrow Y \equiv \neg X \vee Y$
- We will use this later on as a necessary tool for simplifying logical expressions
- The symbol $\equiv$ means “is logically equivalent to”

Conjunction elimination

- Another important rule for simplifying logical expressions allows you to get rid of the conjunction (**and**) operator, $\wedge$:
- This rule simply says that if you have an **and** operator at the top level of a fact (logical expression), you can break the expression up into two separate facts:
  - MaryIsFemale $\wedge$ MaryIsRich
    - becomes:
  - MaryIsFemale
  - MaryIsRich

Inference by computer

- To do inference (reasoning) by computer is basically a *search* process, taking logical expressions and applying inference rules to them
  - Which logical expressions to use?
  - Which inference rules to apply?
- Usually you are trying to “prove” some particular statement
- Example:
  - it_is_raining ∨ it_is_sunny
  - it_is_sunny ⇒ I_stay_dry
  - A => B = NOT A V B
  - NOT(it_is_sunny) ∨ I_stay_dry
  - it_is_rainy ⇒ I_take_umbrella
  - NOT(it_is_rainy) ∨ I_take_umbrella
  - I_take_umbrella ⇒ I_stay_dry
  - NOT(I_take_umbrella) ∨ I_stay_dry
  - To prove: I_stay_dry

Knowledge and Reasoning
Table of Contents

- Knowledge and reasoning-Approaches and issues of knowledge reasoning-Knowledge base agents
- Logic Basics-Logic-Propositional logic-syntax ,semantics and inferences-Propositional logic- Reasoning patterns
- Unification and Resolution-Knowledge representation using rules-Knowledge representation using semantic nets
- Knowledge representation using frames-Inferences-
- Uncertain Knowledge and reasoning-Methods-Bayesian probability and belief network
- Probabilistic reasoning-Probabilistic reasoning over time-Probabilistic reasoning over time
- Other uncertain techniques-Data mining-Fuzzy logic-Dempster -shafer theory

Reasoning Patterns

- Inference in propositional logic is NP-complete!
- However, inference in propositional logic shows monoticity:
  - Adding more rules to a knowledge base does not affect earlier inferences

Forward and backward reasoning

- Situation: You have a collection of logical expressions (premises), and you are trying to prove some additional logical expression (the conclusion)
- You can:
  - Do forward reasoning: Start applying inference rules to the logical expressions you have, and stop if one of your results is the conclusion you want
  - Do backward reasoning: Start from the conclusion you want, and try to choose inference rules that will get you back to the logical expressions you have
- With the tools we have discussed so far, *neither* is feasible

Example

- Given:
  - it_is_raining ∨ it_is_sunny
  - it_is_sunny ⇒ I_stay_dry
  - it_is_raining ⇒ I_take_umbrella
  - I_take_umbrella ⇒ I_stay_dry
- You can conclude:
  - it_is_sunny ∨ it_is_raining
  - I_take_umbrella ∨ it_is_sunny
  - ¬I_stay_dry ⇒ I_take_umbrella
  - Etc., etc. … there are *just too many* things you can conclude!

Predicate calculus

- Predicate calculus is also known as "First Order Logic" (FOL)
- Predicate calculus includes:
  - All of propositional logic
    - Logical values true, false
    - Variables x, y, a, b,...
    - Connectives ¬, ⇒, ∧, ∨, ⇔
  - Constants KingJohn, 2, Villanova,...
  - Predicates Brother, >,...
  - Functions Sqrt, MotherOf,...
  - Quantifiers ∀, ∃

Constants, functions, and predicates

- A constant represents a "thing"--it has no truth value, and it does not occur "bare" in a logical expression
  - Examples: DavidMatuszek, 5, Earth, goodIdea
- Given zero or more arguments, a function produces a constant as its value:
  - Examples: motherOf(DavidMatuszek), add(2, 2), thisPlanet()
- A predicate is like a function, but produces a truth value
  - Examples: greatInstructor(DavidMatuszek), isPlanet(Earth), greater(3, add(2, 2))

Universal quantification

- The universal quantifier, $\forall$, is read as "for each" or "for every"
  - Example: $\forall x,\ x^2 \geq 0$ (for all x, x^2 is greater than or equal to zero)
- Typically, $\Rightarrow$ is the main connective with $\forall$:

  $\forall x,\ at(x,Villanova) \Rightarrow smart(x)$

  means "Everyone at Villanova is smart"
- Common mistake: using $\wedge$ as the main connective with $\forall$:

  $\forall x,\ at(x,Villanova) \wedge smart(x)$

  means "Everyone is at Villanova and everyone is smart"
- If there are no values satisfying the condition, the result is true
  - Example: $\forall x,\ isPersonFromMars(x) \Rightarrow smart(x)$ is true

Existential quantification

- The existential quantifier, $\exists$, is read "for some" or "there exists"
  - Example: $\exists x,\ x^2 < 0$ (there exists an x such that x^2 is less than zero)
- Typically, $\wedge$ is the main connective with $\exists$:

  $\exists x,\ at(x,Villanova) \wedge smart(x)$

  means "There is someone who is at Villanova and is smart"
- Common mistake: using $\Rightarrow$ as the main connective with $\exists$:

  $\exists x,\ at(x,Villanova) \Rightarrow smart(x)$

  This is true if there is someone at Villanova who is smart...

  ...but it is also true if there is someone who is *not* at Villanova

  By the rules of material implication, the result of $F \Rightarrow T$ is T

Properties of quantifiers

- ∀x ∀y is the same as ∀y ∀x LOVES (X,Y)
- ∃x ∃y is the same as ∃y ∃x

- ∃x ∀y is *not* the same as ∀y ∃x
- ∃x ∀y Loves(x,y)
  - "There is a person who loves everyone in the world"
  - More exactly: ∃x ∀y (person(x) ∧ person(y) ⇒ Loves(x,y))
- ∀y ∃x Loves(x,y)
  - "Everyone in the world is loved by at least one person"

- Quantifier duality: each can be expressed using the other
- ∀x Likes(x,IceCream) ¬∃x ¬Likes(x,IceCream)
- ∃x Likes(x,Broccoli) ¬∀x ¬Likes(x,Broccoli)

Parentheses

- Parentheses are often used with quantifiers
- Unfortunately, everyone uses them differently, so don't be upset at any usage you see
- Examples:
  - $(\forall x)\ person(x) \Rightarrow likes(x, iceCream)$
  - $(\forall x)\ (person(x) \Rightarrow likes(x, iceCream))$
  - $(\forall x)\ [\ person(x) \Rightarrow likes(x, iceCream)\ ]$
  - $\forall x,\ person(x) \Rightarrow likes(x, iceCream)$
  - $\forall x\ (person(x) \Rightarrow likes(x, iceCream))$
- I prefer parentheses that show the scope of the quantifier
  - $\exists x\ (x > 0)\ \wedge\ \exists x\ (x < 0)$

More rules

- Now there are numerous additional rules we can apply!
- Here are two exceptionally important rules:
  - $\neg \forall x,\ p(x) \quad \Rightarrow \quad \exists x,\ \neg p(x)$
    "If not every x satisfies p(x), then there exists a x that does not satisfy p(x)"
  - $\neg \exists x,\ p(x) \quad \Rightarrow \quad \forall x,\ \neg p(x)$
    "If there does not exist an x that satisfies p(x), then all x do not satisfy p(x)"
- In any case, the search space is *just too large* to be feasible
- This was the case until 1970, when J. Robinson discovered resolution

Knowledge and Reasoning
Table of Contents

- Knowledge and reasoning-Approaches and issues of knowledge reasoning-Knowledge base agents
- Logic Basics-Logic-Propositional logic-syntax ,semantics and inferences-Propositional logic- Reasoning patterns
- Unification and Resolution-Knowledge representation using rules-Knowledge representation using semantic nets
- Knowledge representation using frames-Inferences-
- Uncertain Knowledge and reasoning-Methods-Bayesian probability and belief network
- Probabilistic reasoning-Probabilistic reasoning over time-Probabilistic reasoning over time
- Other uncertain techniques-Data mining-Fuzzy logic-Dempster -shafer theory

Logic by computer was infeasible

- Why is logic so hard?
  - You start with a large collection of facts (predicates)
  - You start with a large collection of possible transformations (rules)
    - Some of these rules apply to a single fact to yield a new fact
    - Some of these rules apply to a pair of facts to yield a new fact
  - So at every step you must:
    - Choose some rule to apply
    - Choose one or two facts to which you might be able to apply the rule
      - If there are n facts
        - There are n potential ways to apply a single-operand rule
        - There are n * (n - 1) potential ways to apply a two-operand rule
    - Add the new fact to your ever-expanding fact base
  - The search space is huge!

The magic of resolution

- Here's how resolution works:
  - You transform each of your facts into a particular form, called a clause (this is the tricky part)
  - You apply a *single rule,* the resolution principle, to a pair of clauses
    - Clauses are closed with respect to resolution--that is, when you resolve two clauses, you get a new clause
  - You add the new clause to your fact base
- So the number of facts you have grows linearly
  - You still have to choose a pair of facts to resolve
  - You never have to choose a rule, because there's only one

The fact base

- A fact base is a collection of "facts," expressed in predicate calculus, that are presumed to be true (valid)
- These facts are implicitly "**and**ed" together
- Example fact base:
  - seafood(X) ⇒ likes(John, X) (where X is a variable)
  - seafood(shrimp)
  - pasta(X) ⇒ ¬likes(Mary, X) (where X is a *different* variable)
  - pasta(spaghetti)
- That is,
  - (seafood(X) ⇒ likes(John, X)) ∧ seafood(shrimp) ∧ (pasta(Y) ⇒ ¬likes(Mary, Y)) ∧ pasta(spaghetti)
  - Notice that we had to change some Xs to Ys
  - The scope of a variable is the single fact in which it occurs

Clause form

- A clause is a disjunction ("**or**") of zero or more literals, some or all of which may be negated
- Example:
  sinks(X) ∨ dissolves(X, water) ∨ ¬denser(X, water)
- Notice that clauses use only “or” and “not”—they do not use “and,” “implies,” or either of the quantifiers “for all” or “there exists”
- The impressive part is that *any* predicate calculus expression can be put into clause form
  - Existential quantifiers, ∃, are the trickiest ones

Unification

- From the pair of facts (not yet clauses, just facts):
  - seafood(X) ⇒ likes(John, X) (where X is a variable)
  - seafood(shrimp)
- We ought to be able to conclude
  - likes(John, shrimp)
- We can do this by unifying the variable X with the constant shrimp
  - This is the *same* "unification" as is done in Prolog
- This unification turns seafood(X) ⇒ likes(John, X) into seafood(shrimp) ⇒ likes(John, shrimp)
- Together with the given fact seafood(shrimp), the final deductive step is easy

The resolution principle

- Here it is:
  - From X ∨ someLiterals
    and ¬X ∨ someOtherLiterals
    ---
    conclude: someLiterals ∨ someOtherLiterals
- That's all there is to it!
- Example:
  - broke(Bob) ∨ well-fed(Bob)
    ¬broke(Bob) ∨ ¬hungry(Bob)
    ---
    well-fed(Bob) ∨ ¬hungry(Bob)

A common error

- You can only do *one* resolution at a time
- Example:
  - broke(Bob) ∨ well-fed(Bob) ∨ happy(Bob)
    ¬broke(Bob) ∨ ¬hungry(Bob) ∨ ¬happy(Bob)
- You can resolve on broke to get:
  - well-fed(Bob) ∨ happy(Bob) ∨ ¬hungry(Bob) ∨ ¬happy(Bob) ≡ T
- Or you can resolve on happy to get:
  - broke(Bob) ∨ well-fed(Bob) ∨ ¬broke(Bob) ∨ ¬hungry(Bob) ≡ T
- Note that both legal resolutions yield a tautology (a trivially true statement, containing X ∨ ¬X), which is correct but useless
- But you *cannot* resolve on both at once to get:
  - well-fed(Bob) ∨ ¬hungry(Bob)

Contradiction

- A special case occurs when the result of a resolution (the resolvent) is empty, or “NIL”
- Example:
  - hungry(Bob)
    ¬hungry(Bob)

    NIL
- In this case, the fact base is inconsistent
- This will turn out to be a very useful observation in doing resolution theorem proving

A first example

- "Everywhere that John goes, Rover goes. John is at school."
  - at(John, X) ⇒ at(Rover, X) (not yet in clause form)
  - at(John, school) (already in clause form)
- We use implication elimination to change the first of these into clause form:
  - ¬at(John, X) ∨ at(Rover, X)
  - at(John, school)
- We can resolve these on at(-, -), but to do so we have to unify X with school; this gives:
  - at(Rover, school)

Refutation resolution

- The previous example was easy because it had very few clauses
- When we have a lot of clauses, we want to *focus* our search on the thing we would like to prove
- We can do this as follows:
  - Assume that our fact base is consistent (we can't derive NIL)
  - Add the *negation* of the thing we want to prove to the fact base
  - Show that the fact base is now inconsistent
  - Conclude the thing we want to prove

Example of refutation resolution

- "Everywhere that John goes, Rover goes. John is at school. **Prove that Rover is at school.**"
  1. ¬at(John, X) ∨ at(Rover, X)
  2. at(John, school)
  3. ¬at(Rover, school) (this is the added clause)
- Resolve #1 and #3:
  4. ¬at(John, X)
- Resolve #2 and #4:
  4. NIL
- Conclude the negation of the added clause: at(Rover, school)
- This seems a roundabout approach for such a simple example, but it works well for larger problems

A second example

- Start with:
  - it_is_raining ∨ it_is_sunny
  - it_is_sunny ⇒ I_stay_dry
  - it_is_raining ⇒ I_take_umbrella
  - I_take_umbrella ⇒ I_stay_dry
- Convert to clause form:
  1. it_is_raining ∨ it_is_sunny
  2. ¬it_is_sunny ∨ I_stay_dry
  3. ¬it_is_raining ∨ I_take_umbrella
  4. ¬I_take_umbrella ∨ I_stay_dry
- Prove that I stay dry:
  5. ¬I_stay_dry

- Proof:
  6. (5, 2) ¬it_is_sunny
  7. (6, 1) it_is_raining
  8. (5, 4) ¬I_take_umbrella
  9. (8, 3) ¬it_is_raining
  10. (9, 7) NIL
- Therefore, ¬(¬I_stay_dry)
  - I_stay_dry

Converting sentences to CNF

1. Eliminate all ↔ connectives
   $(P \leftrightarrow Q) \Rightarrow ((P \rightarrow Q) \wedge (Q \rightarrow P))$
2. Eliminate all → connectives
   $(P \rightarrow Q) \Rightarrow (\neg P \vee Q)$
3. Reduce the scope of each negation symbol to a single predicate
   $\neg\neg P \Rightarrow P$
   $\neg(P \vee Q) \Rightarrow \neg P \wedge \neg Q$
   $\neg(P \wedge Q) \Rightarrow \neg P \vee \neg Q$
   $\neg(\forall x)P \Rightarrow (\exists x)\neg P$
   $\neg(\exists x)P \Rightarrow (\forall x)\neg P$
4. Standardize variables: rename all variables so that each quantifier has its own unique variable name

Converting sentences to clausal form Skolem constants and functions

5. Eliminate existential quantification by introducing Skolem constants/functions

$(\exists x)P(x) \Rightarrow P(c)$

c is a Skolem constant (a brand-new constant symbol that is not used in any other sentence)

$(\forall x)(\exists y)P(x,y) \Rightarrow (\forall x)P(x, f(x))$

since $\exists$ is within the scope of a universally quantified variable, use a **Skolem function f** to construct a new value that **depends on** the universally quantified variable

f must be a brand-new function name not occurring in any other sentence in the KB.

E.g., $(\forall x)(\exists y)loves(x,y) \Rightarrow (\forall x)loves(x,f(x))$

In this case, f(x) specifies the person that x loves

Converting sentences to clausal form

6. Remove universal quantifiers by (1) moving them all to the left end; (2) making the scope of each the entire sentence; and (3) dropping the "prefix" part

   Ex: $(\forall x)P(x) \Rightarrow P(x)$

7. Put into conjunctive normal form (conjunction of disjunctions) using distributive and associative laws

   $(P \wedge Q) \vee R \Rightarrow (P \vee R) \wedge (Q \vee R)$

   $(P \vee Q) \vee R \Rightarrow (P \vee Q \vee R)$

8. Split conjuncts into separate clauses

9. Standardize variables so each clause contains only variable names that do not occur in any other clause

An example

$(\forall x)(P(x) \rightarrow ((\forall y)(P(y) \rightarrow P(f(x,y))) \wedge \neg(\forall y)(Q(x,y) \rightarrow P(y))))$

2. Eliminate $\rightarrow$

$(\forall x)(\neg P(x) \vee ((\forall y)(\neg P(y) \vee P(f(x,y))) \wedge \neg(\forall y)(\neg Q(x,y) \vee P(y))))$

3. Reduce scope of negation

$(\forall x)(\neg P(x) \vee ((\forall y)(\neg P(y) \vee P(f(x,y))) \wedge (\exists y)(Q(x,y) \wedge \neg P(y))))$

4. Standardize variables

$(\forall x)(\neg P(x) \vee ((\forall y)(\neg P(y) \vee P(f(x,y))) \wedge (\exists z)(Q(x,z) \wedge \neg P(z))))$

5. Eliminate existential quantification

$(\forall x)(\neg P(x) \vee ((\forall y)(\neg P(y) \vee P(f(x,y))) \wedge (Q(x,g(x)) \wedge \neg P(g(x)))))$

6. Drop universal quantification symbols

$(\neg P(x) \vee ((\neg P(y) \vee P(f(x,y))) \wedge (Q(x,g(x)) \wedge \neg P(g(x)))))$

Example

7. Convert to conjunction of disjunctions

   $(\neg P(x) \vee \neg P(y) \vee P(f(x,y))) \wedge (\neg P(x) \vee Q(x,g(x))) \wedge$
   $(\neg P(x) \vee \neg P(g(x)))$

8. Create separate clauses

   $\neg P(x) \vee \neg P(y) \vee P(f(x,y))$
   $\neg P(x) \vee Q(x,g(x))$
   $\neg P(x) \vee \neg P(g(x))$

9. Standardize variables

   $\neg P(x) \vee \neg P(y) \vee P(f(x,y))$
   $\neg P(z) \vee Q(z,g(z))$
   $\neg P(w) \vee \neg P(g(w))$

Resolution

- Resolution is a **sound** and **complete** inference procedure for FOL
- Reminder: Resolution rule for propositional logic:
  - $P_1 \vee P_2 \vee \ldots \vee P_n$
  - $\neg P_1 \vee Q_2 \vee \ldots \vee Q_m$
  - Resolvent: $P_2 \vee \ldots \vee P_n \vee Q_2 \vee \ldots \vee Q_m$
- Examples
  - P and ¬ P ∨ Q : derive Q (Modus Ponens)
  - (¬ P ∨ Q) and (¬ Q ∨ R) : derive ¬ P ∨ R
  - P and ¬ P : derive False [contradiction!]
  - (P ∨ Q) and (¬ P ∨ ¬ Q) : derive True

Resolution in first-order logic

- Given sentences

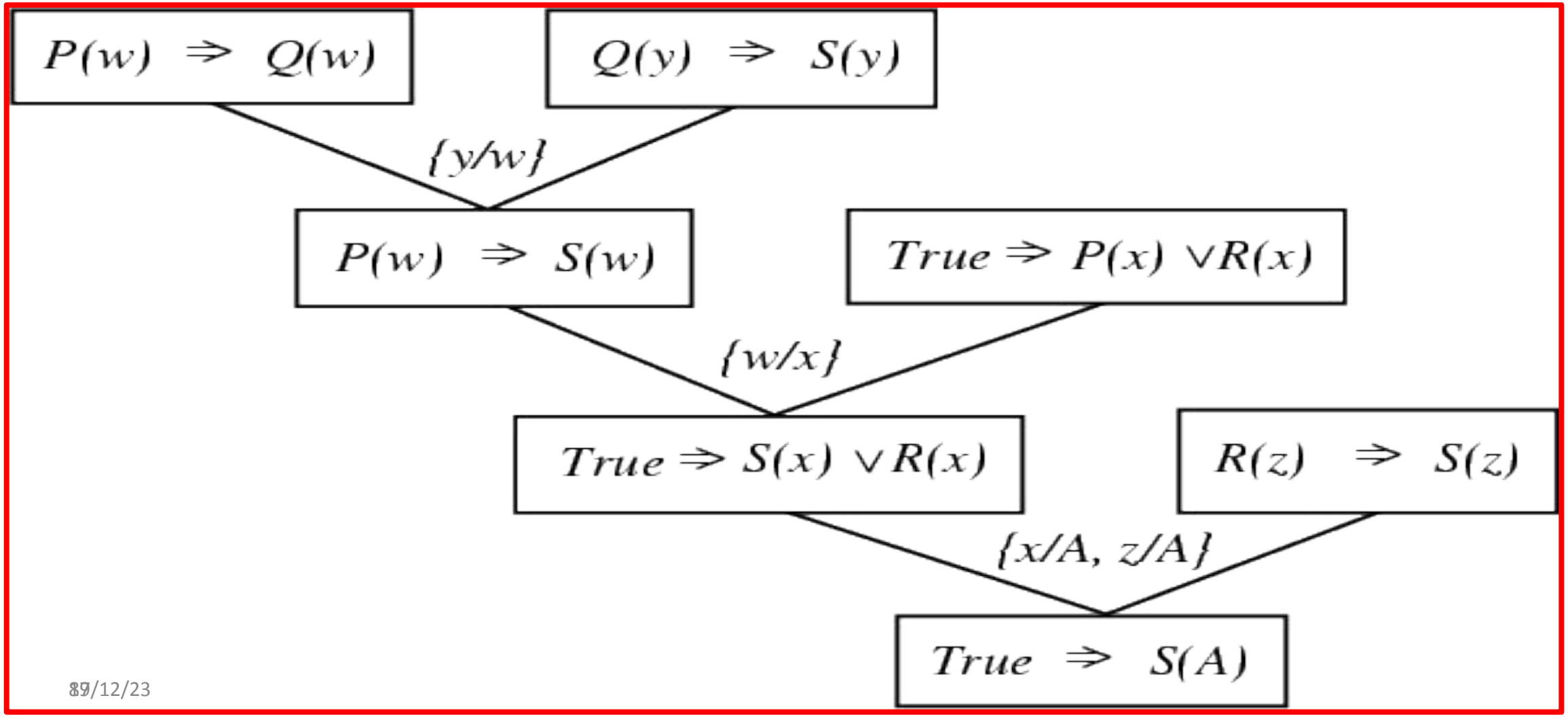
  $P_1 \vee \ldots \vee P_n$

  $Q_1 \vee \ldots \vee Q_m$

- in *conjunctive normal form:*
  - each P_i and Q_i is a literal, i.e., a positive or negated predicate symbol with its terms,
- if P_j and $\neg Q_k$ **unify** with substitution list θ, then derive the resolvent sentence:

  $subst(\theta, P_1 \vee \ldots \vee P_{j-1} \vee P_{j+1} \ldots P_n \vee Q_1 \vee \ldots Q_{k-1} \vee Q_{k+1} \vee \ldots \vee Q_m)$
- Example
  - from clause **$P(x, f(a)) \vee P(x, f(y)) \vee Q(y)$**
  - and clause **$\neg P(z, f(a)) \vee \neg Q(z)$**
  - derive resolvent **$P(z, f(y)) \vee Q(y) \vee \neg Q(z)$**
  - using **$\theta = \{x/z\}$**

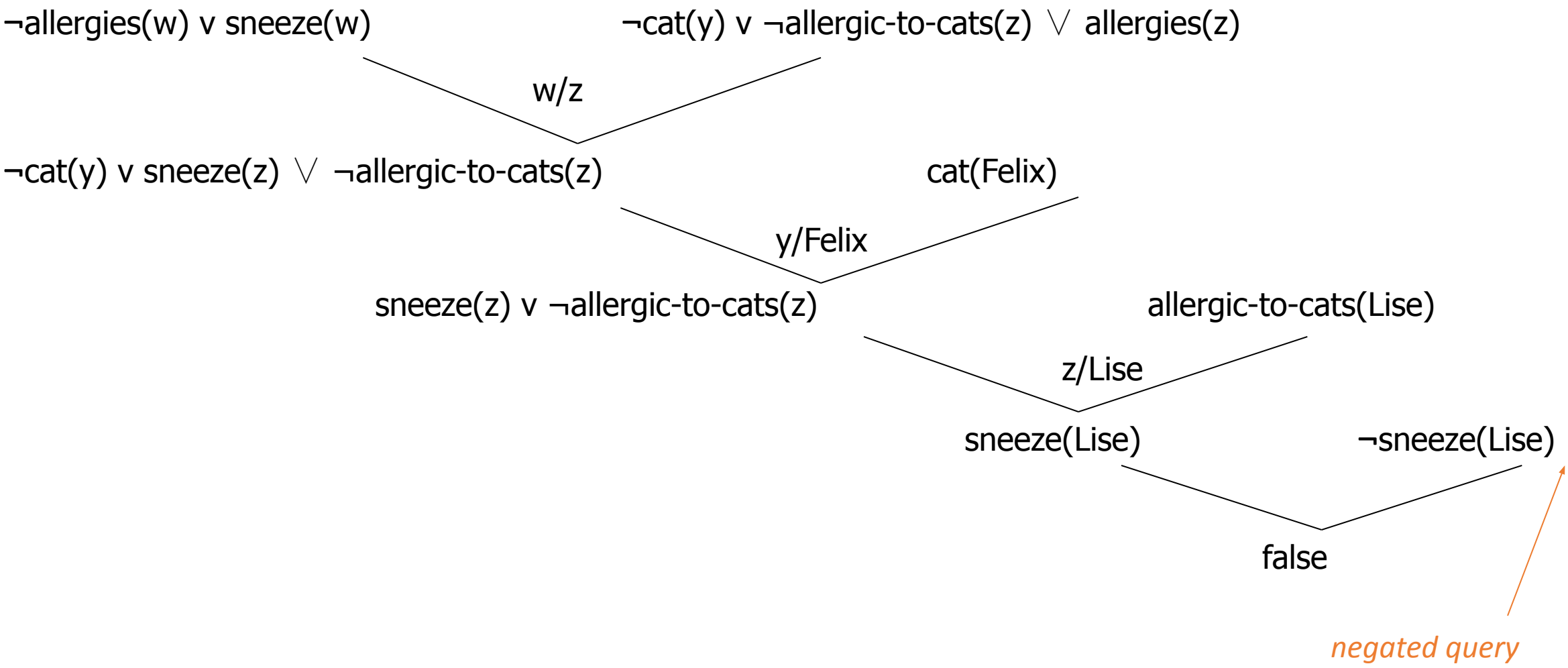
A resolution proof tree



Resolution refutation

- Given a consistent set of axioms KB and goal sentence Q, show that KB |= Q
- **Proof by contradiction:** Add ¬Q to KB and try to prove false.
  i.e., (KB |- Q) ↔ (KB ⋁ ¬Q |- False)
- Resolution is **refutation complete**: it can establish that a given sentence Q is entailed by KB, but can't (in general) be used to generate all logical consequences of a set of sentences
- Also, it cannot be used to prove that Q is **not entailed** by KB.
- Resolution **won't always give an answer** since entailment is only semidecidable
  - And you can't just run two proofs in parallel, one trying to prove Q and the other trying to prove ¬Q, since KB might not entail either one

Refutation resolution proof tree



We need answers to the following questions

- How to convert FOL sentences to conjunctive normal form (a.k.a. CNF, clause form): **normalization and skolemization**
- How to unify two argument lists, i.e., how to find their most general unifier (**mgu**) θ: **unification**
- How to determine which two clauses in KB should be resolved next (among all resolvable pairs of clauses) : **resolution (search) strategy**

Unification

- Unification is a **"pattern-matching"** procedure
  - Takes two atomic sentences, called literals, as input
  - Returns "Failure" if they do not match and a substitution list, θ, if they do
- That is, *unify(p,q) = θ* means *subst(θ, p) = subst(θ, q)* for two atomic sentences, *p* and *q*
- **θ** is called the **most general unifier** (mgu)
- All variables in the given two literals are implicitly universally quantified
- To make literals match, replace (universally quantified) variables by terms

Unification algorithm

```
procedure unify(p, q, θ)
    Scan p and q left-to-right and find the first corresponding
      terms where p and q "disagree" (i.e., p and q not equal)
    If there is no disagreement, return θ  (success!)
    Let r and s be the terms in p and q, respectively,
      where disagreement first occurs
    If variable(r) then {
      Let θ = union(θ, {r/s})
      Return unify(subst(θ, p), subst(θ, q), θ)
    } else if variable(s) then {
      Let θ = union(θ, {s/r})
      Return unify(subst(θ, p), subst(θ, q), θ)
    } else return "Failure"
  end
```

Unification: Remarks

- *Unify* is a linear-time algorithm that returns the most general unifier (mgu), i.e., the shortest-length substitution list that makes the two literals match.
- In general, there is not a **unique** minimum-length substitution list, but unify returns one of minimum length
- A variable can never be replaced by a term containing that variable

  Example: x/f(x) is illegal.
- This "occurs check" should be done in the above pseudo-code before making the recursive calls

Unification examples

- Example:
  - parents(x, father(x), mother(Bill))
  - parents(Bill, father(Bill), y)
  - {x/Bill, y/mother(Bill)}
- Example:
  - parents(x, father(x), mother(Bill))
  - parents(Bill, father(y), z)
  - {x/Bill, y/Bill, z/mother(Bill)}
- Example:
  - parents(x, father(x), mother(Jane))
  - parents(Bill, father(y), mother(y))
  - Failure

Resolution example

Practice example : *Did Curiosity kill the cat*

- Jack owns a dog. Every dog owner is an animal lover. No animal lover kills an animal. Either Jack or Curiosity killed the cat, who is named Tuna. Did Curiosity kill the cat?
- These can be represented as follows:

A. $(\exists x)\ Dog(x) \land Owns(Jack,x)$

B. $(\forall x)\ ((\exists y)\ Dog(y) \land Owns(x, y)) \rightarrow AnimalLover(x)$

C. $(\forall x)\ AnimalLover(x) \rightarrow ((\forall y)\ Animal(y) \rightarrow \neg Kills(x,y))$

D. $Kills(Jack,Tuna) \lor Kills(Curiosity,Tuna)$

E. $Cat(Tuna)$

F. $(\forall x)\ Cat(x) \rightarrow Animal(x)$ ⟵ GOAL

G. Kills(Curiosity, Tuna)

- **Convert to clause form**

  A1. (Dog(D))

  A2. (Owns(Jack,D))

  B. (¬Dog(y), ¬Owns(x, y), AnimalLover(x))

  C. (¬AnimalLover(a), ¬Animal(b), ¬Kills(a,b))

  D. (Kills(Jack,Tuna), Kills(Curiosity,Tuna))

  E. Cat(Tuna)

  F. (¬Cat(z), Animal(z))

- **Add the negation of query:**

  ¬G: (¬Kills(Curiosity, Tuna))

D is a skolem constant

- **The resolution refutation proof**

R1: ¬G, D, {} (Kills(Jack, Tuna))

R2: R1, C, {a/Jack, b/Tuna} (~AnimalLover(Jack), ~Animal(Tuna))

R3: R2, B, {x/Jack} (~Dog(y), ~Owns(Jack, y), ~Animal(Tuna))

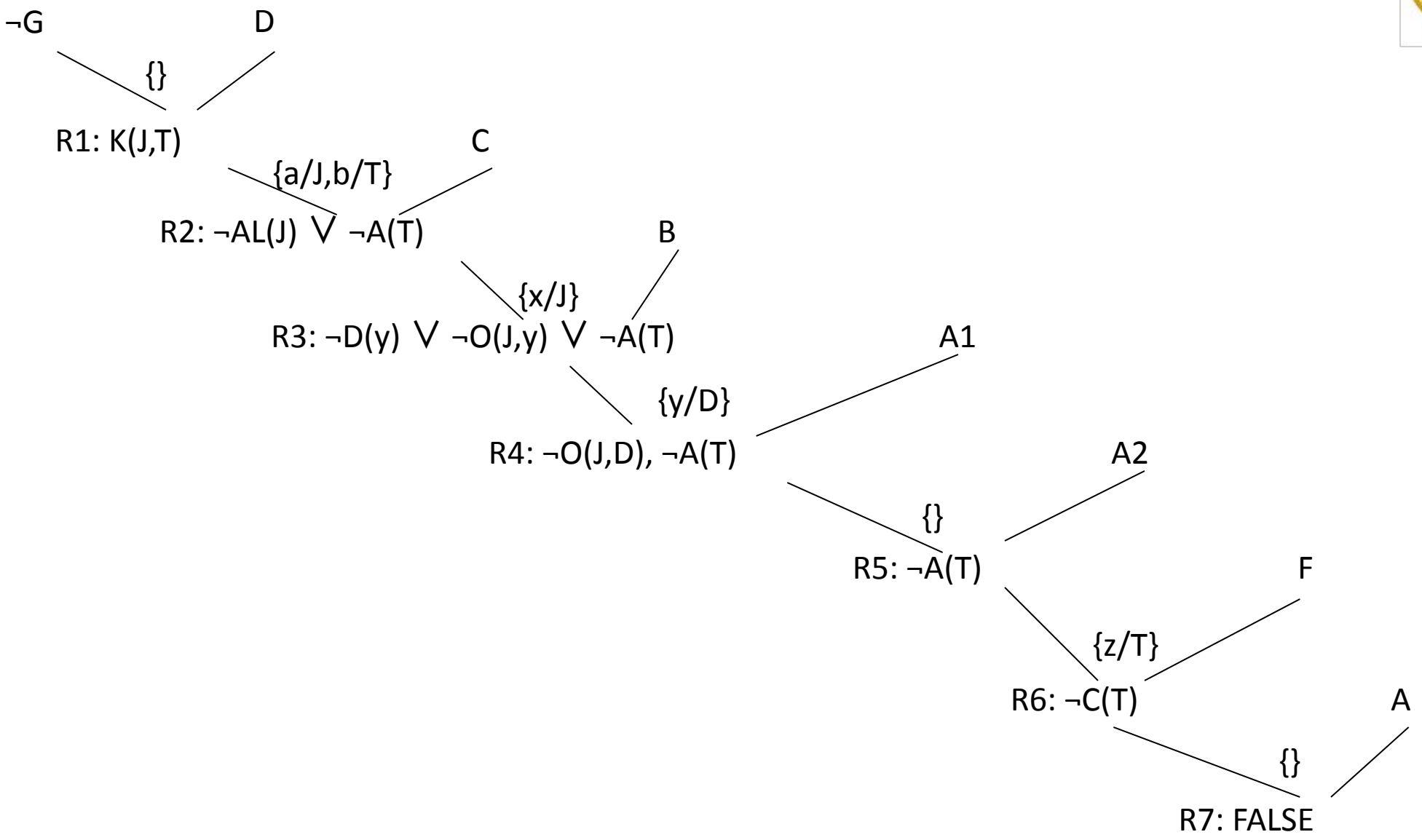
R4: R3, A1, {y/D} (~Owns(Jack, D), ~Animal(Tuna))

R5: R4, A2, {} (~Animal(Tuna))

R6: R5, F, {z/Tuna} (~Cat(Tuna))

R7: R6, E, {} FALSE

- **The proof tree**



Knowledge and Reasoning
Table of Contents

- Knowledge and reasoning-Approaches and issues of knowledge reasoning-Knowledge base agents
- Logic Basics-Logic-Propositional logic-syntax ,semantics and inferences-Propositional logic- Reasoning patterns
- Unification and Resolution
- Knowledge representation using rules-Knowledge representation using semantic nets
- Knowledge representation using frames-Inferences-
- Uncertain Knowledge and reasoning-Methods-Bayesian probability and belief network
- Probabilistic reasoning-Probabilistic reasoning over time-Probabilistic reasoning over time
- Other uncertain techniques-Data mining-Fuzzy logic-Dempster -shafer theory

Production Rules

- Condition-Action Pairs
  - IF this condition (or premise or antecedent) occurs, THEN some action (or result, or conclusion, or consequence) will (or should) occur
  - IF the traffic light is red AND you have stopped, THEN a right turn is OK

Production Rules

- Each production rule in a knowledge base represents an autonomous chunk of expertise
- When combined and fed to the inference engine, the set of rules behaves synergistically
- Rules can be viewed as a simulation of the cognitive behaviour of human experts
- Rules represent a model of actual human behaviour
- Predominant technique used in expert systems, often in conjunction with frames

Forms of Rules

- IF premise, THEN conclusion
  - IF your income is high, THEN your chance of being audited by the Inland Revenue is high
- Conclusion, IF premise
  - Your chance of being audited is high, IF your income is high

Forms of Rules

- Inclusion of ELSE
  - IF your income is high, OR your deductions are unusual, THEN your chance of being audited is high, OR ELSE your chance of being audited is low
- More complex rules
  - IF credit rating is high AND salary is more than £30,000, OR assets are more than £75,000, AND pay history is not "poor," THEN approve a loan up to £10,000, and list the loan in category "B."
- Action part may have more information: THEN "approve the loan" and "refer to an agent"

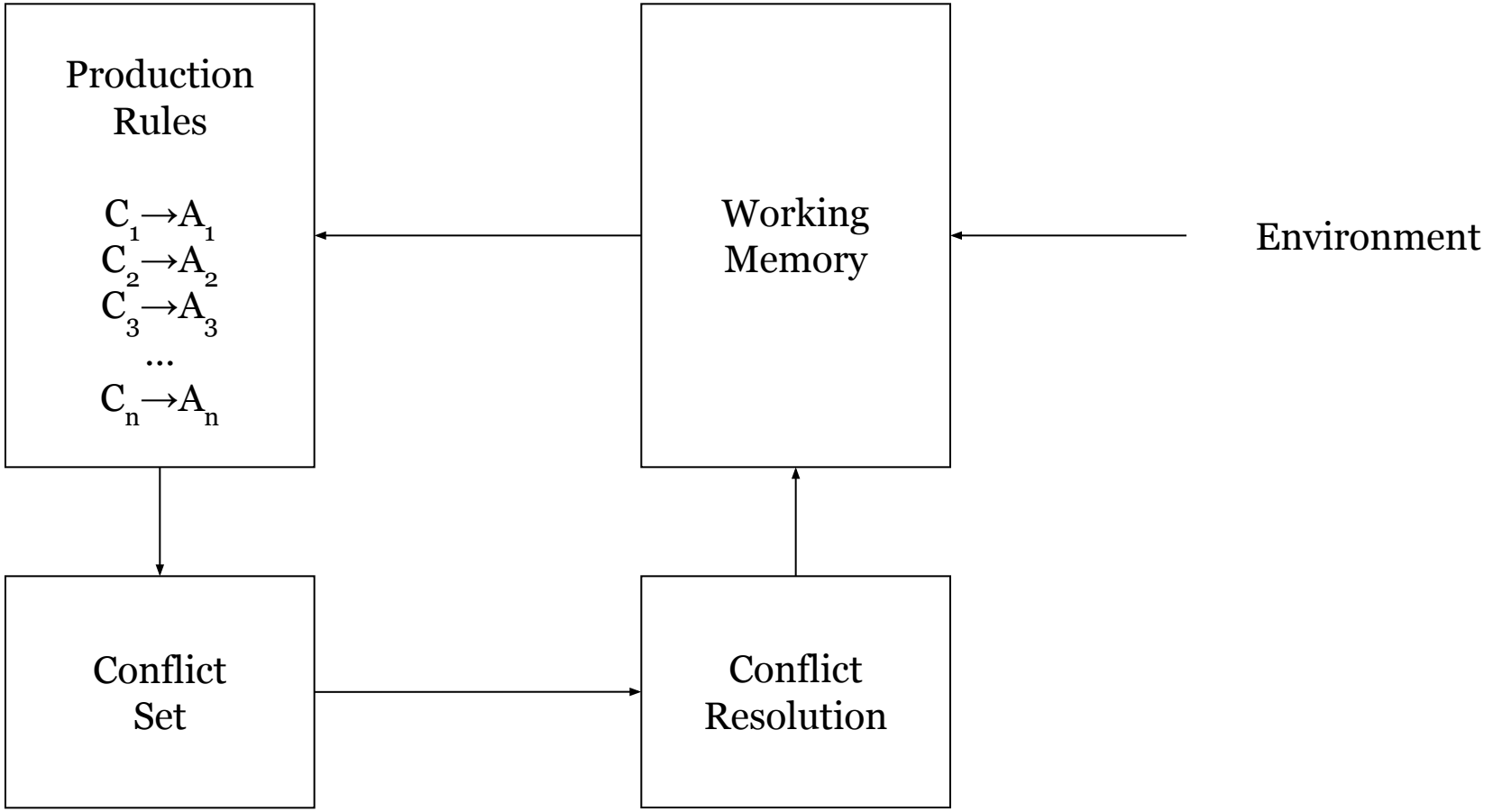
Characteristics of Rules

	First Part	Second Part
Names	Premise Antecedent Situation IF	Conclusion Consequence Action THEN
Nature	Conditions, similar to declarative knowledge	Resolutions, similar to procedural knowledge
Size	Can have many IFs	Usually only one conclusion
Statement	AND statements	All conditions must be true for a conclusion to be true
	OR statements	If any condition is true, the conclusion is true

Rule-based Inference

- Production rules are typically used as part of a *production system*
- Production systems provide pattern-directed control of the reasoning process
- Production systems have:
  - Productions: set of production rules
  - Working Memory (WM): description of current state of the world
  - Recognise-act cycle

Production Systems



Recognise-Act Cycle

- Patterns in WM matched against production rule conditions
- Matching (activated) rules form the **conflict set**
- One of the matching rules is selected (conflict resolution) and **fired**
  - Action of rule is performed
  - Contents of WM updated
- Cycle repeats with updated WM

Conflict Resolution

- Reasoning in a production system can be viewed as a type of search
  - Selection strategy for rules from the conflict set controls search
- Production system maintains the conflict set as an **agenda**
  - Ordered list of **activated rules** (those with their conditions satisfied) which have not yet been executed
  - Conflict resolution strategy determines where a newly-activated rule is inserted

Salience

- Rules may be given a precedence order by assigning a **salience value**
- Newly activated rules are placed in the agenda above all rules of lower salience, and below all rules with higher salience
  - Rule with higher salience are executed first
- Conflict resolution strategy applies between rules of the same salience
- If salience and the conflict resolution strategy can't determine which rule is to be executed next, a rule is chosen at random from the most highly ranked rules

Conflict Resolution Strategies

- Depth-first: newly activated rules placed above other rules in the agenda
- Breadth-first: newly activated rules placed below other rules
- Specificity: rules ordered by the number of conditions in the LHS (simple-first or complex-first)
- Least recently fired: fire the rule that was last fired the longest time ago
- Refraction: don't fire a rule unless the WM patterns that match its conditions have been modified
- Recency: rules ordered by the timestamps on the facts that match their conditions

Salience

- Salience facilitates the modularization of expert systems in which modules work at different levels of abstraction
- Over-use of salience can complicate a system
  - Explicit ordering to rule execution
  - Makes behaviour of modified systems less predictable
- Rule of thumb: if two rules have the same salience, are in the same module, and are activated concurrently, then the order in which they are executed should not matter

Common Types of Rules

- Knowledge rules, or declarative rules, state all the facts and relationships about a problem
- Inference rules, or procedural rules, advise on how to solve a problem, given that certain facts are known
- Inference rules contain rules about rules (metarules)
- Knowledge rules are stored in the knowledge base
- Inference rules become part of the inference engine

Major Advantages of Rules

- Easy to understand (natural form of knowledge)
- Easy to derive inference and explanations
- Easy to modify and maintain
- Easy to combine with uncertainty
- Rules are frequently independent

Major Limitations of Rules

- Complex knowledge requires many rules
- Search limitations in systems with many rules

Knowledge and Reasoning
Table of Contents

- Knowledge and reasoning-Approaches and issues of knowledge reasoning-Knowledge base agents
- Logic Basics-Logic-Propositional logic-syntax ,semantics and inferences-Propositional logic- Reasoning patterns
- Unification and Resolution
- Knowledge representation using rules-Knowledge representation using semantic nets
- Knowledge representation using frames-Inferences-
- Uncertain Knowledge and reasoning-Methods-Bayesian probability and belief network
- Probabilistic reasoning-Probabilistic reasoning over time-Probabilistic reasoning over time
- Other uncertain techniques-Data mining-Fuzzy logic-Dempster -shafer theory

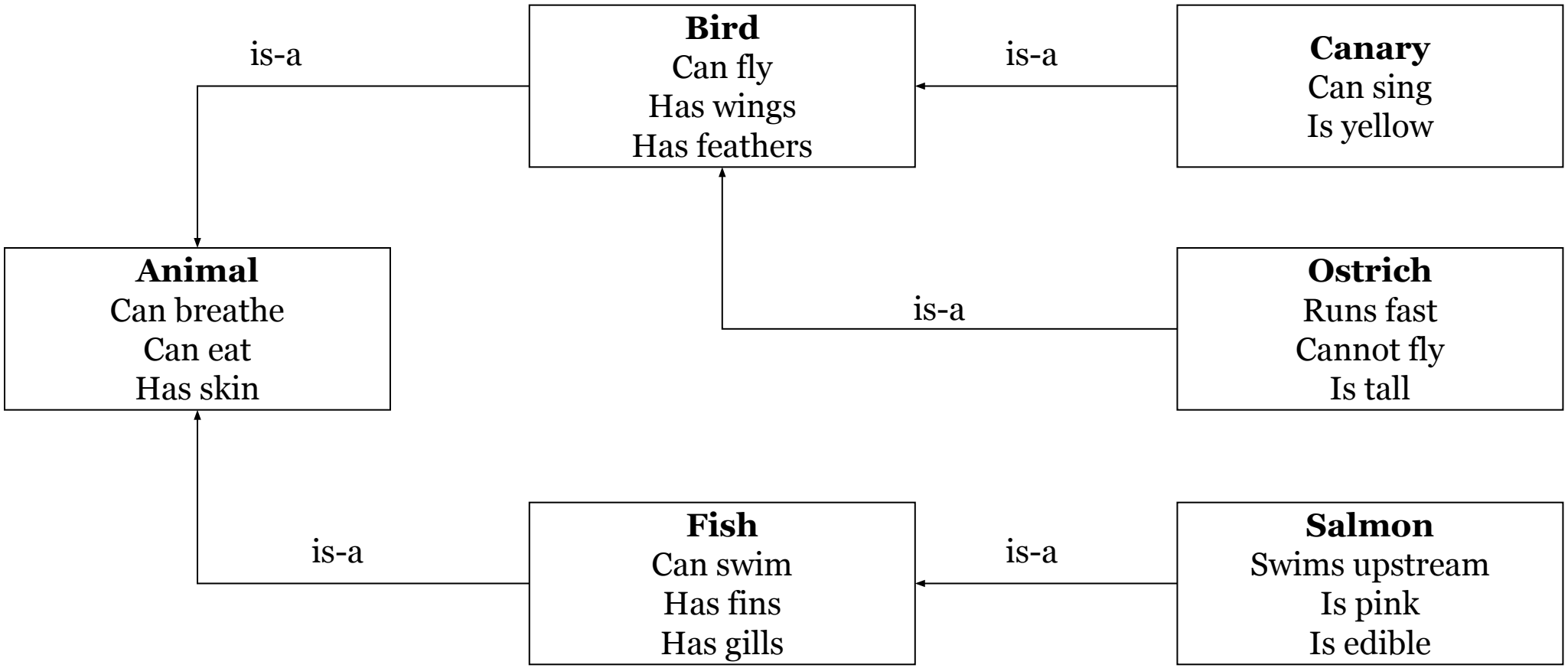
Semantic Networks

- A semantic network is a structure for representing knowledge as a pattern of interconnected nodes and arcs
- Nodes in the net represent concepts of entities, attributes, events, values
- Arcs in the network represent relationships that hold between the concepts

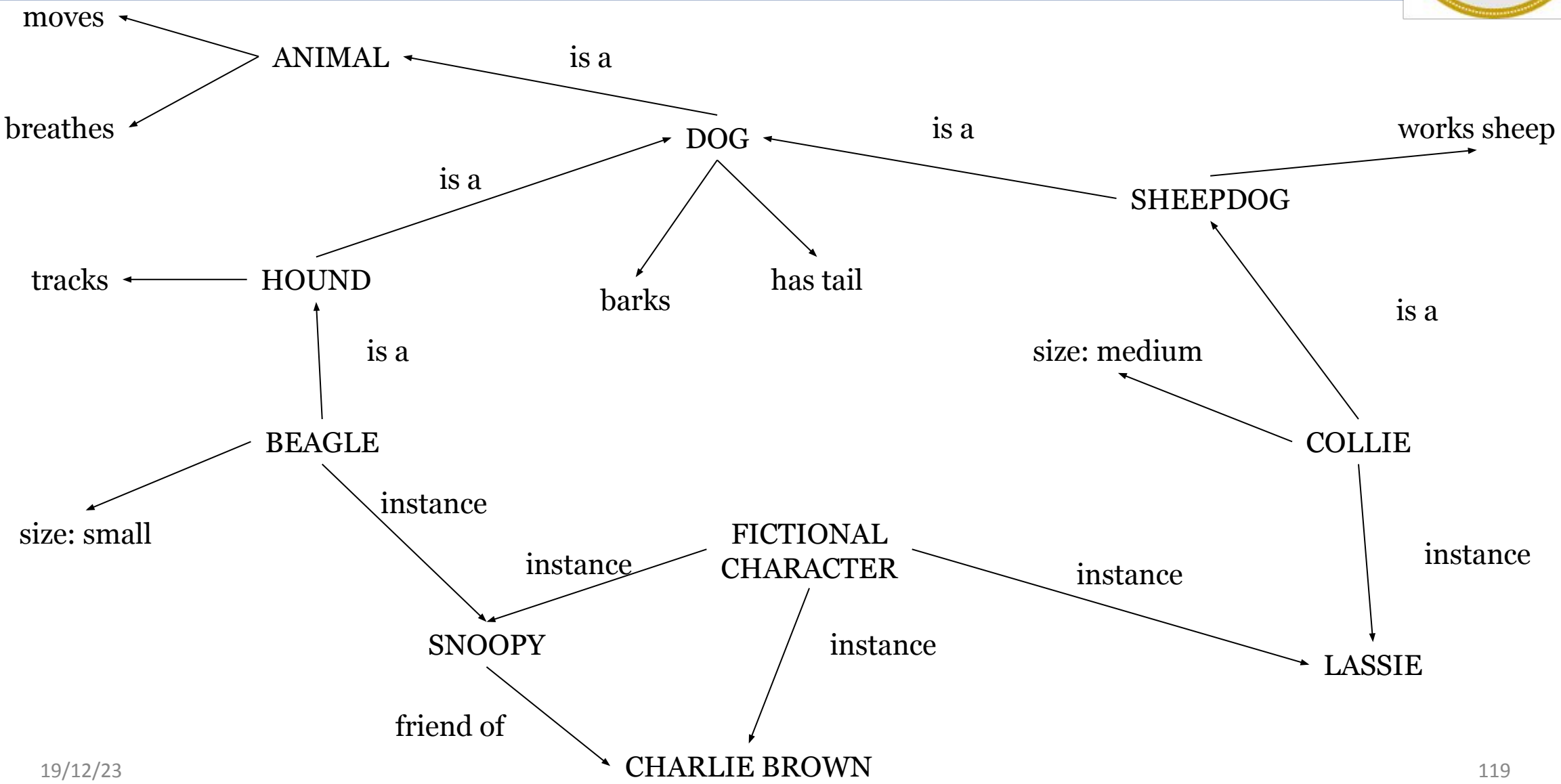
Semantic Networks

- Semantic networks can show inheritance
  - Relationship types – is-a, has-a
- Semantic Nets - visual representation of relationships
- Can be combined with other representation methods

Semantic Networks



Semantic Networks



Semantic Networks

What does or should a node represent?

- A class of objects?
- An instance of an class?
- The canonical instance of a class?
- The set of all instances of a class?

Semantic Networks

- Semantics of links that define new objects and links that relate existing objects, particularly those dealing with 'intrinsic' characteristics of a given object
- How does one deal with the problems of comparison between objects (or classes of objects) through their attributes?
  - Essentially the problem of comparing object instances
- What mechanisms are there are to handle quantification in semantic network formalisms?

Transitive inference, but…

- Clyde is an elephant, an elephant is a mammal: Clyde is a mammal.
- The US President is elected every 4 years, Bush is US President: Bush is elected every 4 years
- My car is a Ford, Ford is a car company: my car is a car company

Knowledge and Reasoning
Table of Contents

- Knowledge and reasoning-Approaches and issues of knowledge reasoning-Knowledge base agents
- Logic Basics-Logic-Propositional logic-syntax ,semantics and inferences-Propositional logic- Reasoning patterns
- Unification and Resolution
- Knowledge representation using rules-Knowledge representation using semantic nets
- Knowledge representation using frames-Inferences-
- Uncertain Knowledge and reasoning-Methods-Bayesian probability and belief network
- Probabilistic reasoning-Probabilistic reasoning over time-Probabilistic reasoning over time
- Other uncertain techniques-Data mining-Fuzzy logic-Dempster -shafer theory

Frames

- A *frame* is a knowledge representation formalism based on the idea of a frame of reference.
- A frame is a data structure that includes all the knowledge about a particular object
- Frames organised in a hierarchy Form of object-oriented programming for AI and ES.
- Each frame describes one object
- Special terminology

M. Minsky (1974) A Framework for Representing Knowledge, *MIT-AI Laboratory Memo 306*

Frames

- There are two types of frame:
  - Class Frame
  - Individual or Instance Frame
- A frame carries with it a set of *slots* that can represent objects that are normally associated with a subject of the frame.

Frames

- The slots can then point to other slots or frames. That gives frame systems the ability to carry out inheritance and simple kinds of data manipulation.
- The use of procedures - also called *demons* in the literature - helps in the incorporation of substantial amounts of procedural knowledge into a particular frame-oriented knowledge base

Frame-based model of semantic memory

- Knowledge is **organised** in a data structure
- Slots in structure are instantiated with particular values for a given instance of data
- …translation to OO terminology:
  - frames == classes or objects
  - slots == variables/methods

General Knowledge as Frames

DOG

Fixed
- legs: 4

Default
- diet: carnivorous
- sound: bark

Variable
- size:
- colour:

COLLIE

Fixed
- breed of: DOG
- type: sheepdog

Default
- size: 65cm

Variable
- colour:

General Knowledge as Frames


```
MAMMAL:
      subclass:    ANIMAL
      has_part:    head

ELEPHANT
      subclass:    MAMMAL
      colour:      grey
      size:              large

Nellie
      instance:    ELEPHANT
      likes:       apples
```

Logic underlies Frames

- $\forall$ x mammal(x) $\Rightarrow$ has_part(x, head)
- $\forall$ x elephant(x) $\Rightarrow$ mammal(x)

- elephant(clyde)
  $\therefore$
  mammal(clyde)
  has_part(clyde, head)

Logic underlies Frames

MAMMAL:
    subclass: ANIMAL
    has_part: head
    *furry: yes

ELEPHANT
    subclass: MAMMAL
    has_trunk: yes
    *colour: grey
    *size: large
    *furry: no

Clyde
    instance: ELEPHANT
    colour: pink
    owner: Fred

Nellie
    instance: ELEPHANT
    size: small

Frames (Contd.)

- Can represent subclass and instance relationships (both sometimes called ISA or “is a”)
- Properties (e.g. colour and size) can be referred to as slots and slot values (e.g. grey, large) as slot fillers
- Objects can inherit all properties of parent class (therefore Nellie is grey and large)
- But can inherit properties which are only typical (usually called default, here starred), and can be overridden
- For example, mammal is typically furry, but this is not so for an elephant

Frames (Contd.)

- Provide a concise, structural representation of knowledge in a natural manner
- Frame encompasses complex objects, entire situations or a management problem as a single entity
- Frame knowledge is partitioned into slots
- Slot can describe declarative knowledge or procedural knowledge
- Hierarchy of Frames: Inheritance

Capabilities of Frames

- Ability to clearly document information about a domain model; for example, a plant's machines and their associated attributes
- Related ability to constrain allowable values of an attribute
- Modularity of information, permitting ease of system expansion and maintenance
- More readable and consistent syntax for referencing domain objects in the rules

Capabilities of Frames

- Platform for building graphic interface with object graphics
- Mechanism to restrict the scope of facts considered during forward or backward chaining
- Access to a mechanism that supports the inheritance of information down a class hierarchy
- Used as underlying model in standards for accessing KBs (Open Knowledge Base Connectivity - OKBC)

Summary

- Frames have been used in conjunction with other, less well-grounded, representation formalisms, like production systems, when used to build to pre-operational or operational expert systems
- Frames cannot be used efficiently to organise ‘a whole computation

Knowledge and Reasoning
Table of Contents

- Knowledge and reasoning-Approaches and issues of knowledge reasoning-Knowledge base agents
- Logic Basics-Logic-Propositional logic-syntax ,semantics and inferences-Propositional logic- Reasoning patterns
- Unification and Resolution-Knowledge representation using rules-Knowledge representation using semantic nets
- Knowledge representation using frames-Inferences-
- Uncertain Knowledge and reasoning-Methods-Bayesian probability and belief network
- Probabilistic reasoning-Probabilistic reasoning over time
- Other uncertain techniques-Data mining-Fuzzy logic-Dempster -shafer theory

Types of Inference

- Deduction
- Induction
- Abduction

Deduction

- Deriving a conclusion from given axioms and facts
- Also called logical inference or truth preservation

Axiom – All kids are naughty

Fact/Premise – Riya is a kid

Conclusion – Riya is naughty

Induction

- Deriving general rule or axiom from background knowledge and observations
- Riya is a kid
- Riya is naughty

General axiom which is derived is:

Kids are naughty

Abduction

- A premise is derived from a known axiom and some observations
- All kids are naughty
- Riya is naughty

Inference

Riya is a kid

Knowledge and Reasoning
Table of Contents

- Knowledge and reasoning-Approaches and issues of knowledge reasoning-Knowledge base agents
- Logic Basics-Logic-Propositional logic-syntax ,semantics and inferences-Propositional logic- Reasoning patterns
- Unification and Resolution-Knowledge representation using rules-Knowledge representation using semantic nets
- Knowledge representation using frames-Inferences-
- Uncertain Knowledge and reasoning-Methods-Bayesian probability and belief network
- Probabilistic reasoning-Probabilistic reasoning over time
- Other uncertain techniques-Data mining-Fuzzy logic-Dempster -shafer theory

Uncertain knowledge and reasoning

- In real life, it is not always possible to determine the state of the environment as it might not be clear. Due to partially observable or non-deterministic environments, agents may need to handle uncertainty and deal with it.
- Uncertain data: Data that is missing, unreliable, inconsistent or noisy
- Uncertain knowledge: When the available knowledge has multiple causes leading to multiple effects or incomplete knowledge of causality in the domain
- Uncertain knowledge representation: The representations which provides a restricted model of the real system, or has limited expressiveness
- Inference: In case of incomplete or default reasoning methods, conclusions drawn might not be completely accurate. Let's understand this better with the help of an example.
- IF primary infection is bacteria cea
- AND site of infection is sterile
- AND entry point is gastrointestinal tract
- THEN organism is bacteriod (0.7).
- In such uncertain situations, the agent does not guarantee a solution but acts on its own assumptions and probabilities and gives some degree of belief that it will reach the required solution.

Uncertain knowledge and reasoning

- For example, In case of Medical diagnosis consider the rule Toothache = Cavity. This is not complete as not all patients having toothache have cavities. So we can write a more generalized rule Toothache = Cavity V Gum problems V Abscess… To make this rule complete, we will have to list all the possible causes of toothache. But this is not feasible due to the following rules:
- *Laziness- It will require a lot of effort to list the complete set of antecedents and consequents to make the rules complete.*
- *Theoretical ignorance- Medical science does not have complete theory for the domain*
- *Practical ignorance- It might not be practical that all tests have been or can be conducted for the patients.*
- Such uncertain situations can be dealt with using

Probability theory

Truth Maintenance systems

Fuzzy logic.

Uncertain knowledge and reasoning

Probability

- Probability is the degree of likeliness that an event will occur. It provides a certain degree of belief in case of uncertain situations. It is defined over a set of events U and assigns value P(e) i.e. probability of occurrence of event e in the range [0,1]. Here each sentence is labeled with a real number in the range of 0 to 1, 0 means the sentence is false and 1 means it is true.
- Conditional Probability or Posterior Probability is the probability of event A given that B has already occurred.
- P(A|B) = (P(B|A) * P(A)) / P(B)
- For example, P(It will rain tomorrow| It is raining today) represents conditional probability of it raining tomorrow as it is raining today.
- P(A|B) + P(NOT(A)|B) = 1
- Joint probability is the probability of 2 independent events happening simultaneously like rolling two dice or tossing two coins together. For example, Probability of getting 2 on one dice and 6 on the other is equal to 1/36. Joint probability has a wide use in various fields such as physics, astronomy, and comes into play when there are two independent events. The full joint probability distribution specifies the probability of each complete assignment of values to random variables.

Uncertain knowledge and reasoning

Bayes Theorem

- It is based on the principle that every pair of features being classified is independent of each other. It calculates probability P(A|B) where A is class of possible outcomes and B is given instance which has to be classified.
- P(A|B) = P(B|A) * P(A) / P(B)
- P(A|B) = Probability that A is happening, given that B has occurred (posterior probability)
- P(A) = prior probability of class
- P(B) = prior probability of predictor
- P(B|A) = likelihood

Uncertain knowledge and reasoning

Consider the following data. Depending on the weather (sunny, rainy or overcast), the children will play(Y) or not play(N).

Here, the total number of observations = 14

Probability that children will play given that weather is sunny :

P(Yes| Sunny) = P(Sunny | Yes) * P(Yes) / P(Sunny)

= 0.33 * 0.64 / 0.36

= 0.59

Weather	Play
Sunny	N
Overcast	Y
Rainy	Y
Sunny	Y
Sunny	Y
Overcast	Y
Rainy	N
Rainy	N
Sunny	Y
Rainy	Y
Sunny	N
Overcast	Y
Overcast	Y
Rainy	N

The Frequency Table will look like this.

Weather	Y	N
Overcast	0	4
Rainy	3	2
Sunny	2	3

The Likelihood Table will look like this.

Weather	Y	N	Total
Overcast	0	4	4 (=4/14)
Rainy	3	2	5 (=5/14)
Sunny	2	3	5 (=5/14)
Total	5 (=5/14)	9 (=9/14)	

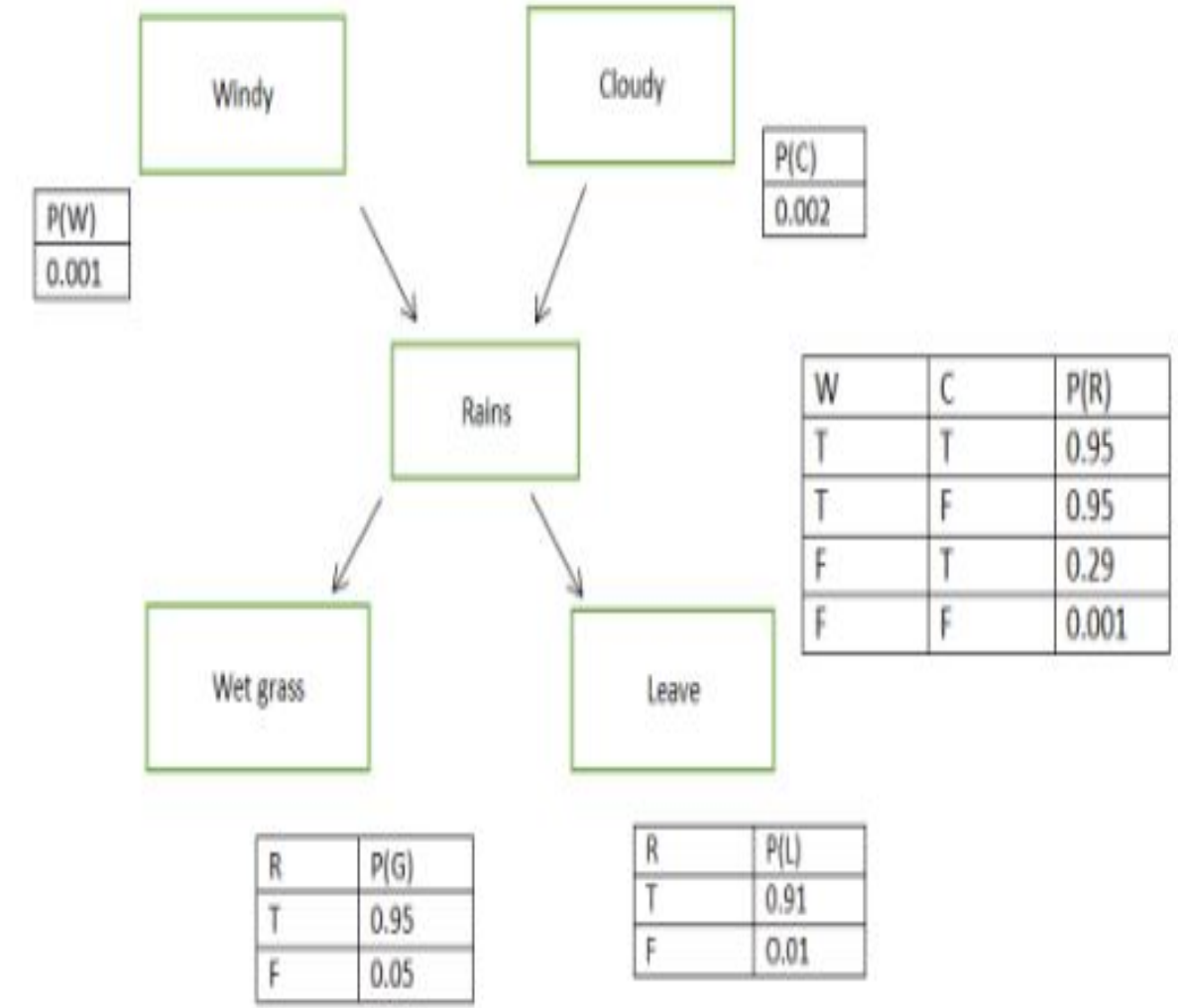
Uncertain knowledge and reasoning

It is a probabilistic graphical model for representing uncertain domain and to reason under uncertainty. It consists of nodes representing variables, arcs representing direct connections between them, called causal correlations. It represents conditional dependencies between random variables through a Directed Acyclic Graph (DAG). A belief network consist of:

1. A DAG with nodes labeled with variable names,

2. Domain for each random variable,

3. Set of conditional probability tables for each variable given its parents, including prior probability for nodes with no parents.



Uncertain knowledge and reasoning

- Let's have a look at the steps followed.

1. Identify nodes which are the random variables and the possible values they can have from the probability domain. The nodes can be boolean (True/ False), have ordered values or integral values.
2. Structure- It is used to represent causal relationships between the variables. Two nodes are connected if one affects or causes the other and the arc points towards the effect. For instance, if it is windy or cloudy, it rains. There is a direct link from Windy/Cloudy to Rains. Similarly, from Rains to Wet grass and Leave, i.e., if it rains, grass will be wet and leave is taken from work.

Uncertain knowledge and reasoning

3. Probability- Quantifying relationship between nodes. Conditional Probability:

- P(B|A) = P(A|B) * P(B) / P(A)
- Joint probability:

4. Markov property- Bayesian Belied Networks require assumption of Markov property, i.e., all direct dependencies are shown by using arcs. Here there is no direct connection between it being Cloudy and Taking a leave. But there is one via Rains. Belief Networks which have Markov property are also called independence maps.

Uncertain knowledge and reasoning

Inference in Belief Networks

- Bayesian Networks provide various types of representations of probability distribution over their variables. They can be conditioned over any subset of their variables, using any direction of reasoning.
- For example, one can perform diagnostic reasoning, i.e. when it Rains, one can update his belief about the grass being wet or if leave is taken from work. In this case reasoning occurs in the opposite direction to the network arcs. Or one can perform predictive reasoning, i.e., reasoning from new information about causes to new beliefs about effects, following direction of the arcs. For example, if the grass is already wet, then the user knows that it has rained and it might have been cloudy or windy. Another form of reasoning involves reasoning about mutual causes of a common effect. This is called inter causal reasoning.
- There are two possible causes of an effect, represented in the form of a 'V'. For example, the common effect 'Rains' can be caused by two reasons 'Windy' and 'Cloudy.' Initially, the two causes are independent of each other but if it rains, it will increase the probability of both the causes. Assume that we know it was windy. This information explains the reasons for the rainfall and lowers probability that it was cloudy.

Thank you

Unit III

Adversarial search Methods-Game playing-Important concepts

- Adversarial search: Search based on Game theory- Agents- Competitive environment
- ***According to game theory, a game is played between two players. To complete the game, one has to win the game and the other looses automatically.'***
- Such Conflicting goal- adversarial search
- Game playing technique- Those games- Human Intelligence and Logic factor- Excluding other factors like Luck factor
  - Tic-Tac-Toe, Checkers, Chess – Only mind works, no luck works

Adversarial search Methods-Game playing-Important concepts

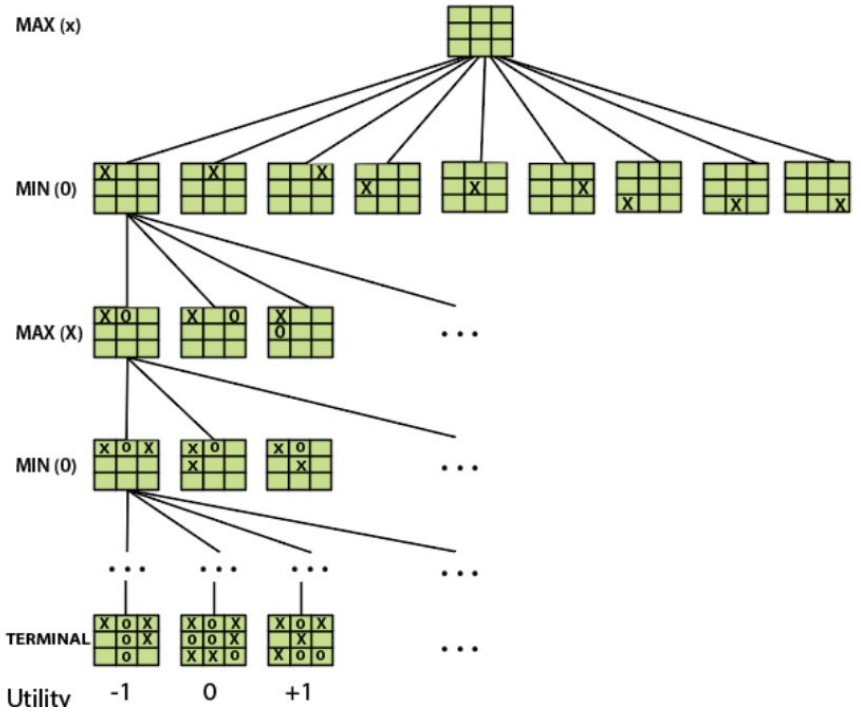
We are opponents- I win, you loose.

- **Techniques required to get the best optimal solution (Choose Algorithms for best optimal solution within limited time)**
  - **Pruning:** A technique which allows ignoring the unwanted portions of a search tree which make no difference in its final result.
  - **Heuristic Evaluation Function:** It allows to approximate the cost value at each level of the search tree, before reaching the goal node.

Game playing and knowledge structure- Elements of Game Playing search

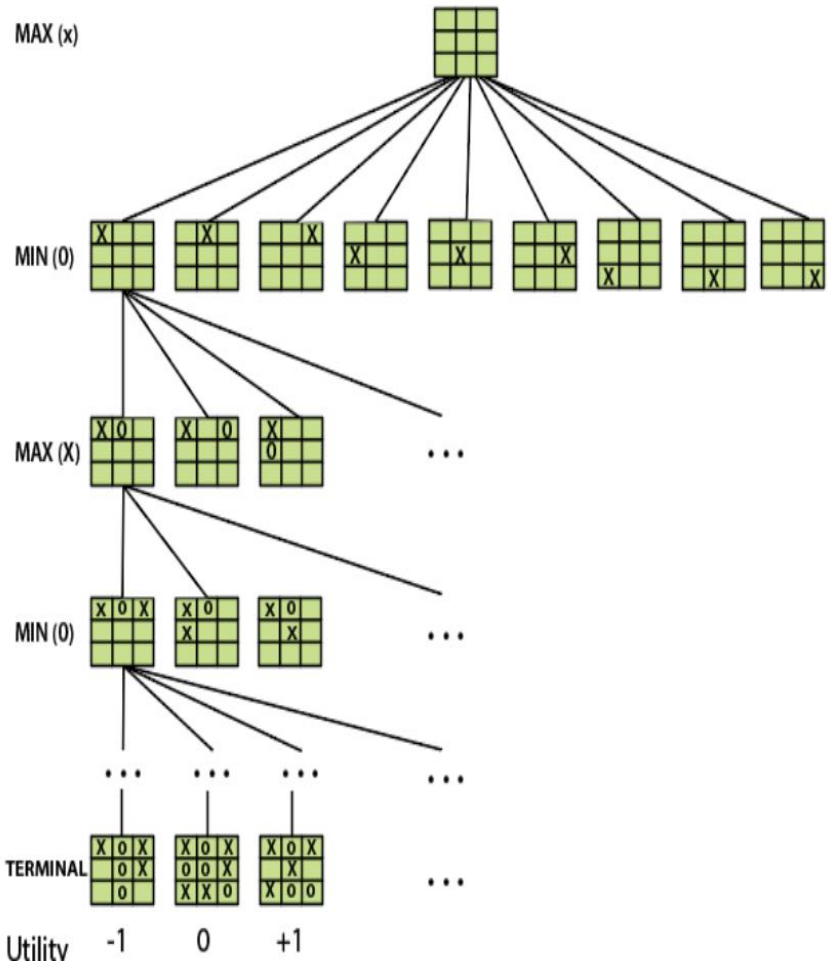
- To play a game, we use a game tree to know all the possible choices and to pick the best one out. There are following elements of a game-playing:
- **S_0:** It is the initial state from where a game begins.
- **PLAYER (s):** It defines which player is having the current turn to make a move in the state.
- **ACTIONS (s):** It defines the set of legal moves to be used in a state.
- **RESULT (s, a):** It is a transition model which defines the result of a move.
- **TERMINAL-TEST (s)**: It defines that the game has ended and returns true.
- **UTILITY (s,p):** It defines the final value with which the game has ended. This function is also known as **Objective function** or **Payoff function**. The price which the winner will get i.e.
- **(-1):** If the PLAYER loses.
- **(+1):** If the PLAYER wins.
- **(0):** If there is a draw between the PLAYERS.

- ***For example**, in **chess, tic-tac-toe,** we have two or three possible outcomes. Either to win, to lose, or to draw the match with values **+1,-1 or 0.***
- ***Game Tree for Tic-Tac-Toe***
  - Node: Game states, Edges: Moves taken by players



Game playing and knowledge structure- Elements of Game Playing search

- **INITIAL STATE (S_0):** The top node in the game-tree represents the initial state in the tree and shows all the possible choice to pick out one.
- **PLAYER (s):** There are two players, **MAX and MIN**. **MAX** begins the game by picking one best move and place **X** in the empty square box.
- **ACTIONS (s):** Both the players can make moves in the empty boxes chance by chance.
- **RESULT (s, a):** The moves made by **MIN** and **MAX** will decide the outcome of the game.
- **TERMINAL-TEST(s):** When all the empty boxes will be filled, it will be the terminating state of the game.
- **UTILITY:** At the end, we will get to know who wins: **MAX** or **MIN,** and accordingly, the price will be given to them**.**



Game as a search problem

- **Types of algorithms in Adversarial search**
  - In a **normal search**, we follow a sequence of actions to reach the goal or to finish the game optimally. But in an **adversarial search**, the result depends on the players which will decide the result of the game. It is also obvious that the solution for the goal state will be an optimal solution because the player will try to win the game with the shortest path and under limited time.
    - **Minmax Algorithm**
    - **Alpha-beta Pruning**

Game Playing vs. Search

- Game vs. search problem
- "Unpredictable" opponent □ specifying a move for every possible opponent reply
- Time limits □ unlikely to find goal, must approximate

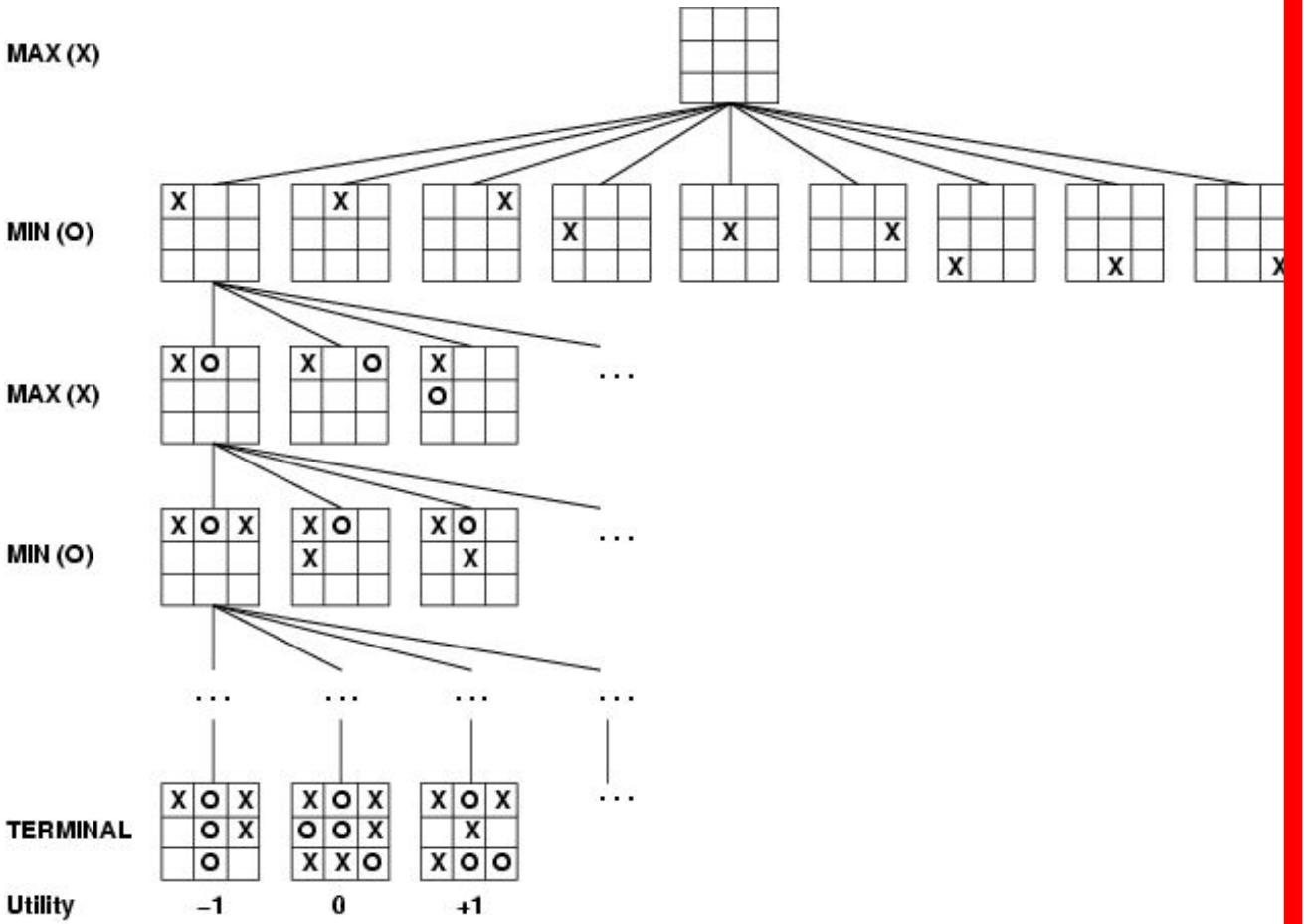
Game Playing

- Formal definition of a game:
  - Initial state
  - Successor function: returns list of *(move, state)* pairs
  - Terminal test: determines when game over

    Terminal states: states where game ends
  - Utility function (objective function or payoff function): gives numeric value for terminal states

We will consider games with 2 players (**Max and Min**);
Max moves first.

Game Tree Example: Tic-Tac-Toe

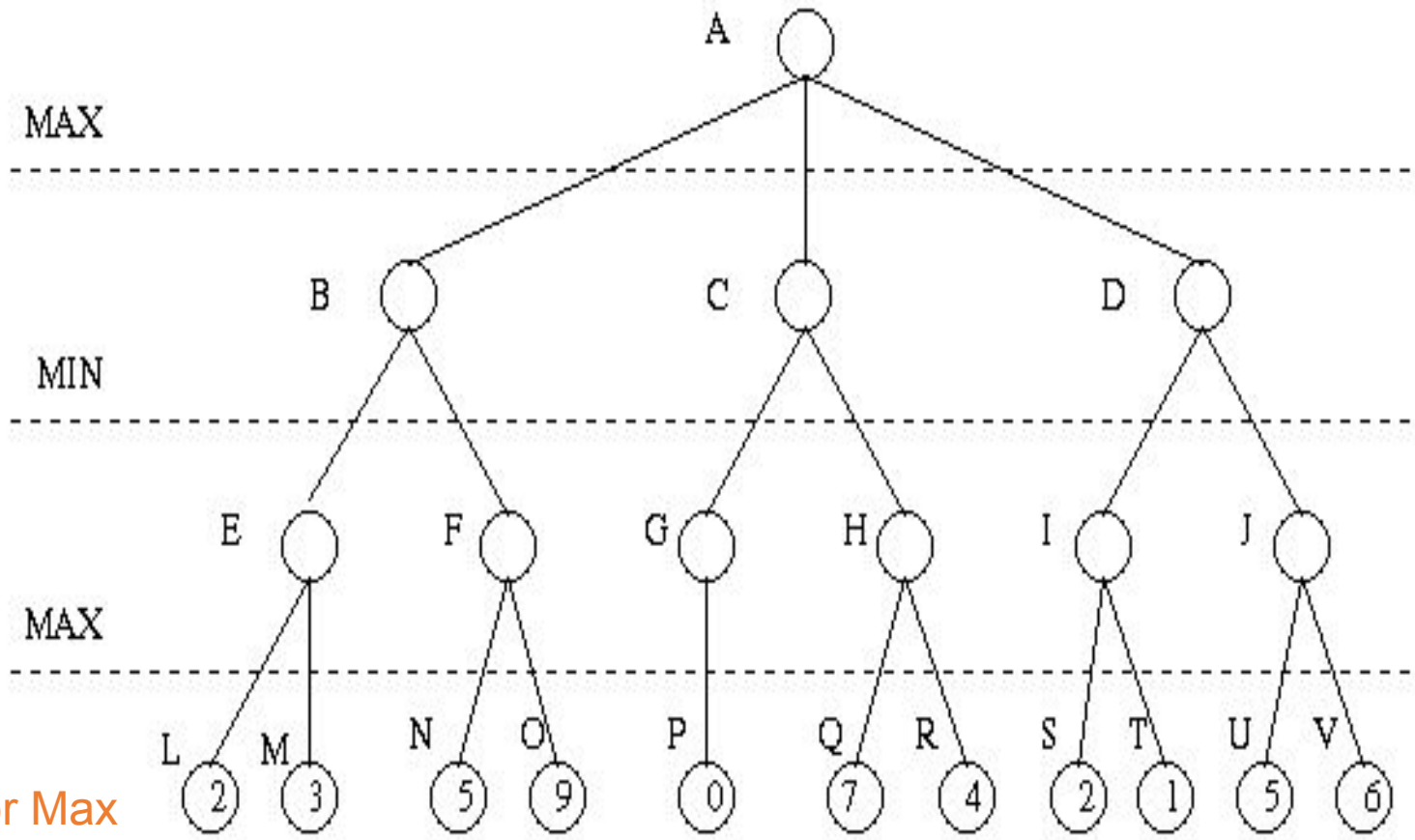


Tree from Max's perspective

Minimax Algorithm

- Minimax algorithm
  - Perfect play for deterministic, 2-player game
  - Max tries to maximize its score
  - Min tries to minimize Max's score (Min)
  - Goal: move to position of highest minimax value
    - Identify best achievable payoff against best play

Minimax Algorithm



Payoff for Max

Minimax Rule

- Goal of game tree search: to determine **one move** for Max player that **maximizes** the **guaranteed payoff** for a given game tree for MAX

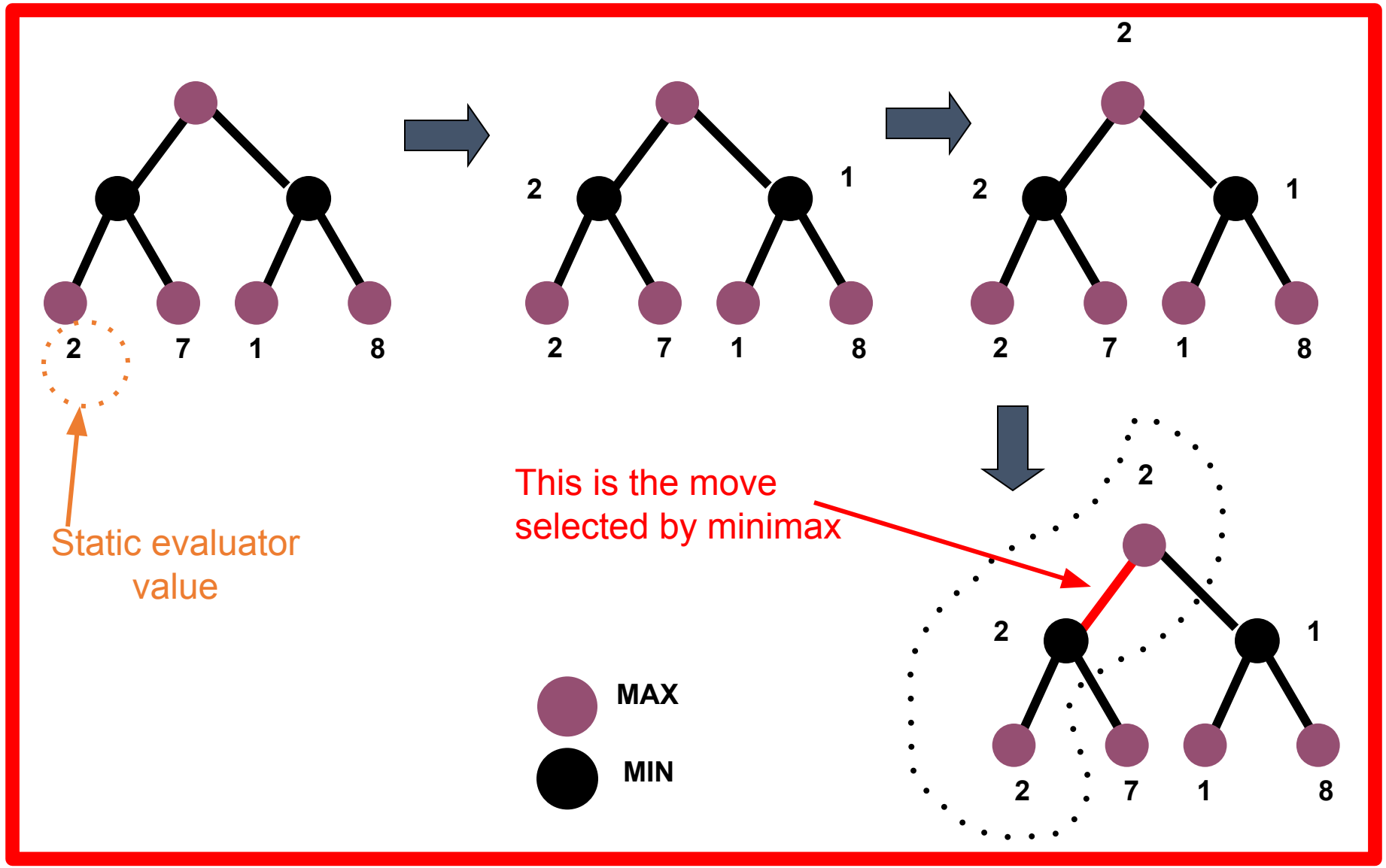
  Regardless of the moves the MIN will take
- The value of each node (Max and MIN) is determined by (back up from) the values of its children
- MAX plays the worst case scenario:

  Always assume MIN to take moves to maximize his pay-off (i.e., to minimize the pay-off of MAX)
- For a MAX node, the backed up value is the **maximum** of the values associated with its children
- For a MIN node, the backed up value is the **minimum** of the values associated with its children

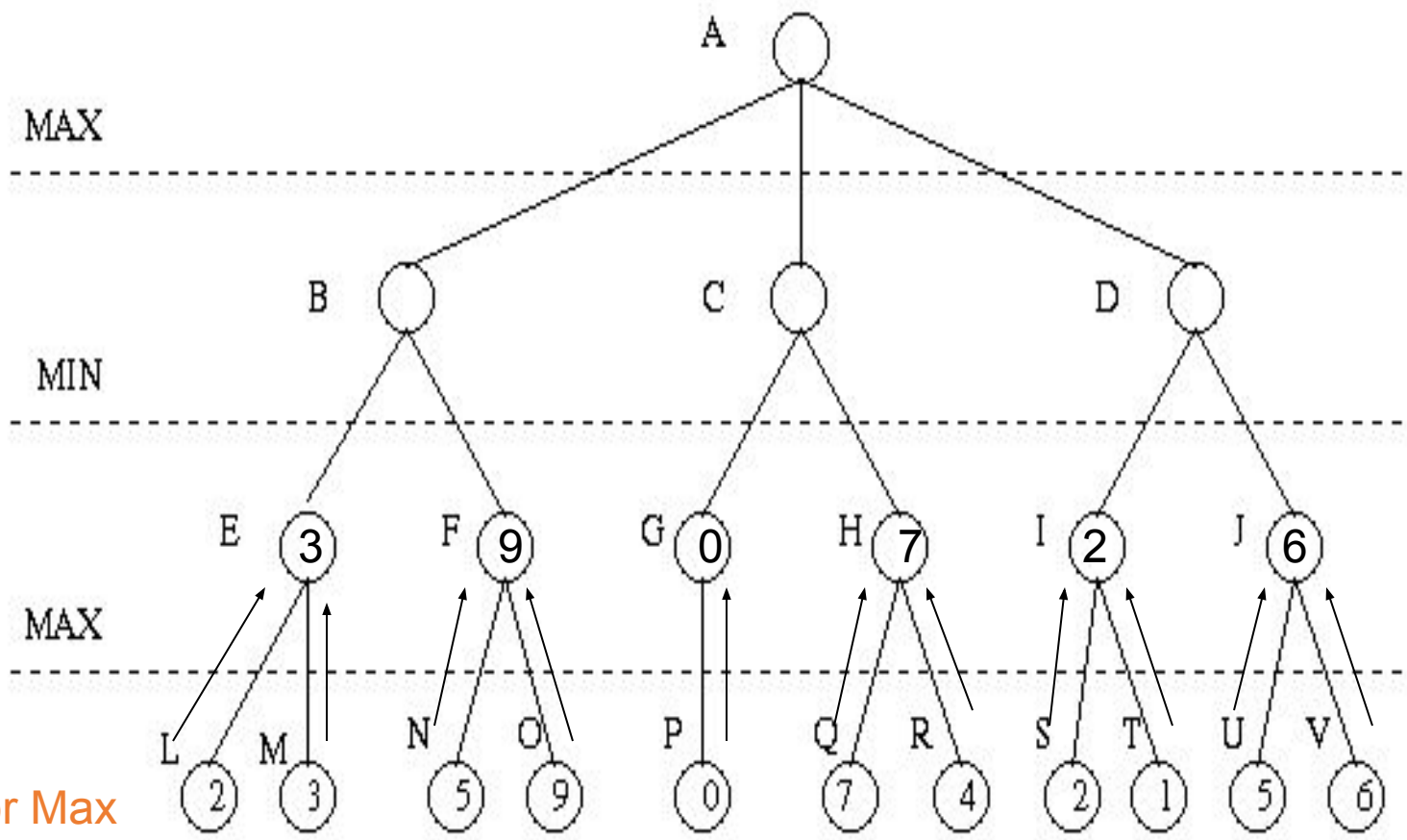
Minimax procedure

1. Create start node as a MAX node with current board configuration
2. Expand nodes down to some depth (i.e., ply) of lookahead in the game.
3. Apply the evaluation function at each of the leaf nodes
4. Obtain the “back up" values for each of the non-leaf nodes from its children by Minimax rule until a value is computed for the root node.
5. Pick the operator associated with the child node whose backed up value determined the value at the root as the move for MAX

Minimax Search

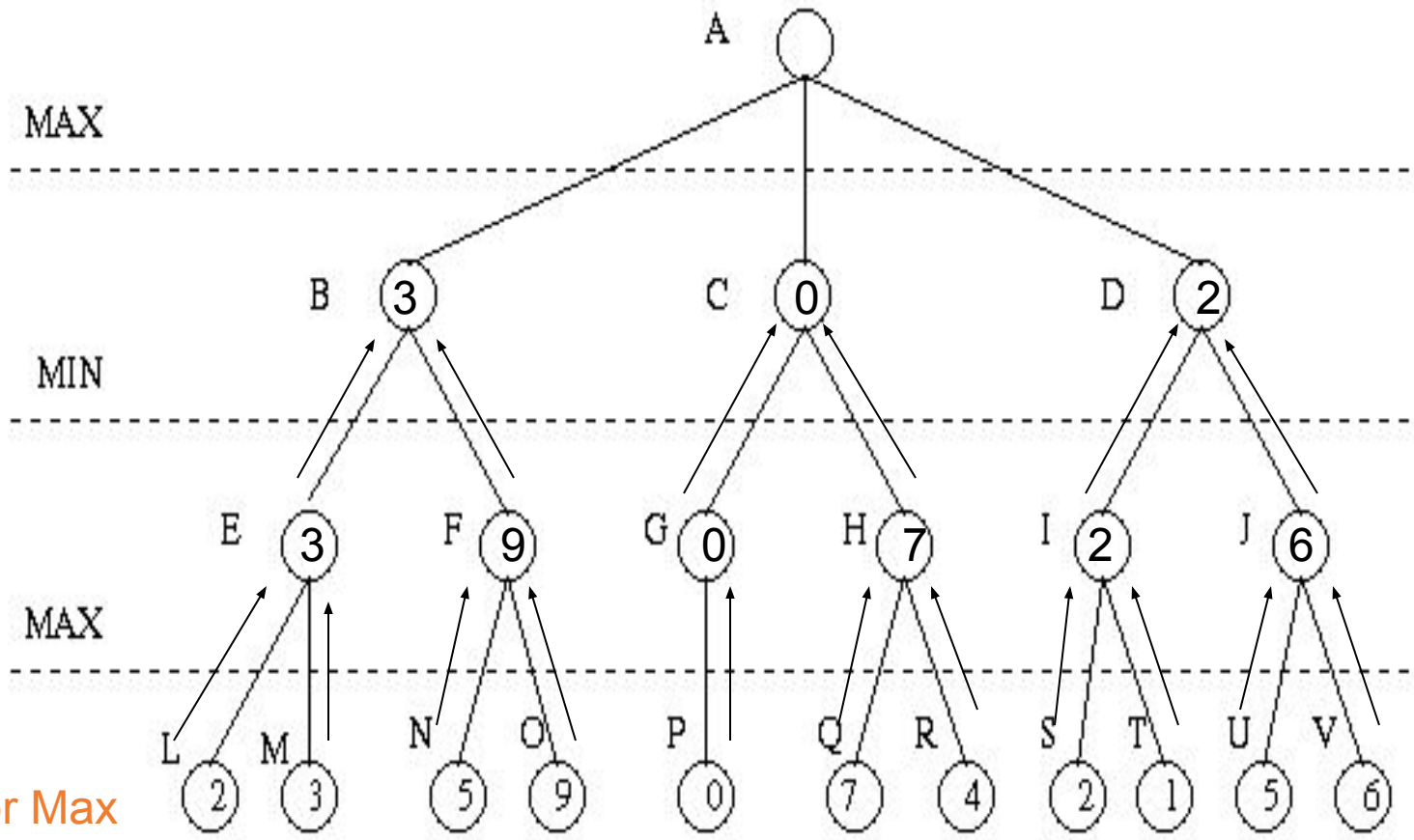


Minimax Algorithm (cont'd)



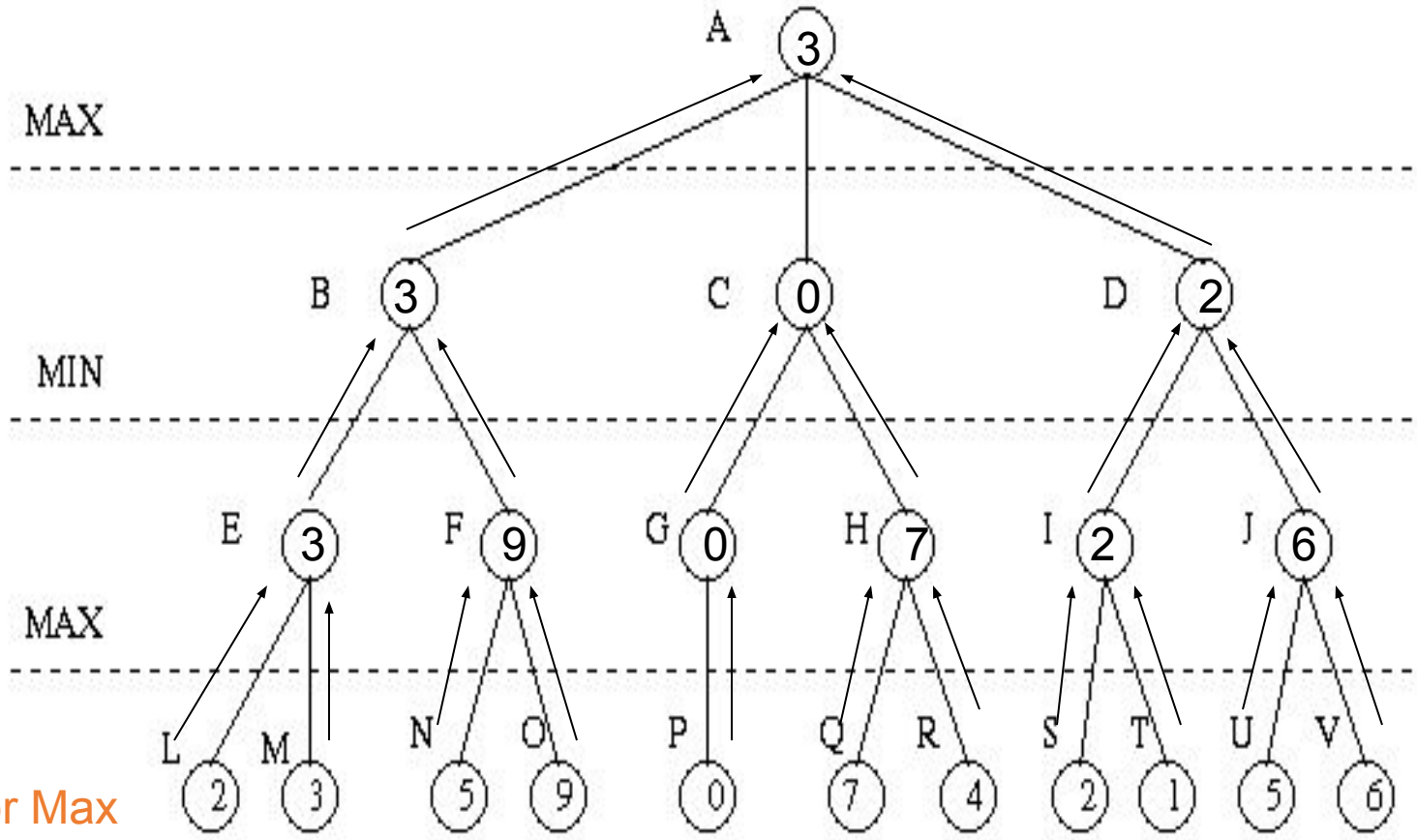
Payoff for Max

Minimax Algorithm (cont'd)



Payoff for Max

Minimax Algorithm (cont'd)



Payoff for Max

Minimax Algorithm (cont'd)

- Properties of minimax algorithm:
- Complete? Yes (if tree is finite)
  Optimal? Yes (against an optimal opponent)
  Time complexity? $O(b^m)$

- Space complexity? O(bm) (depth-first exploration, if it generates all successors at once)

m – maximum depth of tree; b branching factor

m – maximum depth of the tree; b – legal moves;

Minimax Algorithm

- Limitations
  - Not always feasible to traverse entire tree
  - Time limitations
- Key Improvement
  - Use evaluation function instead of utility
    - Evaluation function provides estimate of utility at given position

Unit 2 List of Topics

- Searching techniques – Uninformed search – General search Algorithm
- Uninformed search Methods – Breadth First Search
- Uninformed search Methods – Depth First Search
- Uninformed search Methods – Depth limited Search
- *Uniformed search Methods- Iterative Deepening search*
- *Bi-directional search*
- *Informed search- Generate and test, Best First search*
- *Informed search-A* Algorithm*
- *AO* search*
- *Local search Algorithms-Hill Climbing, Simulated Annealing*
- *Local Beam Search*
- *Genetic Algorithms*
- *Adversarial search Methods-Game playing-Important concepts*
- *Game playing and knowledge structure.*
- *Game as a search problem-Minimax Approach*
- *Minimax Algorithm*
- *Alpha beta pruning*
- *Game theory problems*

Alpha Beta Pruning

•Alpha-beta pruning is a modified version of the minimax algorithm. It is an optimization technique for the minimax algorithm.

•As we have seen in the minimax search algorithm that the number of game states it has to examine are exponential in depth of the tree.

•Since we cannot eliminate the exponent, but we can cut it to half.

•Hence there is a technique by which without checking each node of the game tree we can compute the correct minimax decision, and this technique is called **pruning**.

•This involves two threshold parameter Alpha and beta for future expansion, so it is called **alpha-beta pruning**. It is also called as **Alpha-Beta Algorithm**.

•Alpha-beta pruning can be applied at any depth of a tree, and sometimes it not only prune the tree leaves but also entire

Alpha Beta Pruning

- The two-parameter can be defined as:
  - **Alpha:** The best (highest-value) choice we have found so far at any point along the path of Maximizer. The initial value of alpha is **-∞**.
  - **Beta:** The best (lowest-value) choice we have found so far at any point along the path of Minimizer. The initial value of beta is **+∞**.
- The Alpha-beta pruning to a standard minimax algorithm returns the same move as the standard algorithm does, but it removes all the nodes which are not really affecting the final decision but making algorithm slow. Hence by pruning these nodes, it makes the algorithm fast.

Alpha Beta Pruning

Condition for Alpha-beta pruning:

- The main condition which required for alpha-beta pruning is:
  $\alpha >= \beta$

Key points about alpha-beta pruning:

- The Max player will only update the value of alpha.
- The Min player will only update the value of beta.
- While backtracking the tree, the node values will be passed to upper nodes instead of values of alpha and beta.
- We will only pass the alpha, beta values to the child nodes.

Alpha Beta Pruning

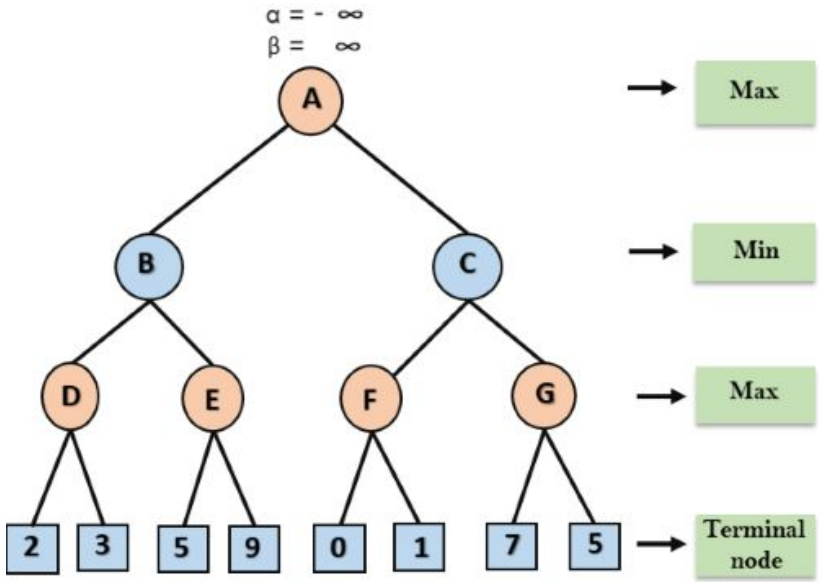
```
function minimax(node, depth, alpha, beta, maximizingPlayer) is
if depth ==0 or node is a terminal node then
return static evaluation of node

if MaximizingPlayer then      // for Maximizer Player
  maxEva= -infinity
  for each child of node do
  s= minimax(child, depth-1, alpha, beta, False)
 maxEva= max(maxEva, s)
 alpha= max(alpha, maxEva)
  if beta<=alpha
 break
 return maxEva
else                    // for Minimizer player
  minEva= +infinity
for each child of node do
 s= minimax(child, depth-1, alpha, beta, tru
      e)
  minEva= min(minEva, s)
  beta= min(beta, minEva)
   if beta<=alpha
 break
return minEva
```

Alpha Beta Pruning

Working of Alpha-Beta Pruning:

- Let's take an example of two-player search tree to understand the working of Alpha-beta pruning
- **Step 1:** At the first step the, Max player will start first move from node A where α= -∞ and β= +∞, these value of alpha and beta passed down to node B where again α= -∞ and β= +∞, and Node B passes the same value to its child D.

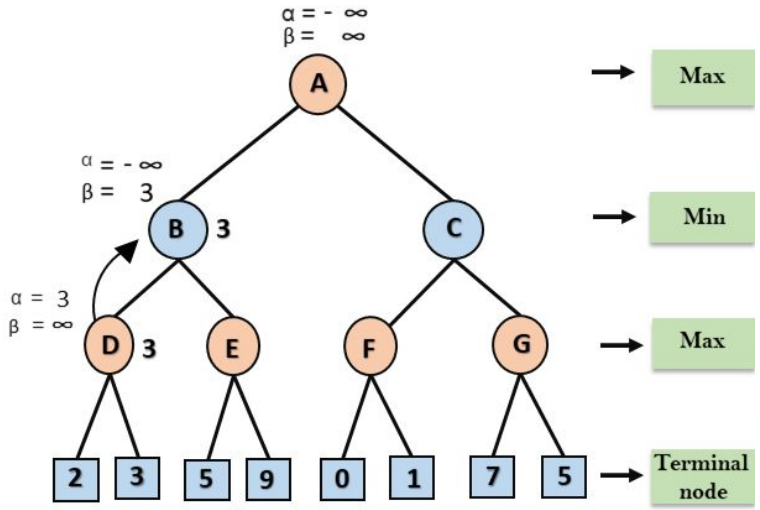


Alpha Beta Pruning

Step 2: At Node D, the value of α will be calculated as its turn for Max. The value of α is compared with firstly 2 and then 3, and the max (2, 3) = 3 will be the value of α at node D and node value will also 3.

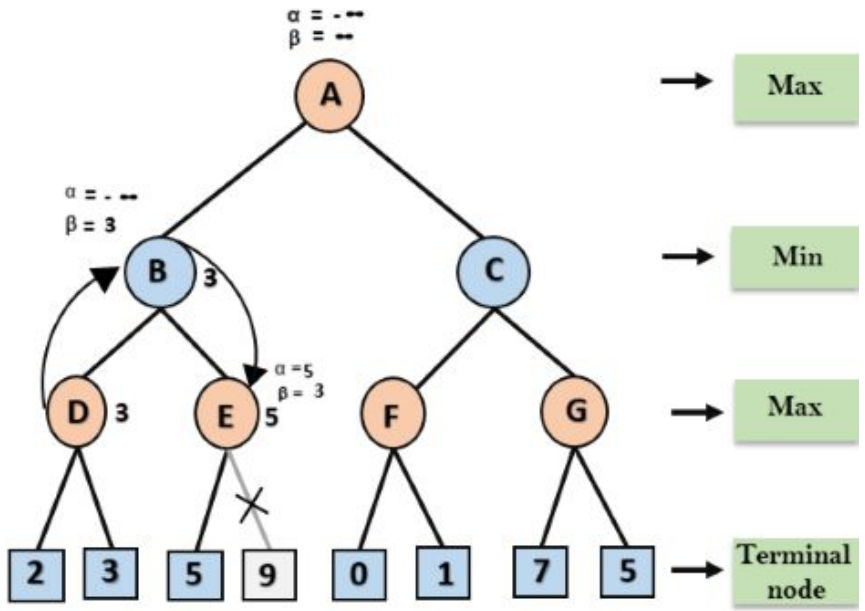
Step 3: Now algorithm backtrack to node B, where the value of β will change as this is a turn of Min, Now β= +∞, will compare with the available subsequent nodes value, i.e. min (∞, 3) = 3, hence at node B now α= -∞, and β= 3.

In the next step, algorithm traverse the next successor of Node B which is node E, and the values of α= -∞, and β= 3 will also be passed.



Alpha Beta Pruning

Step 4: At node E, Max will take its turn, and the value of alpha will change. The current value of alpha will be compared with 5, so max (-∞, 5) = 5, hence at node E α= 5 and β= 3, where α>=β, so the right successor of E will be pruned, and algorithm will not traverse it, and the value at node E will be 5.

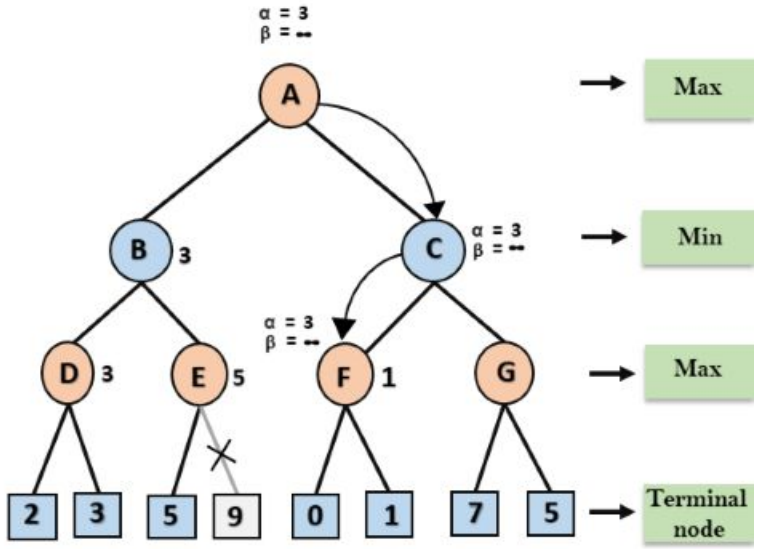


Alpha Beta Pruning

Step 5: At next step, algorithm again backtrack the tree, from node B to node A. At node A, the value of alpha will be changed the maximum available value is 3 as max (-∞, 3)= 3, and β= +∞, these two values now passes to right successor of A which is Node C.

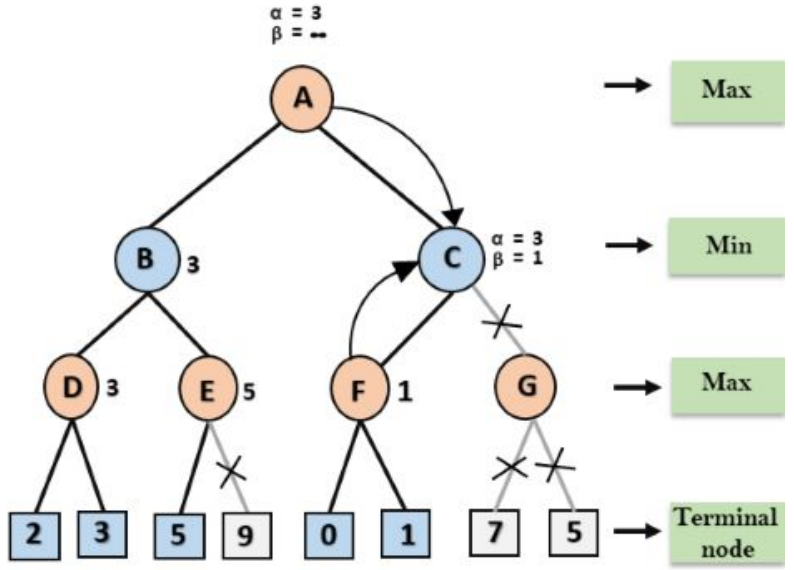
At node C, α=3 and β= +∞, and the same values will be passed on to node F.

Step 6: At node F, again the value of α will be compared with left child which is 0, and max(3,0)= 3, and then compared with right child which is 1, and max(3,1)= 3 still α remains 3, but the node value of F will become 1.



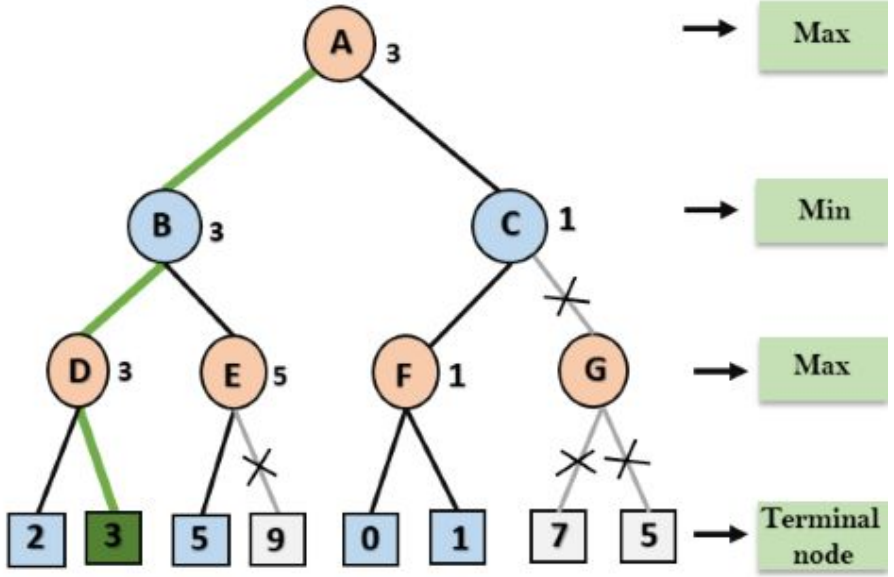
Alpha Beta Pruning

- **Step 7:** Node F returns the node value 1 to node C, at C α= 3 and β= +∞, here the value of beta will be changed, it will compare with 1 so min (∞, 1) = 1. Now at C, α=3 and β= 1, and again it satisfies the condition α>=β, so the next child of C which is G will be pruned, and the algorithm will not compute the entire sub-tree G.



Alpha Beta Pruning

Step 8: C now returns the value of 1 to A here the best value for A is max (3, 1) = 3. Following is the final game tree which is the showing the nodes which are computed and nodes which has never computed. Hence the optimal value for the maximizer is 3 for this example.



Alpha Beta Pruning

Move Ordering in Alpha-Beta pruning:

- The effectiveness of alpha-beta pruning is highly dependent on the order in which each node is examined. Move order is an important aspect of alpha-beta pruning.
- It can be of two types:
- **Worst ordering:** In some cases, alpha-beta pruning algorithm does not prune any of the leaves of the tree, and works exactly as minimax algorithm. In this case, it also consumes more time because of alpha-beta factors, such a move of pruning is called worst ordering. In this case, the best move occurs on the right side of the tree. The time complexity for such an order is $O(b^m)$.
- **Ideal ordering:** The ideal ordering for alpha-beta pruning occurs when lots of pruning happens in the tree, and best moves occur at the left side of the tree. We apply DFS hence it first search left of the tree and go deep twice as minimax algorithm in the same amount of time. Complexity in ideal ordering is $O(b^{m/2})$.

Unit 2 List of Topics

- Searching techniques – Uninformed search – General search Algorithm
- Uninformed search Methods – Breadth First Search
- Uninformed search Methods – Depth First Search
- Uninformed search Methods – Depth limited Search
- *Uniformed search Methods- Iterative Deepening search*
- *Bi-directional search*
- *Informed search- Generate and test, Best First search*
- *Informed search-A* Algorithm*
- *AO* search*
- *Local search Algorithms-Hill Climbing, Simulated Annealing*
- *Local Beam Search*
- *Genetic Algorithms*
- *Adversarial search Methods-Game playing-Important concepts*
- *Game playing and knowledge structure.*
- *Game as a search problem-Minimax Approach*
- *Minimax Algorithm*
- *Alpha beta pruning*
- *Game theory problems*

What is Game Theory?

- It deals with Bargaining.
- The whole process can be expressed Mathematically
- Based on Behavior Theory, has a more casual approach towards study of Human Behavior.
- It also considers how people Interact in Groups.

Game Theory Definition

•Theory of rational behavior for interactive decision problems.

• In a *game*, several agents strive to maximize their (expected) utility index by choosing particular courses of action, and each agent's final utility payoffs depend on the profile of courses of action chosen by *all* agents.

•The interactive situation, specified by the set of participants, the possible courses of action of each agent, and the set of all possible utility payoffs, is called a *game*;

• the agents 'playing' a game are called the *players*.

Definitions

Definition: *Zero-Sum Game* – A game in which the payoffs for the players always adds up to zero is called a zero-sum game.

Definition: *Maximin strategy* – If we determine the least possible payoff for each strategy, and choose the strategy for which this minimum payoff is largest, we have the maximin strategy.

A Further Definition

Definition: *Constant-sum and nonconstant-sum game* – If the payoffs to all players add up to the same constant, regardless which strategies they choose, then we have a constant-sum game. The constant may be zero or any other number, so zero-sum games are a class of constant-sum games. If the payoff does not add up to a constant, but varies depending on which strategies are chosen, then we have a non-constant sum game.

Game theory: assumptions

(1) Each decision maker has available to him two or more well-specified choices or sequences of choices.

(2) Every possible combination of plays available to the players leads to a well-defined end-state (win, loss, or draw) that terminates the game.

(3) A specified payoff for each player is associated with each end-state.

Game theory: assumptions (Cont)

(4) Each decision maker has perfect knowledge of the game and of his opposition.

(5) All decision makers are rational; that is, each player, given two alternatives, will select the one that yields him the greater payoff.

Rules, Strategies, Payoffs, and Equilibrium

- A **game** is a contest involving two or more decision makers, each of whom wants to win
  - **Game theory** is the study of how optimal strategies are formulated in conflict
- A player's payoff is the amount that the player wins or loses in a particular situation in a game.
- A players has a dominant strategy if that player's best strategy does not depend on what other players do.
- A two-person game involves two parties (X and Y)
  - A **zero-sum game** means that the sum of losses for one player must equal the sum of gains for the other. Thus, the overall sum is zero

Payoff Matrix - Store X

- Two competitors are planning radio and newspaper advertisements to increase their business. This is the payoff matrix for store X. A negative number means store Y has a positive payoff

		GAME PLAYER Y's STRATEGIES	
		Y_1 (Use radio)	Y_2 (Use newspaper)
GAME PLAYER X's STRATEGIES	X_1 (Use radio)	3	5
	X_2 (Use newspaper)	1	–2

Game Outcomes

Game Outcomes

STORE *X*'s STRATEGY	STORE *Y*'s STRATEGY	OUTCOME (% CHANGE IN MARKET SHARE)
X_1 (use radio)	Y_1 (use radio)	*X* wins 3 and *Y* loses 3
X_1 (use radio)	Y_2 (use newspaper)	*X* wins 5 and *Y* loses 5
X_2 (use newspaper)	Y_1 (use radio)	*X* wins 1 and *Y* loses 1
X_2 (use newspaper)	Y_2 (use newspaper)	*X* loses 2 and *Y* wins 2

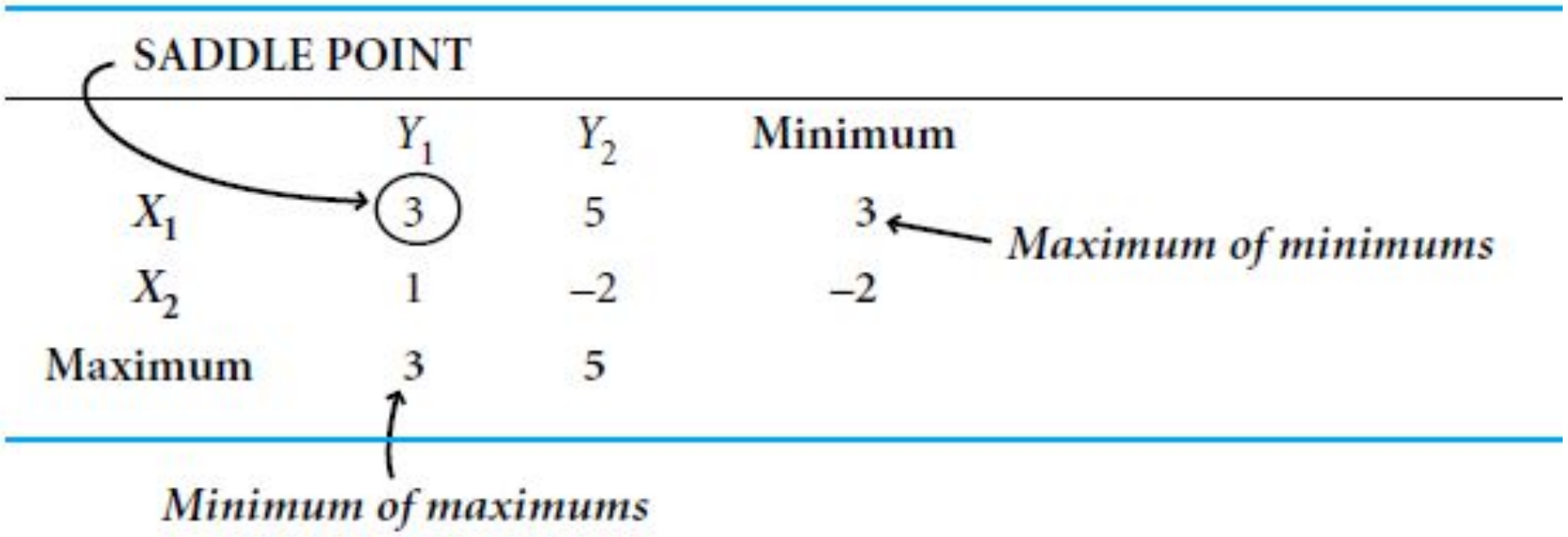
Minimax Criterion

- Look to the “cake cutting problem” to explain
  - Cutter – maximize the minimum the Chooser will leave him
  - Chooser – minimize the maximum the Cutter will get

Chooser □ Cutter	Choose bigger piece	Choose smaller piece
Cut cake as evenly as possible	Half the cake minus a crumb	Half the cake plus a crumb
Make one piece bigger than the other	Small piece	Big piece

Minimax Criterion

- If the upper and lower values are the same, the number is called the **value of the game** and an equilibrium or **saddle point condition** exists
- The value of a game is the average or expected game outcome if the game is played an infinite number of times
- A saddle point indicates that each player has a **pure strategy** i.e., the strategy is followed no matter what the opponent does



Saddle Point

- Von Neumann likened the solution point to the point in the middle of a saddle shaped mountain pass
  - It is, at the same time, the maximum elevation reached by a traveler going through the pass to get to the other side and the minimum elevation encountered by a mountain goat traveling the crest of the range

Pure Strategy - Minimax Criterion

		Player Y's Strategies		Minimum Row Number
		Y1	Y2	
Player X's strategies	X1	10	6	6
	X2	-12	2	-12
Maximum Column Number		10	6	

Mixed Strategy Game

- When there is no saddle point, players will play each strategy for a certain percentage of the time
- The most common way to solve a mixed strategy is to use the expected gain or loss approach
- A player plays each strategy a particular percentage of the time so that the expected value of the game does not depend upon what the opponent does

	Y1 P	Y2 1-P	Expected Gain
X1 Q	4	2	4P+2(1-P)
X2 1-Q	1	10	1P+10(1-p)
	4Q+1(1-Q)	2Q+10(1-q)	

Mixed Strategy Game
: Solving for P & Q

4P+2(1-P) = 1P+10(1-P)

or: P = 8/11 and 1-p = 3/11

Expected payoff:

1P+10(1-P)

=1(8/11)+10(3/11)

$EP_X = 3.46$

4Q+1(1-Q)=2Q+10(1-q)

or: Q=9/11 and 1-Q = 2/11

Expected payoff:

$EP_Y = 3.46$

Mixed Strategy Game : Example

- Using the solution procedure for a mixed strategy game, solve the following game

	Y_1	Y_2
X_1	4	2
X_2	0	10

Mixed Strategy Game

Example

- This game can be solved by setting up the mixed strategy table and developing the appropriate equations:

		Y_1	Y_2	
		P	$1 - P$	**Expected gain**
X_1	Q	4	2	$4P + 2(1 - P)$
X_1	$1 - Q$	1	10	$0P + 10(1 - P)$
Expected gain		$4Q + 0(1 - Q)$	$2Q + 10(1 - Q)$	

Mixed Strategy Game: Example

The equations for Q are

$$4Q + 0(1 - Q) = 2Q + 10(1 - Q)$$
$$4Q = 2Q + 10 - 10Q$$
$$12Q = 10 \text{ or } Q = {}^{10}/_{12} \text{ and } 1 - Q = {}^{2}/_{12}$$

The equations for P are

$$4P + 2(1 - P) = 0P + 10(1 - P)$$
$$4P + 2 - 2P = 10 - 10P$$
$$12P = 8 \text{ or } P = {}^{8}/_{12} \text{ and } 1 - P = {}^{4}/_{12}$$

Two-Person Zero-Sum and Constant-Sum Games

Two-person zero-sum and constant-sum games are played according to the following basic assumption:

Each player chooses a strategy that enables him/her to do the best he/she can, given that his/her opponent *knows the strategy he/she is following*.

A two-person zero-sum game has a saddle point if and only if

Max (row minimum) = min (column maximum)

all *all*

rows *columns*

(1)

Two-Person Zero-Sum and Constant-Sum Games (Cont)

If a two-person zero-sum or constant-sum game has a saddle point, the row player should choose any strategy (row) attaining the maximum on the right side of (1). The column player should choose any strategy (column) attaining the minimum on the right side of (1).
In general, we may use the following method to find the optimal strategies and value of two-person zero-sum or constant-sum game:

Step 1 Check for a saddle point. If the game has none, go on to step 2.

Two-Person Zero-Sum and Constant-Sum Games (Cont)

Step 2 Eliminate any of the row player's dominated strategies. Looking at the reduced matrix (dominated rows crossed out), eliminate any of the column player's dominated strategies and then those of the row player. Continue until no more dominated strategies can be found. Then proceed to step 3.

Step 3 If the game matrix is now 2 x 2, solve the game graphically. Otherwise, solve by using a linear programming method.

Zero Sum Games

- Game theory assumes that the decision maker and the opponent are rational, and that they subscribe to the maximin criterion as the decision rule for selecting their strategy
- This is often reasonable if when the other player is an opponent out to maximize his/her own gains, e.g. competitor for the same customers.
- Consider:

  Player 1 with three strategies S1, S2, and S3 and Player 2 with four strategies OP1, OP2, OP3, and OP4.

Zero Sum Games (Cont)

- The value 4 achieved by both players is called the *value* of the game
- The intersection of S2 and OP2 is called a *saddle point*. A game with a saddle point is also called a game with an *equilibrium solution*.
- At the saddle point, neither player can improve their payoff by switching strategies

Zero Sum Games- To do problem!

Let's take the following example: Two TV channels (1 and 2) are competing for an audience of 100 viewers. The rule of the game is to simultaneously announce the type of show the channels will broadcast. Given the payoff matrix below, what type of show should channel 1 air?

		Channel 1		
		Movie	Reality Show	Stand-up Comedy
Channel 2	Movie	37	17	62
	Reality Show	47	60	52
	Stand-up Comedy	40	16	72

Two-person zero-sum game – Dominance property

dominance method Steps (Rule)	
Step-1:	1. If all the elements of Column-i are greater than or equal to the corresponding elements of any other Column-j, then the Column-i is dominated by the Column-j and it is removed from the matrix. eg. If Column-2 $\geq$ Column-4, then remove Column-2
Step-2:	1. If all the elements of a Row-i are less than or equal to the corresponding elements of any other Row-j, then the Row-i is dominated by the Row-j and it is removed from the matrix. eg. If Row-3 $\leq$ Row-4, then remove Row-3
Step-3:	Again repeat Step-1 & Step-2, if any Row or Column is dominated, otherwise stop the procedure.

Two-person zero-sum game – Dominance property- To do problem!

Player A \ Player B	B1	B2	B3	B4
A1	3	5	4	2
A2	5	6	2	4
A3	2	1	4	0
A4	3	3	5	2

Solution

		Player B	
		$B3$	$B4$
Player A	$A2$	2	4
	$A4$	5	2

The Prisoner's Dilemma

- The prisoner's dilemma is a universal concept. Theorists now realize that prisoner's dilemmas occur in biology, psychology, sociology, economics, and law.
- The prisoner's dilemma is apt to turn up anywhere a conflict of interests exists -- and the conflict need not be among sentient beings.
- Study of the prisoner's dilemma has great power for explaining why animal and human societies are organized as they are. It is one of the great ideas of the twentieth century, simple enough for anyone to grasp and of fundamental importance (...).
- The prisoner's dilemma has become one of the premier philosophical and scientific issues of our time. It is tied to our very survival (W. Poundstone,1992, p. 9).

Prisoner's Dilemma

- Two members of a criminal gang are arrested and imprisoned.
  - They are placed under solitary confinement and have no chance of communicating with each other
- The district attorney would like to charge them with a recent major crime but has insufficient evidence
  - He has sufficient evidence to convict each of them of a lesser charge
  - If he obtains a confession from one or both the criminals, he can convict either or both on the major charge.

Prisoner's Dilemma

- The district attorney offers each the chance to turn state's evidence.
  - If only one prisoner turns state's evidence and testifies against his partner he will go free while the other will receive a 3 year sentence.
  - Each prisoner knows the other has the same offer
  - The catch is that if both turn state's evidence, they each receive a 2 year sentence
  - If both refuse, each will be imprisoned for 1 year on the lesser charge

A game is described by

- The number of players
- Their strategies and their turn
- Their payoffs (profits, utilities etc) at the outcomes of the game

Game Theory Definition

Payoff matrix

Normal- or strategic form

Player B

Player A	Left	Right
Top	3, 0	0, -4
Bottom	2, 4	-1, 3

Game Playing

How to solve a situation like this?

- The most simple case is where there is a optimal choice of strategy no matter what the other players do; *dominant strategies*.
- Explanation: For Player A it is always better to choose Top, for Player B it is always better to choose left.
- A *dominant strategy* is a strategy that is best no matter what the other player does.

Nash equilibrium

- If Player A's choice is optimal given Player B's choice, and B's choice is optimal given A's choice, a pair of strategies is a *Nash equilibrium*.
- When the other players' choice is revealed neither player like to change her behavior.
- If a set of strategies are best responses to each other, the strategy set is a *Nash equilibrium*.

Payoff matrix
Normal- or strategic form

Player B

Player A	Left	Right
Top	1, 1	2, 3*
Bottom	2, 3*	1, 2

Solution

- Here you can find a *Nash equilibrium*; Top is the best response to Right and Right is the best response to Top. Hence, (Top, Right) is a *Nash equilibrium*.
- But there are two problems with this solution concept.

Problems

- A game can have several *Nash equilibriums*. In this case also (Bottom, Left).
- There may not be a *Nash equilibrium* (in pure strategies).

Payoff matrix
Normal- or strategic form

Player B

Player A	Left	Right
Top	1, -1	-1, 1
Bottom	-1, 1	1, -1

Nash equilibrium in mixed strategies

- Here it is not possible to find strategies that are best responses to each other.
- If players are allowed to randomize their strategies we can find s solution; a *Nash equilibrium* in mixed strategies.
- An equilibrium in which each player chooses the optimal frequency with which to play her strategies given the frequency choices of the other agents.

The prisoner's dilemma

Two persons have committed a crime, they are held in separate rooms. If they both confess they will serve two years in jail. If only one confess she will be free and the other will get the double time in jail. If both deny they will be hold for one year.

Prisoner's dilemma
Normal- or strategic form

Prisoner B

Prisoner A	Confess	Deny
Confess	-2, -2	0, -4
Deny	-4, 0	-1, -1*

Solution

Confess is a *dominant strategy* for both. If both Deny they would be better off. This is the dilemma.

Nash Equilibrium – To do Problems!

	HENRY		
JANE		L	R
	U	8,7	4,6
	D	6,5	7,8

	McD (1)		
KFC (1)		L	R
	U	9,9*	1,10
	D	10,1	2,2

	COKE		
PEPSI		L	R
	U	6,8*	4,7
	D	7,6	3,7

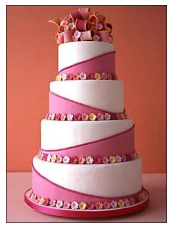
	B		
A		L	R
	U	7,6*	5,5
	D	4,5	6,4

GAME PLAYING & MECHANISM DESIGN

Mechanism Design is the design of games or reverse engineering of games; could be called *Game Engineering*

Involves inducing a game among the players such that in some equilibrium of the game, a desired social choice function is implemented

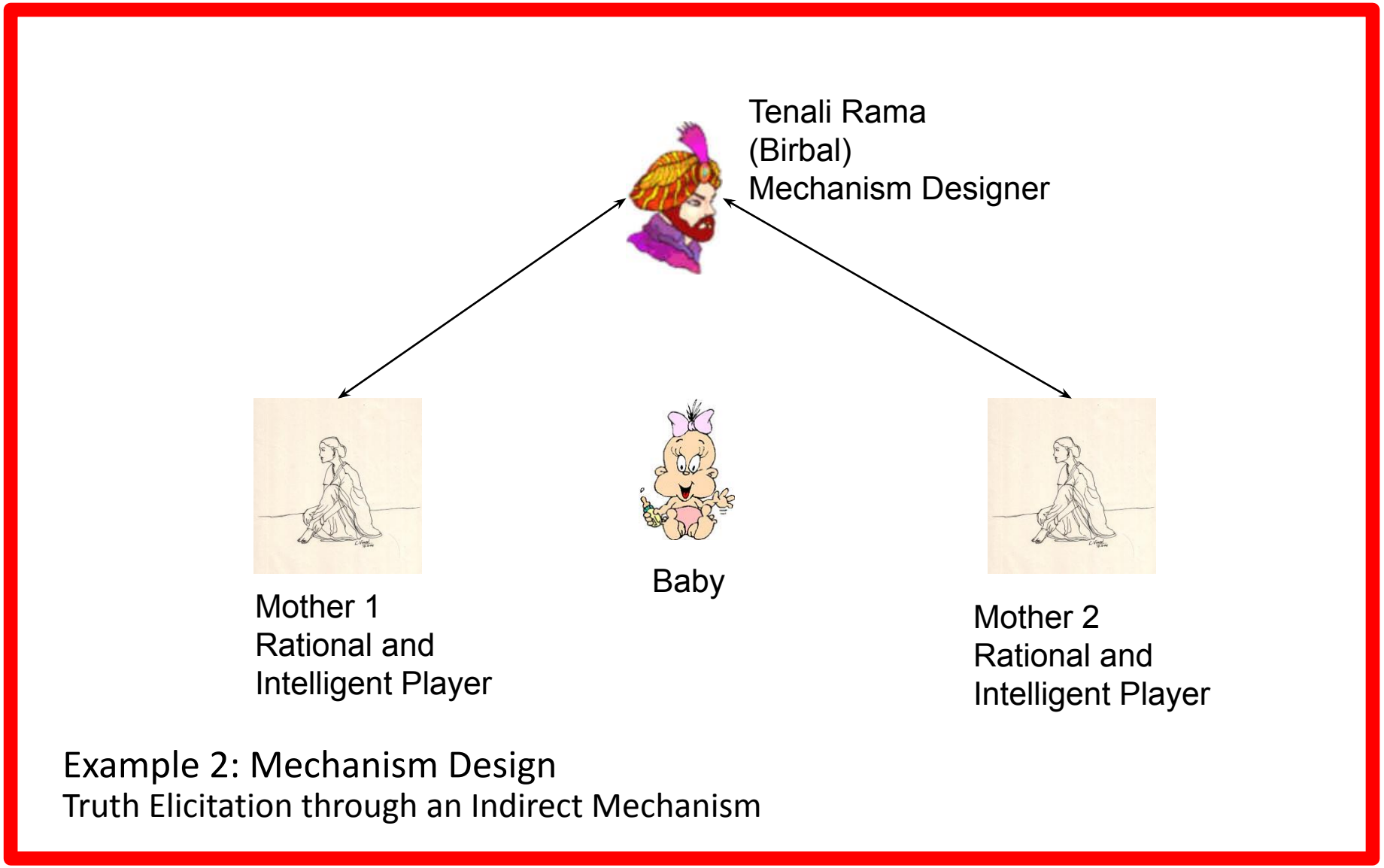
Mother
Social Planner
Mechanism Designer



Kid 1
Rational and
Intelligent

Kid 2
Rational and
Intelligent

Example 1: Mechanism Design
Fair Division of a Cake



Example 2: Mechanism Design
Truth Elicitation through an Indirect Mechanism

One Seller, Multiple Buyers, Single Indivisible Item

Example: B1: 40, B2: 45, B3: 60, B4: 80

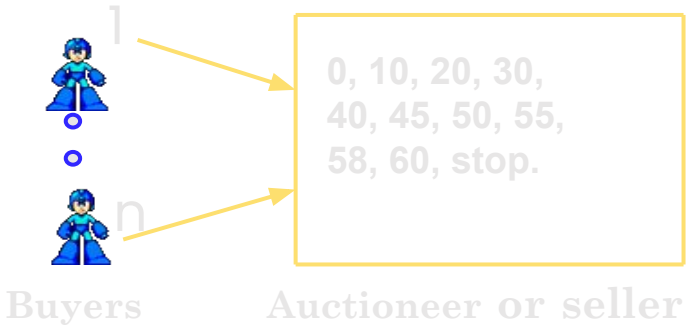
Winner: whoever bids the highest; in this case B4

Payment: Second Highest Bid: in this case, 60.

Vickrey showed that this mechanism is Dominant Strategy Incentive Compatible (DSIC) ;Truth Revelation is good for a player irrespective of what other players report

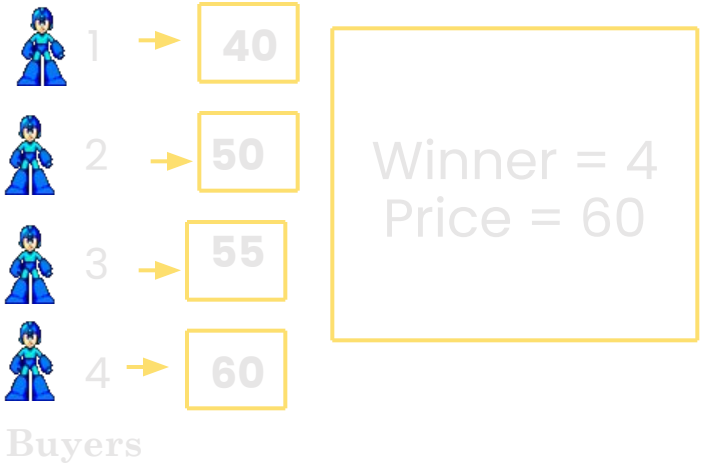
MECHANISM DESIGN: EXAMPLE 3 : VICKREY AUCTION

English Auction

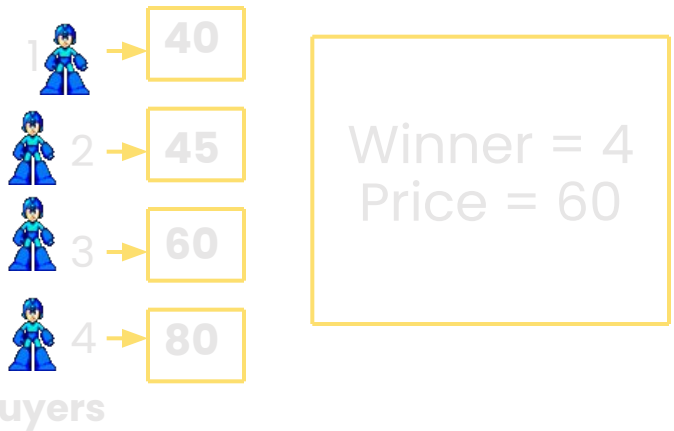


Dutch Auction

First Price Auction



Vickrey Auction



Four Basic Types of Auctions

Simple reflex agents

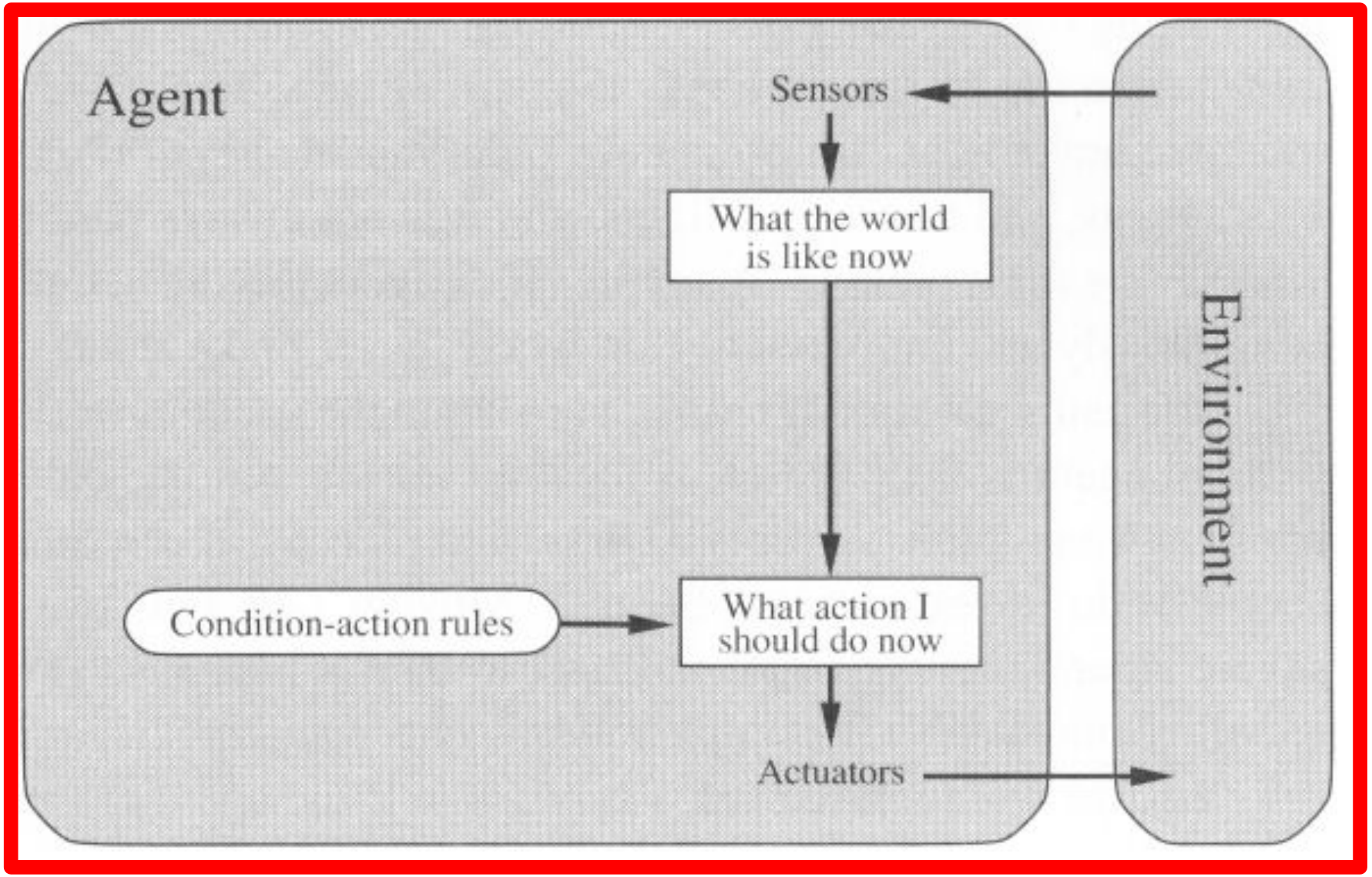
- It uses just ***condition-action rules***
  - The rules are like the form “if … then …”
  - efficient but have narrow range of applicability
  - Because knowledge sometimes cannot be stated explicitly
  - Work only
    - if the environment is fully observable

Simple reflex agents

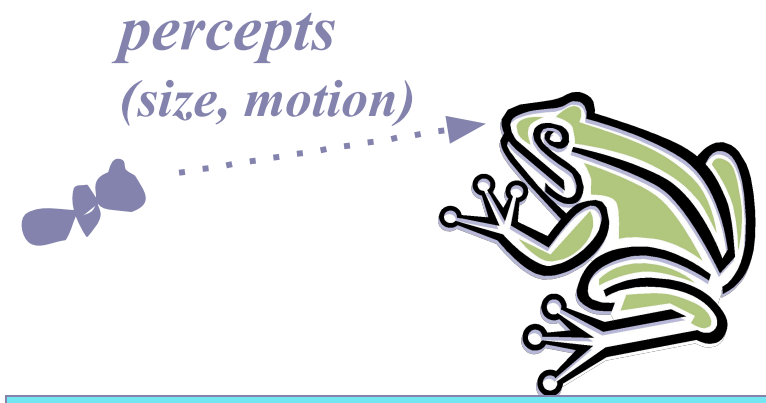

```
function SIMPLE-REFLEX-AGENT(percept) returns action
  static: rules, a set of condition-action rules

  state ← INTERPRET-INPUT(percept)
  rule ← RULE-MATCH(state, rules)
  action ← RULE-ACTION[rule]
  return action
```

Simple reflex agents



A Simple Reflex Agent in Nature



RULES:

(1) If small moving object,
then activate SNAP

(2) If large moving object,
then activate AVOID and inhibit SNAP

ELSE (not moving) then NOOP

needed for completeness

Action: SNAP or AVOID or NOOP

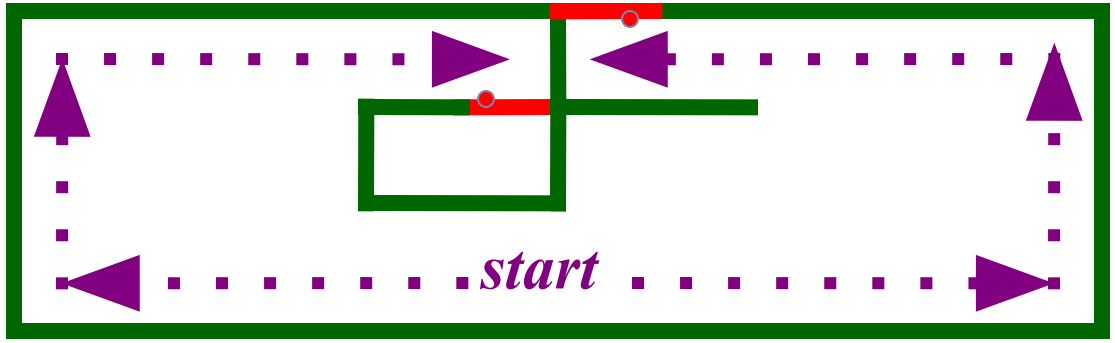
Model-based Reflex Agents

- For the world that is partially observable
  - the agent has to keep track of an internal state
    - That depends on the percept history
    - Reflecting some of the unobserved aspects
    - E.g., driving a car and changing lane
- Requiring two types of knowledge
  - How the world evolves independently of the agent
  - How the agent's actions affect the world

Example Table Agent With Internal State

IF	THEN
Saw an object ahead, and turned right, and it's now clear ahead	Go straight
Saw an object Ahead, turned right, and object ahead again	Halt
See no objects ahead	Go straight
See an object ahead	Turn randomly

Example Reflex Agent With Internal State
Wall-Following



Actions: left, right, straight, open-door

Rules:

1. If open(left) & open(right) and open(straight) then choose randomly between right and left
2. If wall(left) and open(right) and open(straight) then straight
3. If wall(right) and open(left) and open(straight) then straight
4. If wall(right) and open(left) and wall(straight) then left
5. If wall(left) and open(right) and wall(straight) then right
6. If wall(left) and door(right) and wall(straight) then open-door
7. If wall(right) and wall(left) and open(straight) then straight.
8. (Default) Move randomly

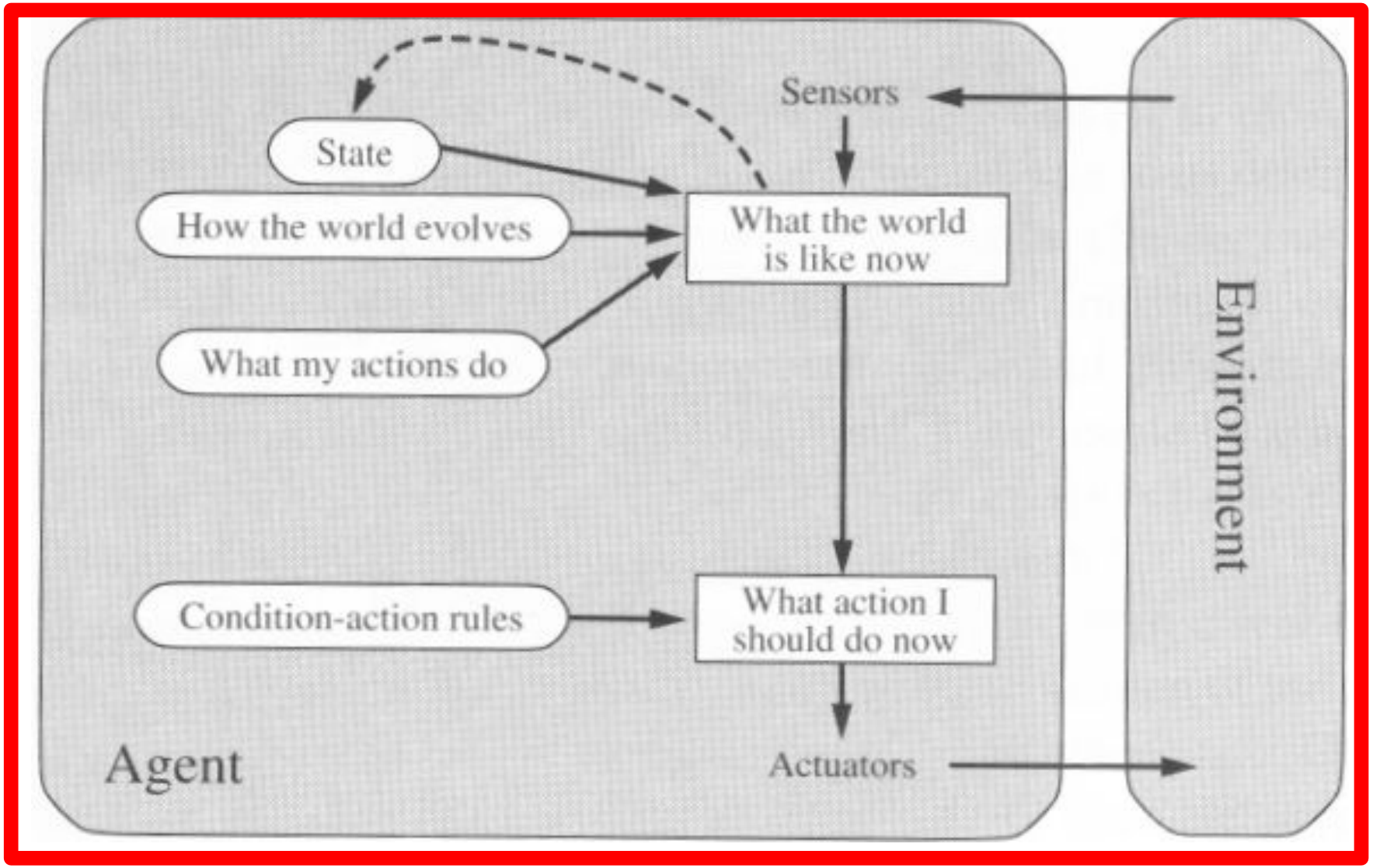
Model-based Reflex Agents


```
function REFLEX-AGENT-WITH-STATE(percept) returns action
  static: state, a description of the current world state
          rules, a set of condition-action rules

  state ← UPDATE-STATE(state, percept)
  rule ← RULE-MATCH(state, rules)
  action ← RULE-ACTION[rule]
  state ← UPDATE-STATE(state, action)
  return action
```

The agent is with memory

Model-based Reflex Agents



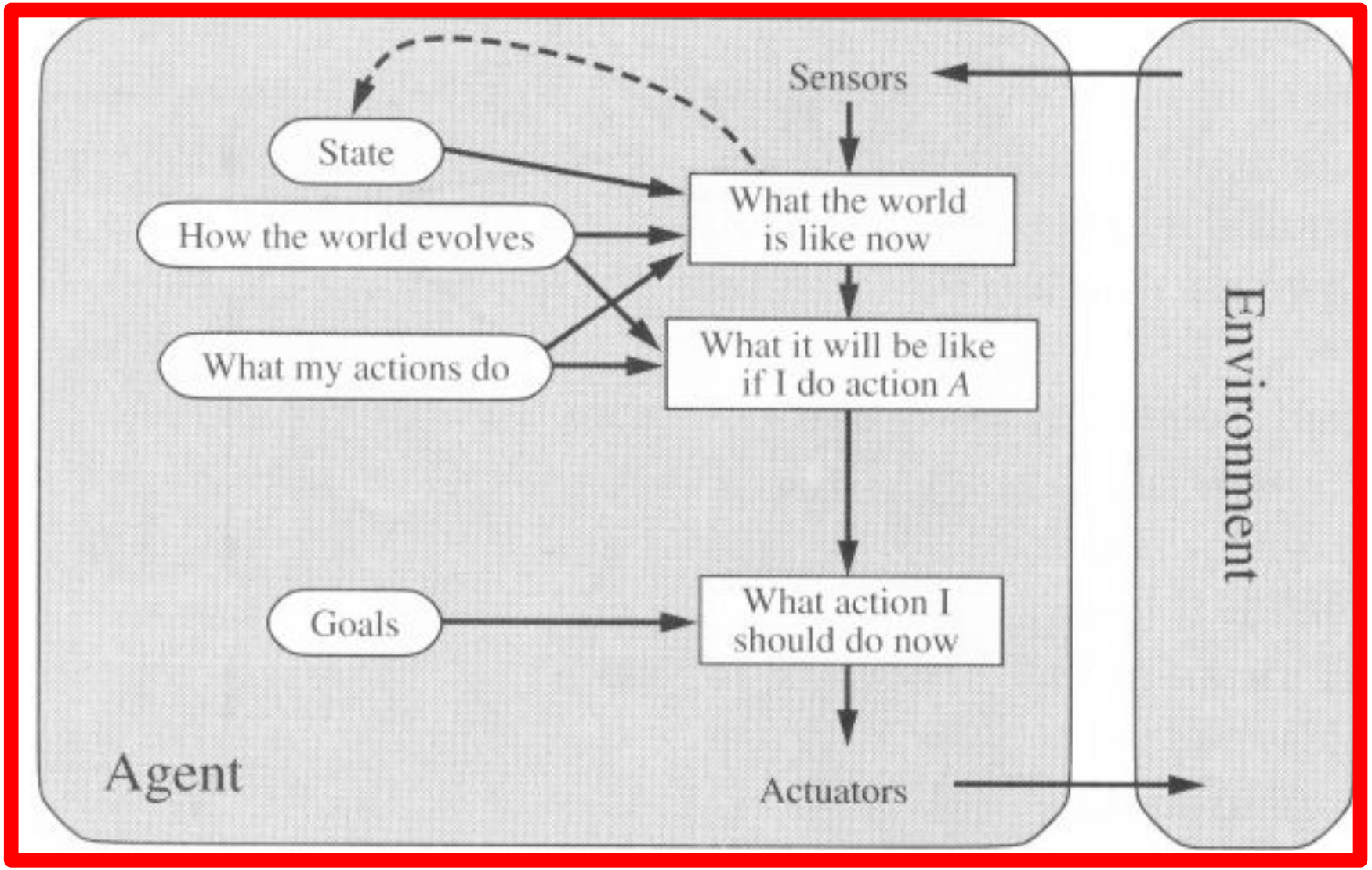
Goal-based agents

- Current state of the environment is always not enough
- The goal is another issue to achieve
  - Judgment of rationality / correctness
- Actions chosen goals, based on
  - the current state
  - the current percept

Goal-based agents

- Conclusion
  - Goal-based agents are less efficient
  - but more flexible
    - Agent Different goals different tasks
  - Search and planning
    - two other sub-fields in AI
    - to find out the action sequences to achieve its goal

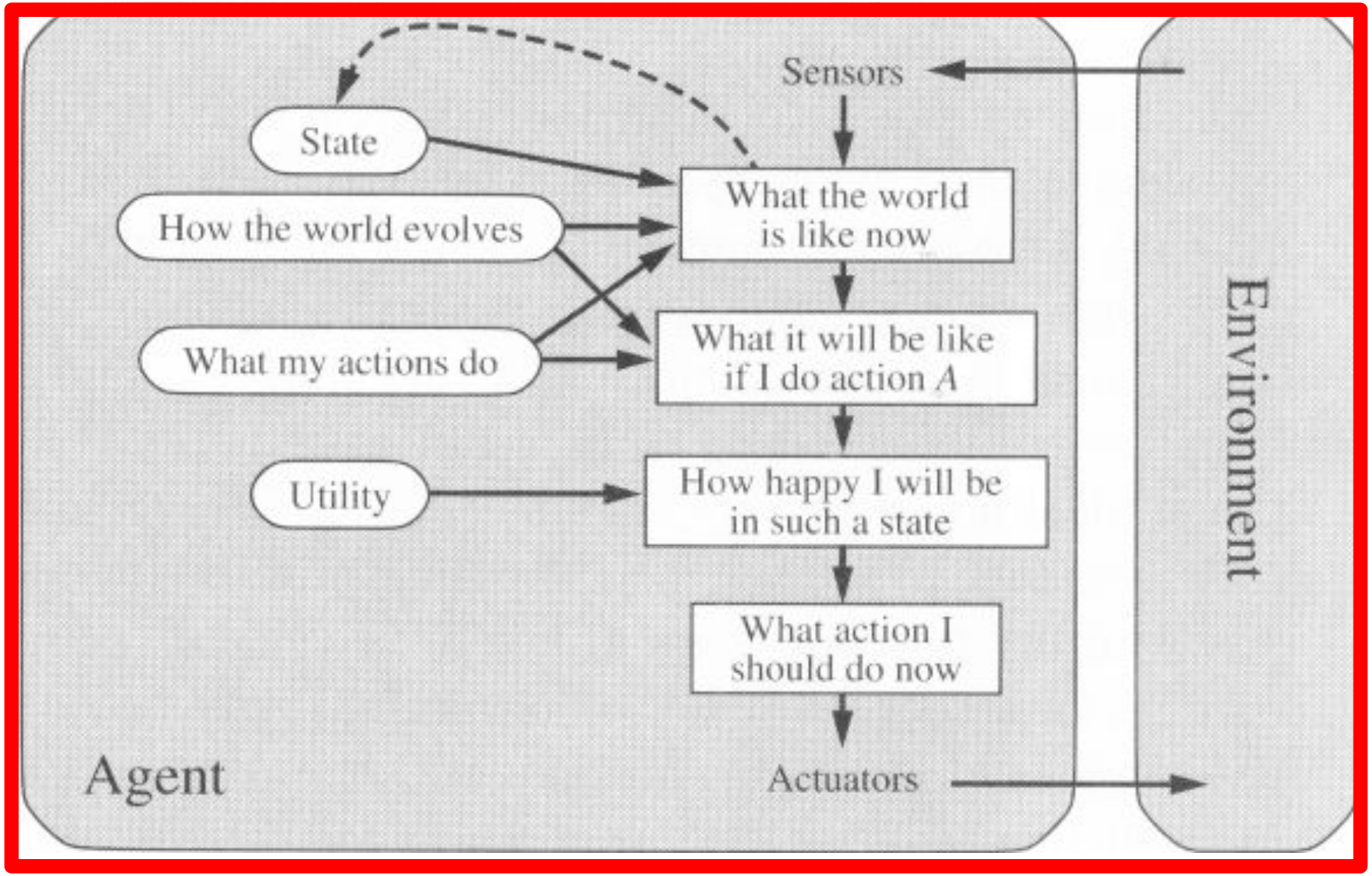
Goal-based agents



Utility-based agents

- Goals alone are not enough
  - to generate **high-quality** behavior
  - E.g. meals in Canteen, good or not ?
- Many action sequences the goals
  - some are better and some worse
  - If goal means success,
  - then **utility** means the degree of success (how successful it is)

Utility-based agents(4)



Utility-based agents

- it is said state A has higher utility
  - If state A is more preferred than others
- Utility is therefore a function
  - that maps a state onto a real number
  - the degree of success

Utility-based agents (3)

- Utility has several advantages:
  - When there are conflicting goals,
    - Only some of the goals but not all can be achieved
    - utility describes the appropriate trade-off
  - When there are several goals
    - None of them are achieved **certainly**
    - utility provides a way for the decision-making

Learning Agents

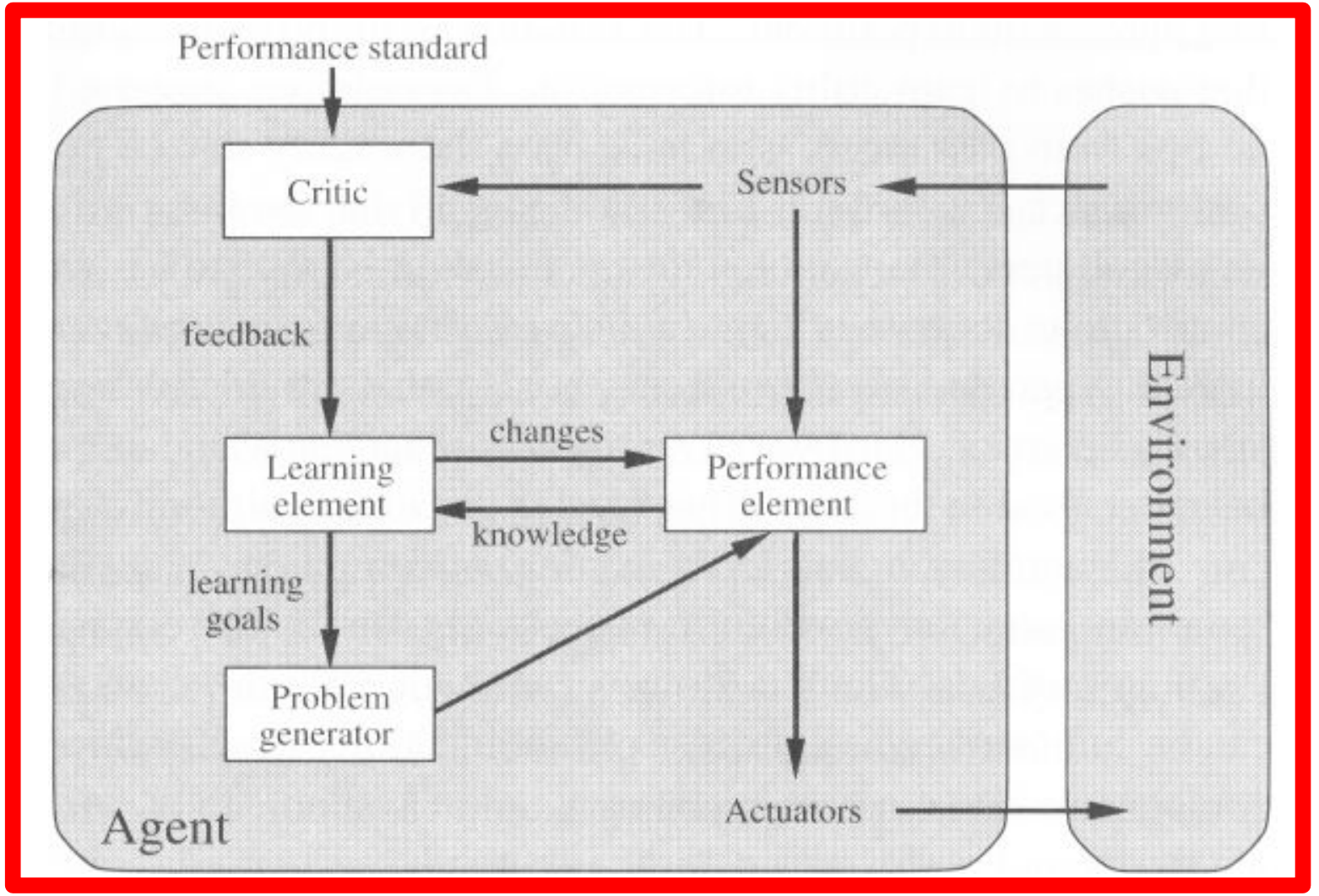
- After an agent is programmed, can it work immediately?
  - No, it still need teaching
- In AI,
  - Once an agent is done
  - We teach it by giving it a set of examples
  - Test it by using another set of examples
- We then say the agent learns
  - A learning agent

Learning Agents

- Four conceptual components
  - Learning element
    - Making improvement
  - Performance element
    - Selecting external actions
  - Critic
    - Tells the Learning element how well the agent is doing with respect to fixed performance standard.

      (Feedback from user or examples, good or not?)
  - Problem generator
    - Suggest actions that will lead to new and informative experiences.

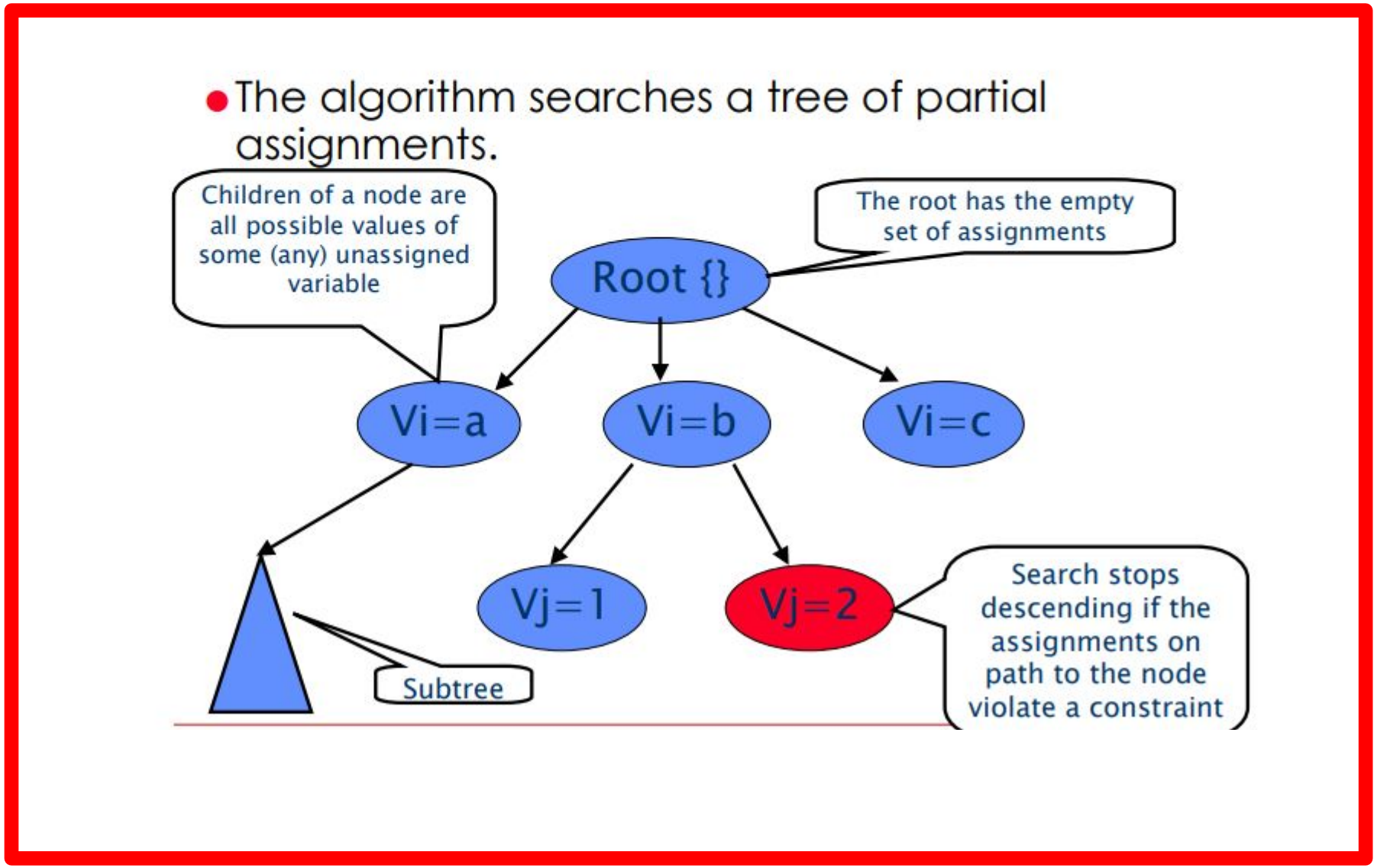
Learning Agents



Constraint Satisfaction Problem

- Search Space is constrained by a set of conditions and dependencies
- Class of problems where the search space is constrained called as CSP
- For example, Time-table scheduling problem for lecturers.
- Constraint,
  - Two Lecturers can not be assigned to same class at the same time
- To solve CSP, the problem is to be decomposed and analyse the structure
- Constraints are typically mathematical or logical relationships

Solving Constraint Satisfaction Problem



CSP

- Any problem in the world can be mathematically represented as CSP
- Hypothetical problem does not have constraints
- Constraint restricts movement, arrangement, possibilities and solutions
- Example: if only concurrency notes of 2,5,10 rupees are available and we need to give certain amount of money, say Rs. 111 to a salesman and the total number of notes should be between 40 and 50.
- Expressed as, Let n be number of notes,
  - $40 < n < 50$
- and
  - $c_1X_1 + c_2X_2 + c_3X_3 = 111$

CSP

- Types of Constraints
  - Unary Constraints - Single variable
  - Binary Constraints - Two Variables
  - Higher Order Constraints - More than two variables
- CSP can be represented as Search Problem
- Initial state is empty assignment, while successor function is a non-conflicting value assigned to an unassigned variables
- Goal test checks whether the current assignment is complete and path cost is the cost for the path to reach the goal state
- CSP Solutions leads to the final and complete assignment with no exception

Cryptarithmetic puzzles

- Cryptarithmetic puzzles are also represented as CSP
- Example: MIKE + JACK = JOHN
- Replace every letter in puzzle with single number (number should not be repeated for two different alphabets)
- The domain is *{ 0,1, ... , 9 }*

- Often treated as the ten-variable constraint problem
- where the constraints are:
  - All the variables should have a different value
  - The sum must work out

Cryptarithmetic puzzles

- M * 1000 + I * 100 + K * 10 + E + J * 1000 + A * 100 + C * 10 + K = J * 1000 + O * 100 + H * 10 + N
- Constraint Domain is represented by Five-tuple and represented by,
  - *D = {var, f , O, dv, rg}*
- *Var* stands for set variables, *f* is set functions, *O* stands for the set of legitimate operators to be used, *dv* is domain variable and *rg* is range of function in the constraint
- Constraint without conjunction is referred as Primitive constraint (for Eg., $x < 9$)
- Constraint with conjunction is called as non-primitive constraint or a generic constraint (For Eg., $x < 9$ and $x > 2$)

Crypt arithmetic puzzles – Solved Example


```
  TO
  GO
  ---
 OUT
 ------
  21
  81        G = 8/9
  ---
 102
 ------     2 + G = U + 10
```

Var	Value
T	2
O	1
G	8
U	0

Cryptarithmetic puzzles

- SEND + MORE = MONEY

c4 c3 c2 c1

```
   S  E  N  D
   M     O  R  E
   ----------------------
M O     N  E  Y
  9 5 6 7
  1  0 8 5
 ----------
1  0 8 52
```

D + E = Y + 10C1
N + R + C1 = E + 10C2
E + O + C2 = N + 10C3
S + M + C3 = O + 10C4
C4 = M

CSP- Room Coloring Problem
-CSP as a Search Problem

- Let *K* for Kitchen, *D* for Dining Room, *H* is for Hall, B_2 and B_3 are bedrooms 2 and 3, MB_1 is master bedroom, *SR* is the store Room, *GR* is Guest Room and *Lib* is Library
- Constraints
  - All bedrooms should not be colored red, only one can
  - No two adjacent rooms can have the same color
  - The colors available are red, blue, green and violet
  - Kitchen should not be colored green
  - Recommended to color the kitchen as blue
  - Dining room should not have violet color

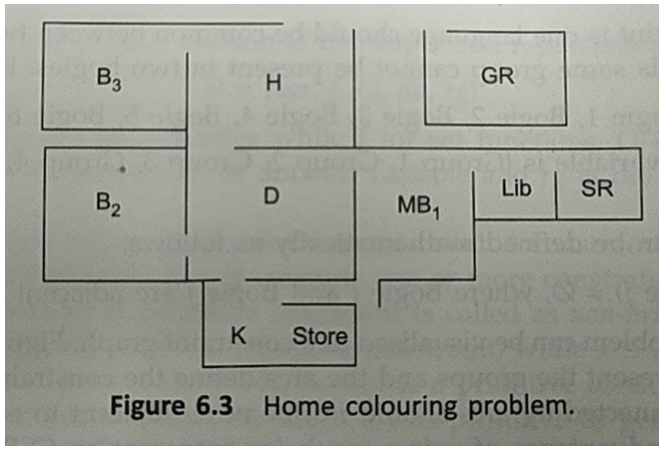


Figure 6.3 Home colouring problem.

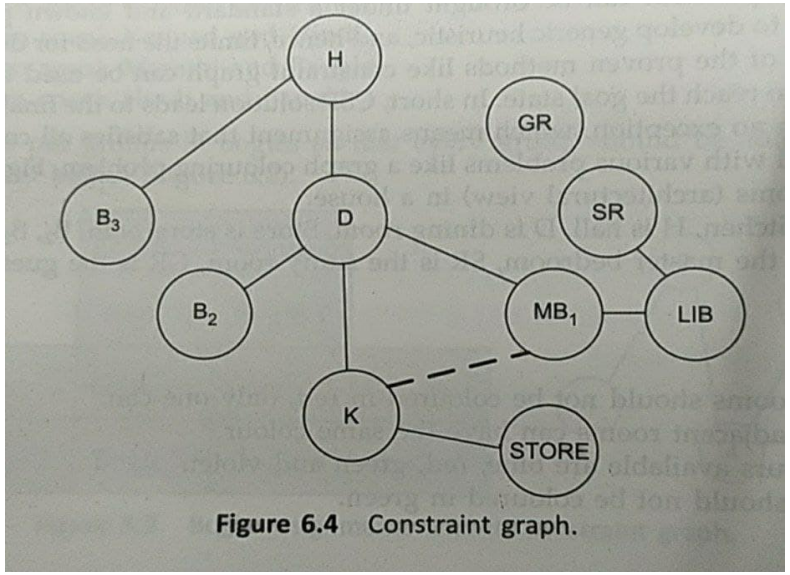


Figure 6.4 Constraint graph.

Room Coloring Problem – Representation as a Search Tree

- Soft Constraints are that they are cost-oriented or preferred choice
- All paths in the Search tree can not be accepted because of the violation in constraints

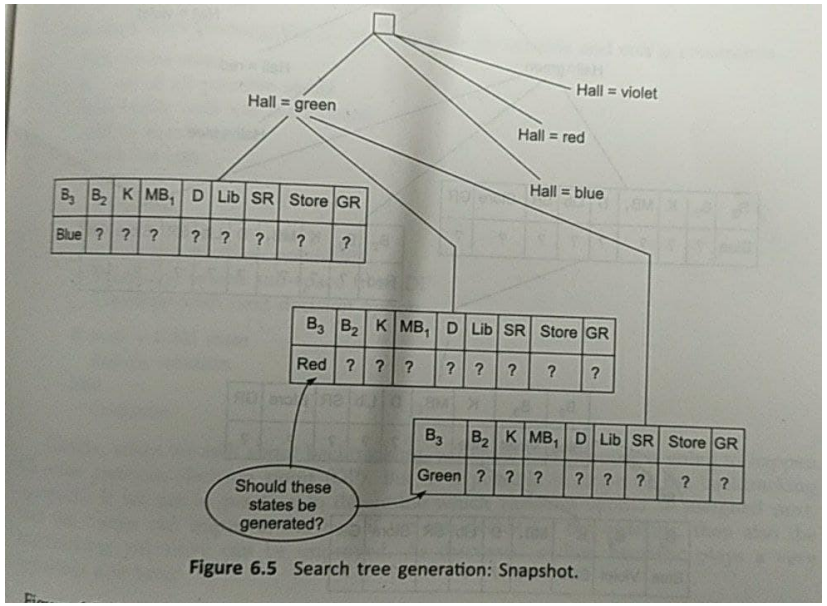


Figure 6.5 Search tree generation: Snapshot.

Backtracking Search for CSP

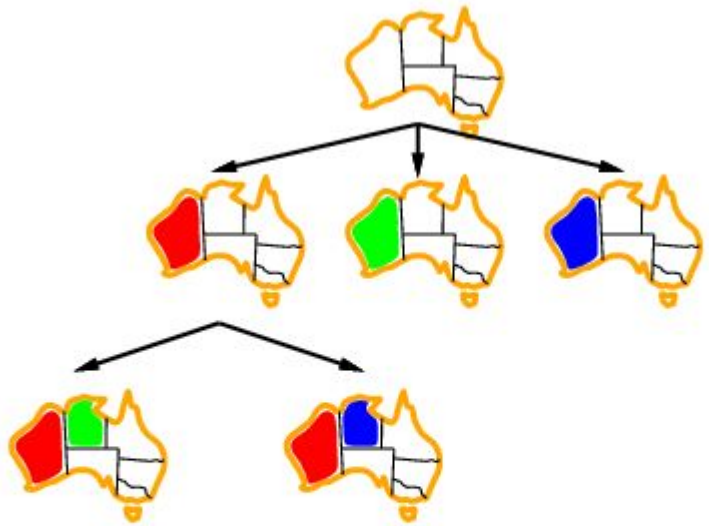
- Assignment of value to any additional variable within constraint can generate a legal state (Leads to successor state in search tree)
- Nodes in a branch backtracks when there is no options are available

Example: Map-Coloring

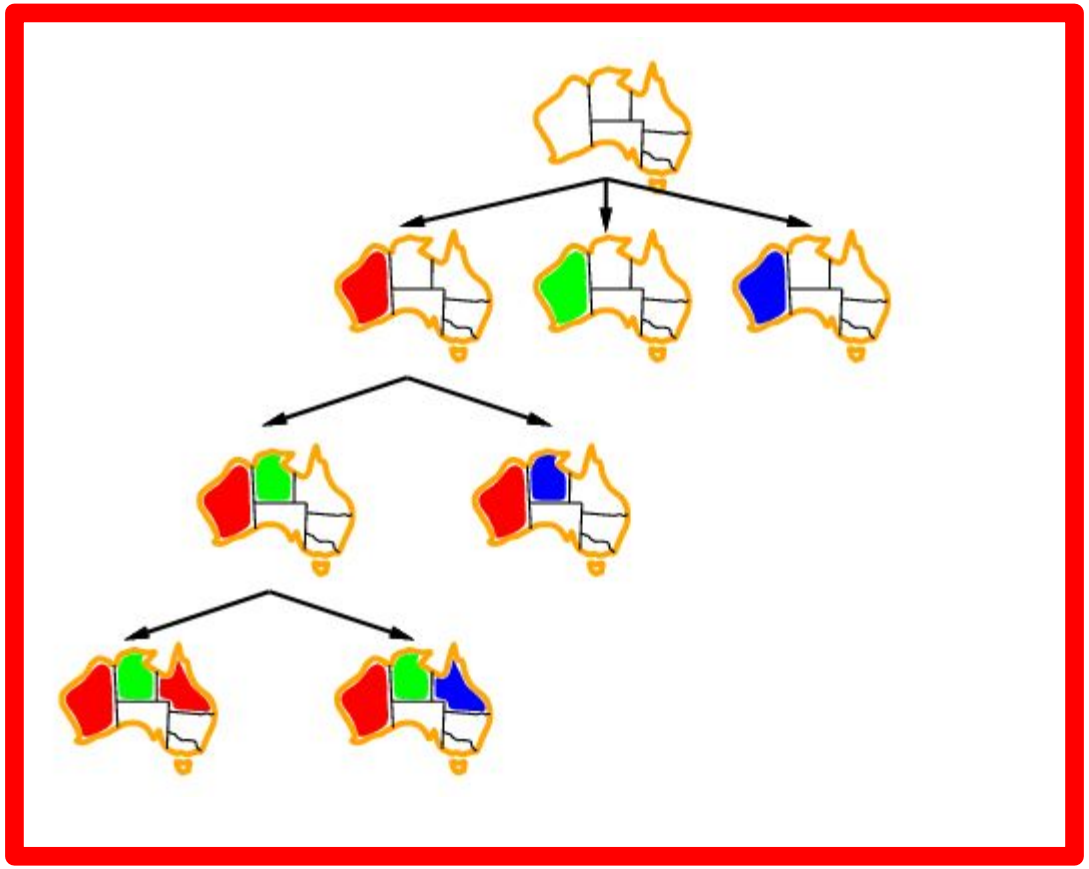
- Variables *WA, NT, Q, NSW, V, SA, T*
- Domains D_i = {red,green,blue}
- Constraints: adjacent regions must have different colors
  e.g., WA ≠ NT, or (WA,NT) in {(red,green),(red,blue),(green,red), (green,blue),(blue,red),(blue,green)}

Backtracking example

Backtracking example



Backtracking example



Algorithm for Backtracking

Pick initial state

R = set of all possible states

Select state with var assignment

Add to search space

check for con

If Satisfied

Continue

Else

Go to last Decision Point (DP)

Prune the search sub-space from DP

Continue with next decision option

If state = Goal State

Return Solution

Else

Continue

CSP-Backtracking, Role of heuristic

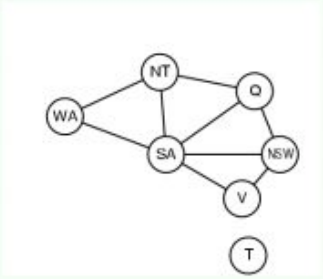
- Backtracking allows to go to the previous decision-making node to eliminate the invalid search space with respect to constraints
- Heuristics plays a very important role here
- If we are in position to determine which variables should be assigned next, then backtracking can be improved
- **Heuristics** help in deciding the initial state as well as subsequent selected states
- Selection of a variable with minimum number of possible values can help in simplifying the search
- This is called as Minimum Remaining Values Heuristic (MRV) or Most Constraint Variable Heuristic
- Restricts the most search which ends up in same variable (which would make the backtracking ineffective)

Heuristic

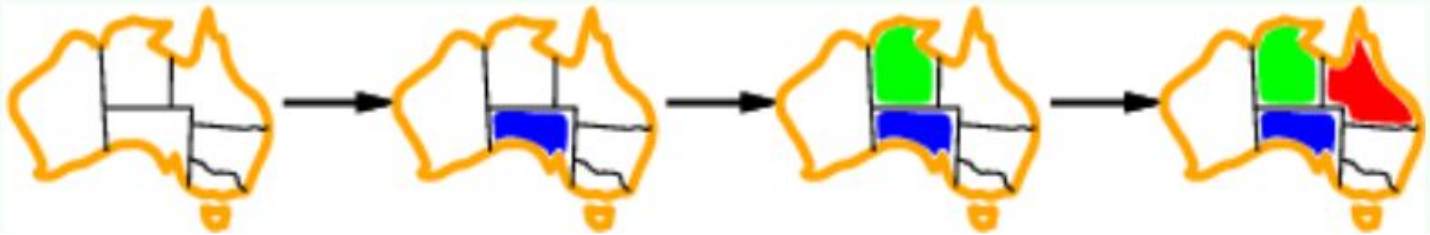
- MRV cannot have hold on initial selection process
- Node with maximum constraint is selected over other unassigned variables - Degree Heuristics
- By degree heuristics, branching factor can not be reduced
- Selection of variables are considered not the values for it, so the order in which the values of particular variable can be arranged is tackled by least constraining value heuristic

Heuristic – Most Constraining Variable

- Tie-breaker among most constrained variables

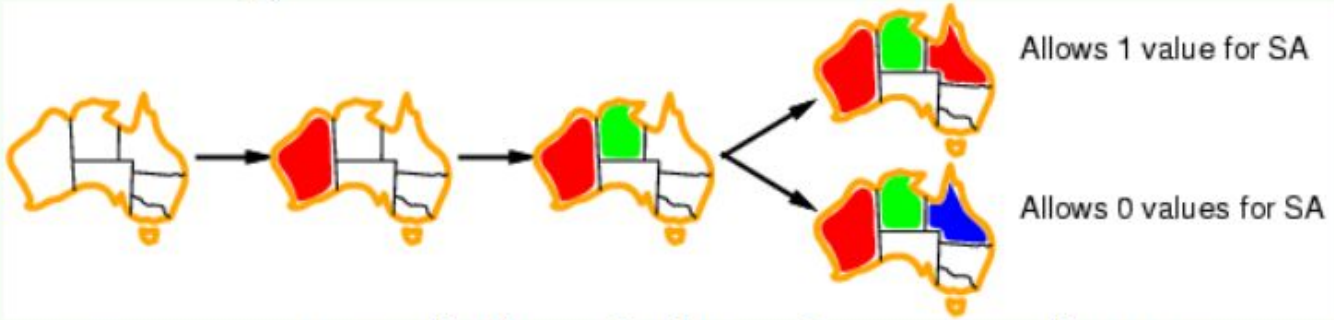


- Most constrain*ing* variable:
  - choose the variable with the most constraints on remaining variables (most edges in graph)



Heuristic- Minimum Remaining Values

- *Given a variable*, choose the least constraining value:
  - the one that rules out the fewest values in the remaining variables



- Leaves maximal flexibility for a solution.

Forward Checking

- To understand the forward checking, we shall see 4 Queens problem
- If an arrangement on the board of a queen *x* , hampers the position of $queens_{x+1}$, then this forward check ensures that the queen *x* should not be placed at the selected position and a new position is to be looked upon

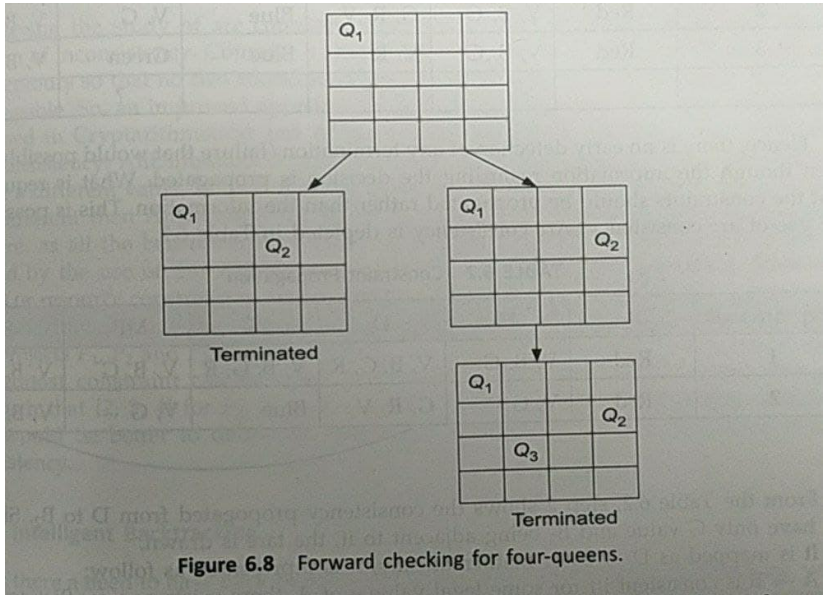


Figure 6.8 Forward checking for four-queens.

Forward Checking

- Q_1 and Q_2 are placed in row 1 nad 2 in the left sub-tree, so, search is halted, since No positions are left for Q_3 and Q_4
- Forward Checking keeps track of the next moves that are available for the unassigned variables
- The search will be terminated when there is no legal move available for the unassigned variables

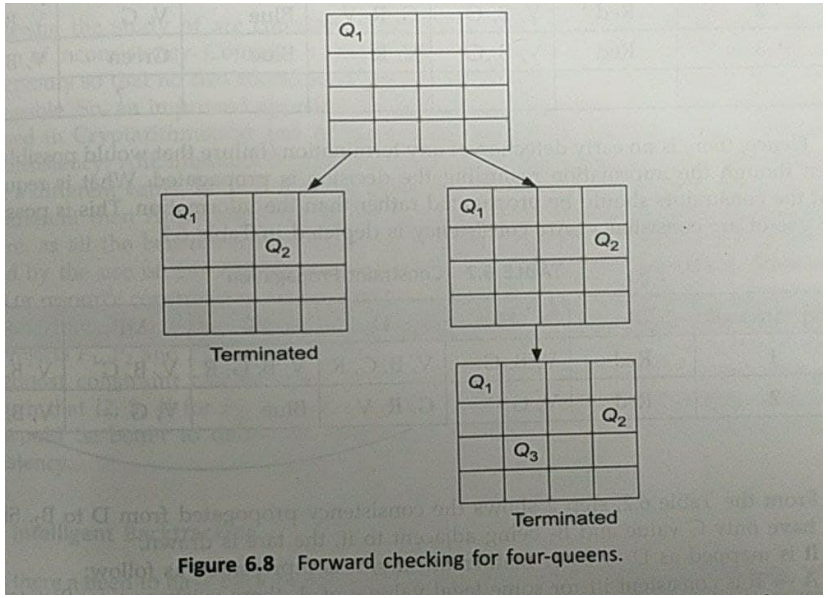


Figure 6.8 Forward checking for four-queens.

CSP- Room Coloring Problem
-CSP as a Search Problem

- Let *K* for Kitchen, *D* for Dining Room, *H* is for Hall, B_2 and B_3 are bedrooms 2 and 3, MB_1 is master bedroom, *SR* is the store Room, *GR* is Guest Room and *Lib* is Library
- Constraints
  - All bedrooms should not be colored red, only one can
  - No two adjacent rooms can have the same color
  - The colors available are red, blue, green and violet
  - Kitchen should not be colored green
  - Recommended to color the kitchen as blue
  - Dining room should not have violet color

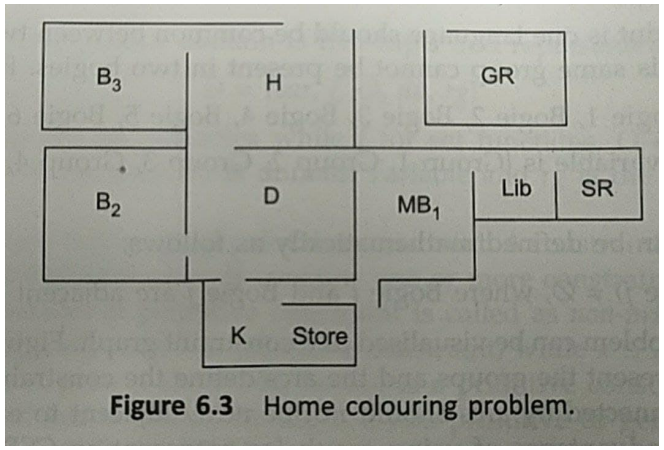


Figure 6.3 Home colouring problem.

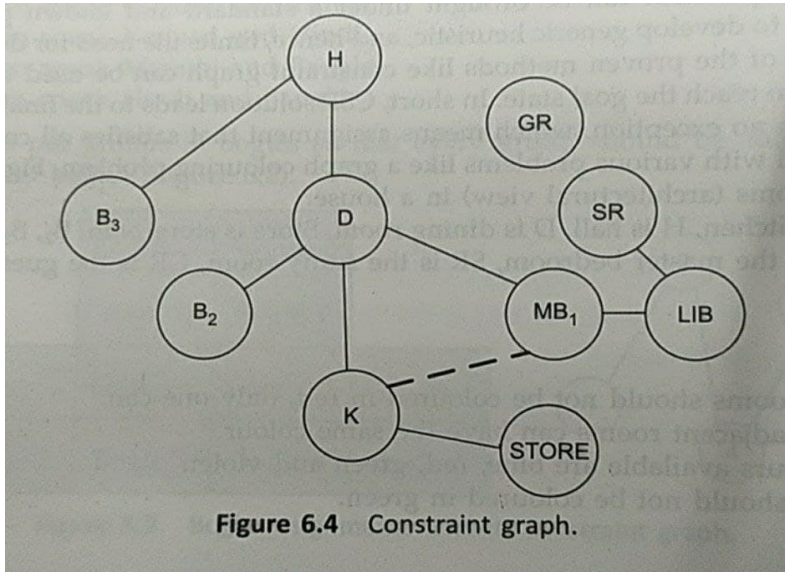


Figure 6.4 Constraint graph.

Forward Checking – Room Coloring Problem

- For Room Coloring problem, Considering all the constraints the mapping can be done in following ways;
  - At first, B_2 is selected with Red (R). Accordingly, R is deleted from the adjacent nodes
  - Kitchen is assigned with Blue (B). So, B is deleted form the adjacent Nodes
  - Furthermore, as MB_1 is selected green, no color is left for *D*.

TABLE 6.1 Decision and Information Propagation

Step number	B_3	H	D	K	MB_1	B_2
1	Red	V, B, G	V, B, G, R	V, B, G, R	V, B, G	V, B, G
2	Red	V, B, G	G, R, V	Blue	V, G	V, B, G
3	Red	V, B, G	V, R	Blue	Green	V, B
			?		?	

Constraint Propagation

- There is no early detection of any termination / failure that would possible occur even though the information regarding the decision is propagated
- Constraint should be propagated rather than the information

TABLE 6.2 Constraint Propagation

Step number	B_3	H	D	K	MB_1	B_2
1	Red	V, B, G	V, B, G, R	V, B, G, R	V, B, G	V, B, G
2	Red	V, G	G, R, ~~V~~	Blue	V, G	V, B, G

Constraint Propagation

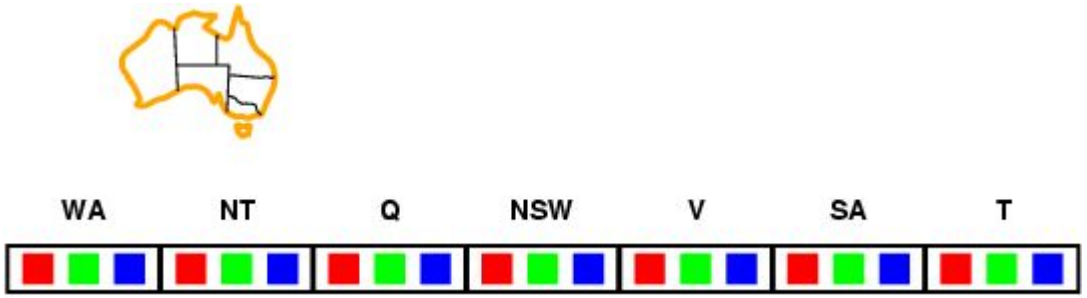
- Step 2 shows the consistency propagated from D to B_2
- Since D is can have only *G* value and B_2 being adjacent to it, the arc is drawn
- It is mapped as $D \rightarrow B_2$ or Mathematically,
  - $A \rightarrow B$ is consistent $\leftrightarrow$ $\forall$ legal value $a \in A$, $\exists$
    - non-conflicting value b $\in B$
- Failure detection can take place at early stage

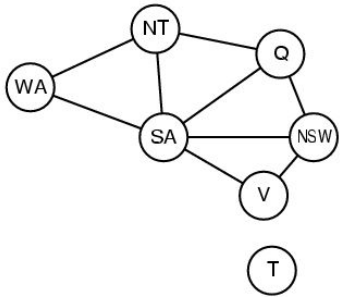
Algorithm for Arc Assignment

- Algorithm for arc assignment is:
  - Let C be the variable which is being assigned at a given instance
  - X will have some value from D{} where D is domain
  - For each and every assigned variable, that is adjacent to X, Say X^j
    - 1Perform forward check (remove values from domain D that conflict the decision of the current assignment)
    - 2For every other variable X^{jj} that are adjacent or connected to
    - X^j;
      - iRemove the values from D from X^{jj} that can't be taken as further unassigned variables
    - iiRepeat step 2, till no more values can be removed or discarded
  - Inconsistency is considered and constraints are propagated in Step (2)

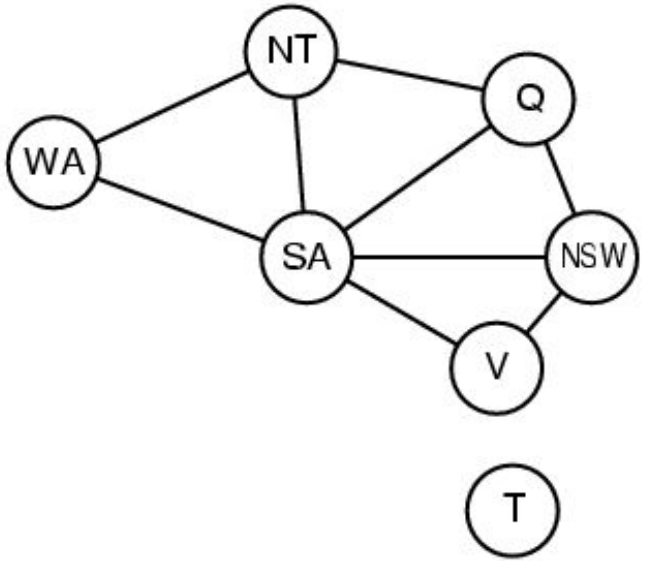
Forward checking

- Idea:
  - Keep track of remaining legal values for unassigned variables
  - Terminate search when any variable has no legal values



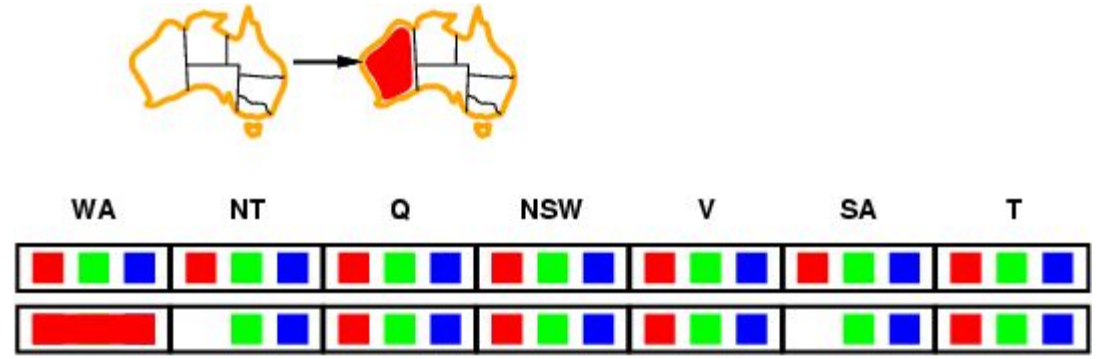


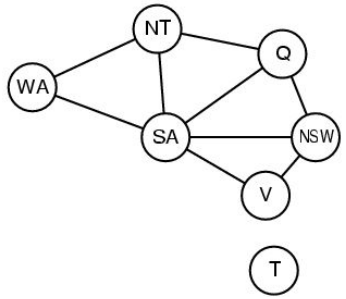
WA	RGB
NT	GB
SA	B
Q	R
NSW	G
Y	R
T	RGB



Forward checking

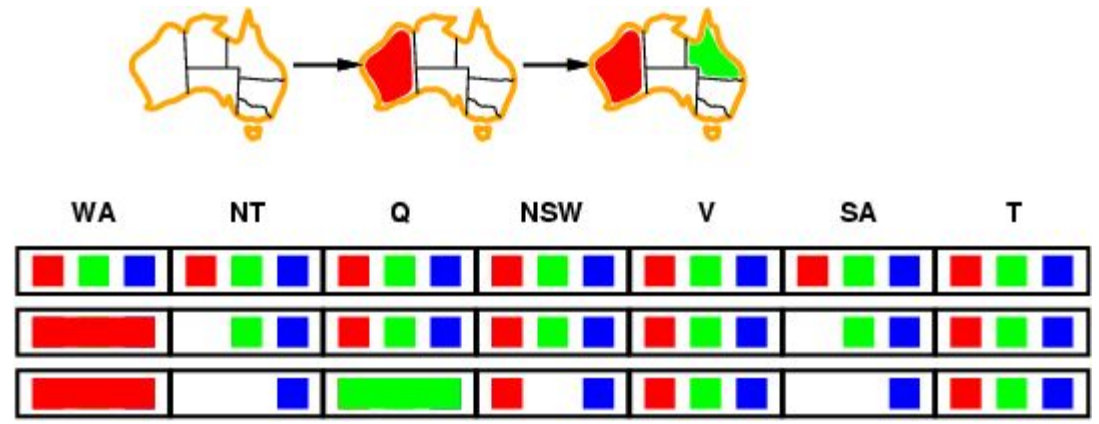
- Idea:
  - Keep track of remaining legal values for unassigned variables
  - Terminate search when any variable has no legal values

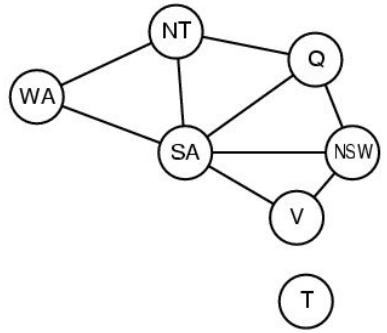




Forward checking

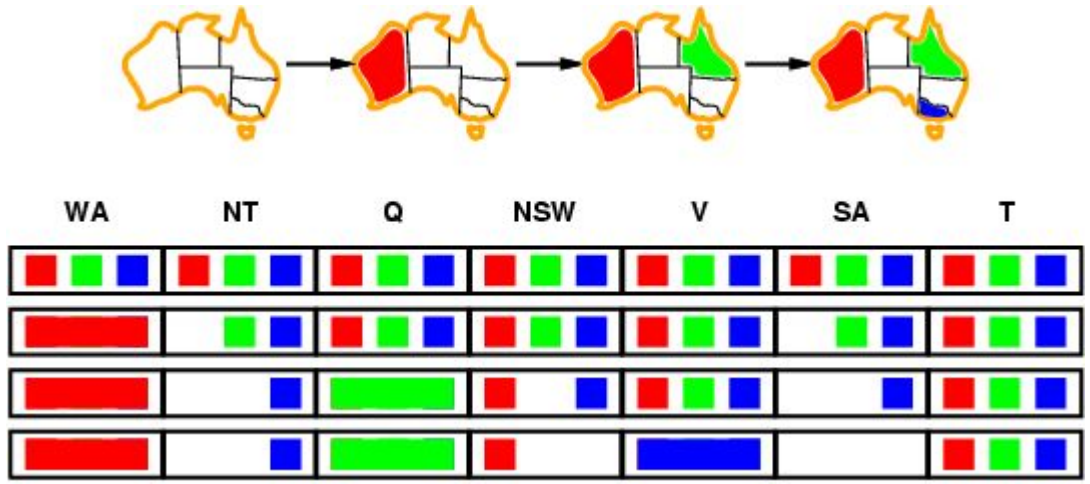
- Idea:
  - Keep track of remaining legal values for unassigned variables
  - Terminate search when any variable has no legal values

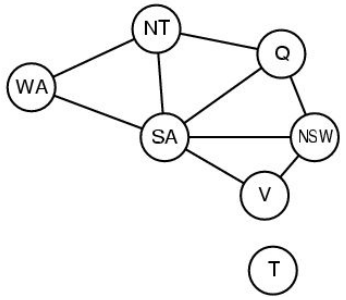




Forward checking

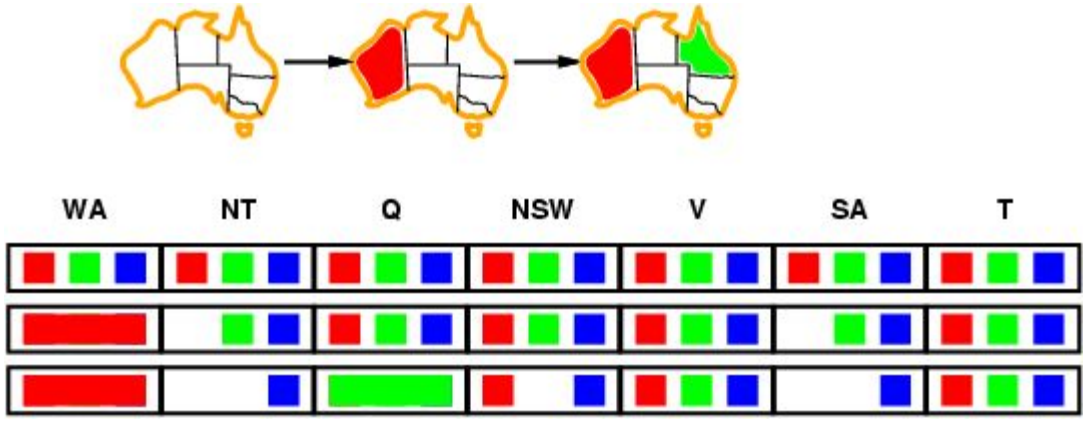
- Idea:
  - Keep track of remaining legal values for unassigned variables
  - Terminate search when any variable has no legal values

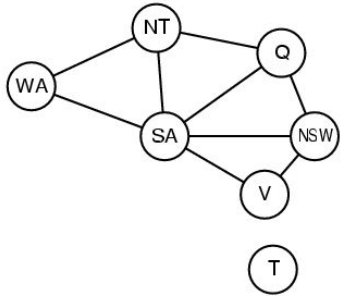




Constraint propagation

- Forward checking propagates information from assigned to unassigned variables, but doesn't provide early detection for all failures:



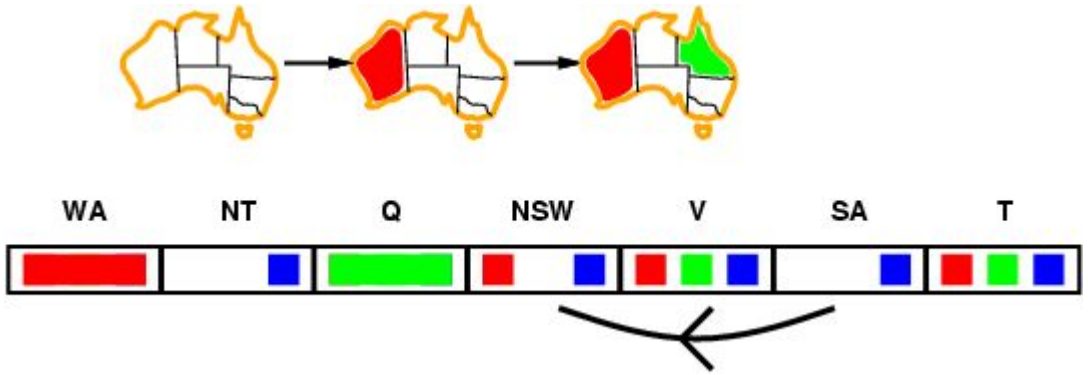


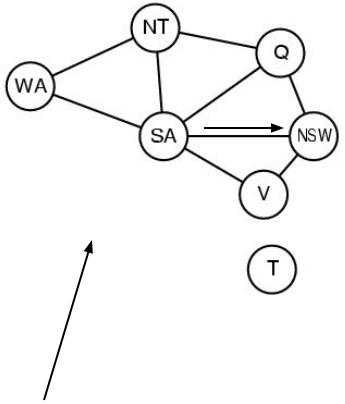
- NT and SA cannot both be blue!
- Constraint propagation repeatedly enforces constraints locally

Arc consistency

- Simplest form of propagation makes each arc consistent
- $X \square Y$ is consistent iff

  for every value x of X there is some allowed y



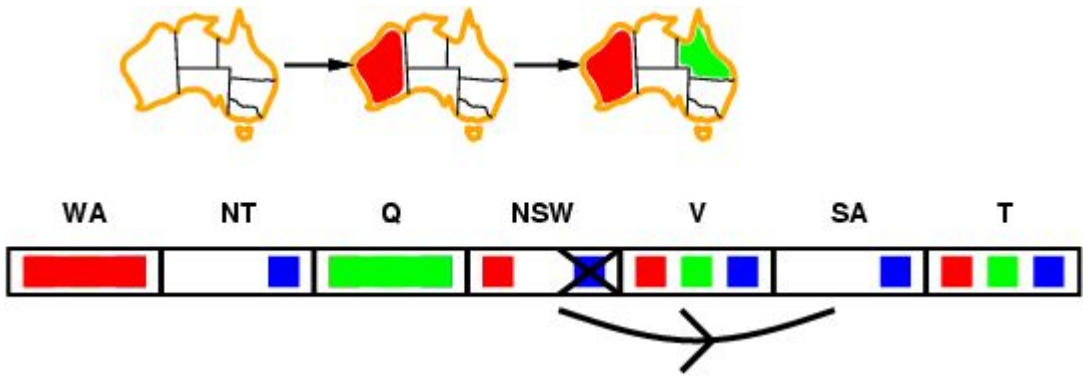


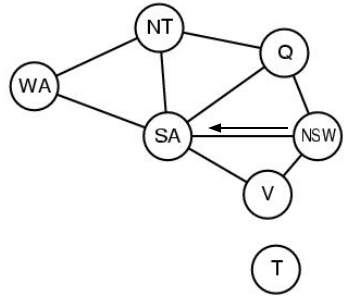
constraint propagation propagates arc consistency on the graph.

Arc consistency

- Simplest form of propagation makes each arc consistent
- *X* □ *Y* is consistent iff

  for every value *x* of *X* there is some allowed *y*

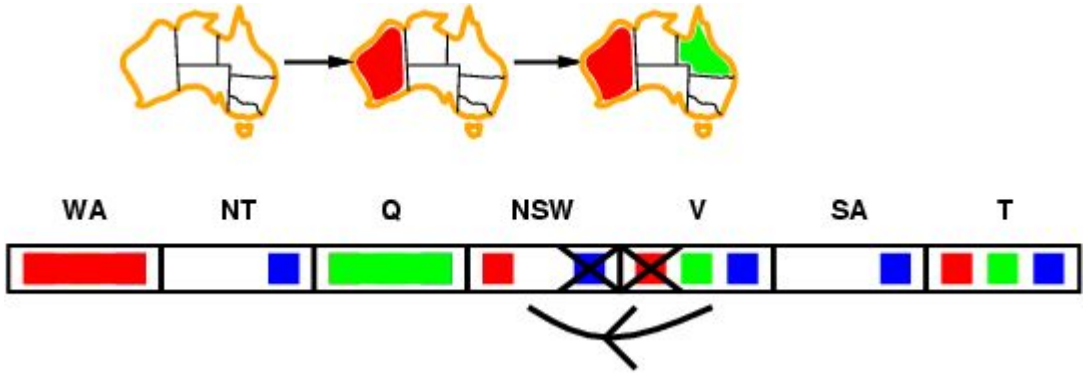


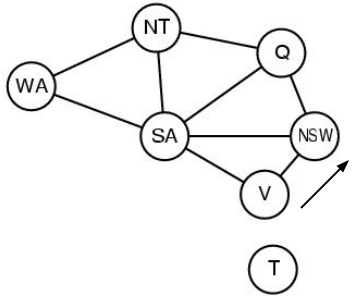


Arc consistency

- Simplest form of propagation makes each arc consistent
- *X* □ *Y* is consistent iff

  for every value *x* of *X* there is some allowed *y*

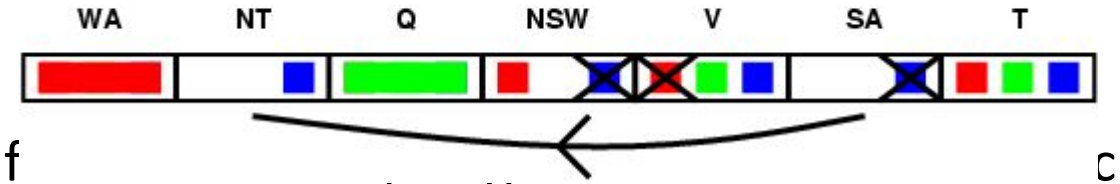


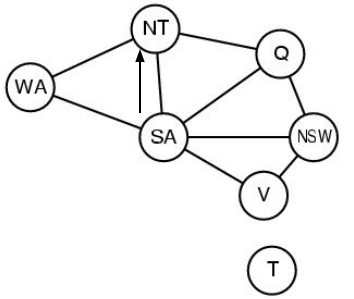


- If *X* loses a value, neighbors of *X* need to be rechecked

Arc consistency

- Simplest form of propagation makes each arc consistent
- $X \square Y$ is consistent iff
  for every value x of X there is some allowed y





- If checked
- Arc consistency detects failure earlier than forward checking
- Can be run as a preprocessor or after each assignment

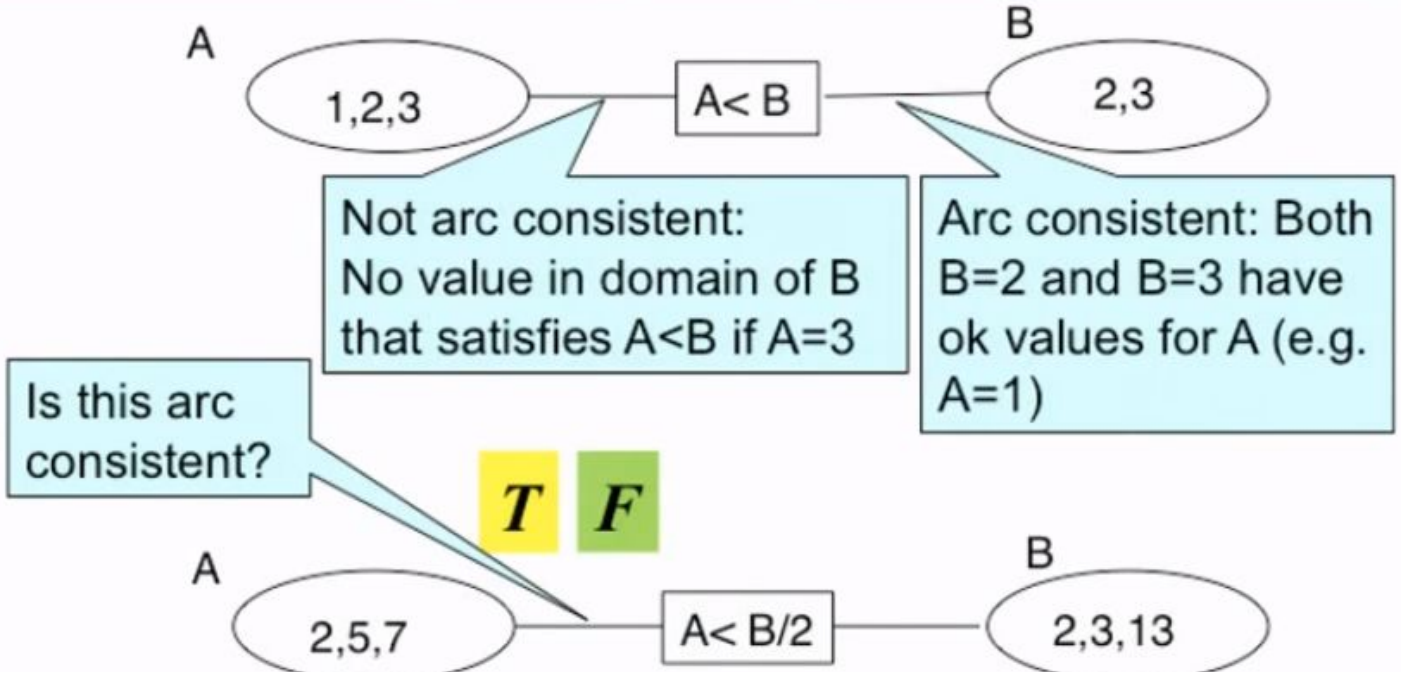
- Time complexity: $O(n^2d^3)$

Arc Consistency

Definition:

An arc <x, r(x,y)> is arc consistent if for each value x in dom(X) there is some value y in dom(Y) such that r(x,y) is satisfied.

A network is arc consistent if all its arcs are arc consistent.

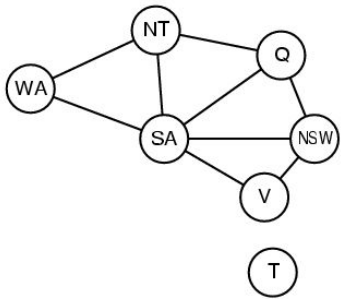


Intelligent Backtracking

- Conflict set is maintained using forward checking and maintained
- Considering the 4 Queens problem, Conflict needs to be detected by the user of conflict set so that a backtrack can occur
- Backtracking with respect to the conflict set is called as conflict-directed back jumping
- Back jumping approach can't actually restrict the earlier committed mistakes in some other branches

Intelligent Backtracking

- **Chronological backtracking:** The BACKGRACKING-SEARCH in which, when a branch of the search fails, back up to the preceding variable and try a different value for it. (The most recent decision point is revisited).
  - e.g: Suppose we have generated the partial assignment {Q=red, NSW=green, V=blue, T=red}.
  - When we try the next variable SA, we see every value violates a constraint.
  - We back up to T and try a new color, it cannot resolve the problem.



- **Intelligent backtracking:** Backtrack to a variable that was responsible for making one of the possible values of the next variable (e.g. SA) impossible.

Conflict set for a variable: A set of assignments that are in conflict with some value for that variable.
(e.g. The set {Q=red, NSW=green, V=blue} is the conflict set for SA.)
Backjumping method: Backtracks to the most recent assignment in the conflict set.
(e.g. backjumping would jump over T and try a new value for V.)

Thank You